

C++ Introduction

- C++ is an **object-oriented programming** language.
- It is an extension to C programming.
- C++ is a **cross-platform language** that can be used to create **high-performance** applications. It runs on lots of platforms like (**windows, Linux, UNIX**).
- Most helpful for development of **new languages**.
- C++ was developed by **Bjarne Stroustrup** in **1979** at **Bell laboratories**.
- C++ is generally used to create **high-level-performance** applications and provides better **control of memory** and **system resources**.
- C++ is a programming language that is the foundation of many modern **technologies** like **game engines, web browsers, operating systems, financial systems**, etc.,.
- C++ is used to create **computer programs**, and is one of the most used languages in **game development, compilers, Engineering applications , medical apps**.
- It helps to design **web browsers**.
- It also supports different ways of programming like **procedural (c – language concepts), object oriented concept**.
- It is **powerful** and **flexible** and also **case sensitive**.



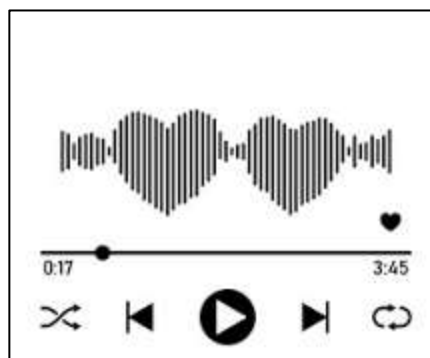
What is software?

- Software is a collection of **programs, digitalized** and **automated** process.

Program?

- A set of **instructions** that tells a computer how to **perform specific tasks**.

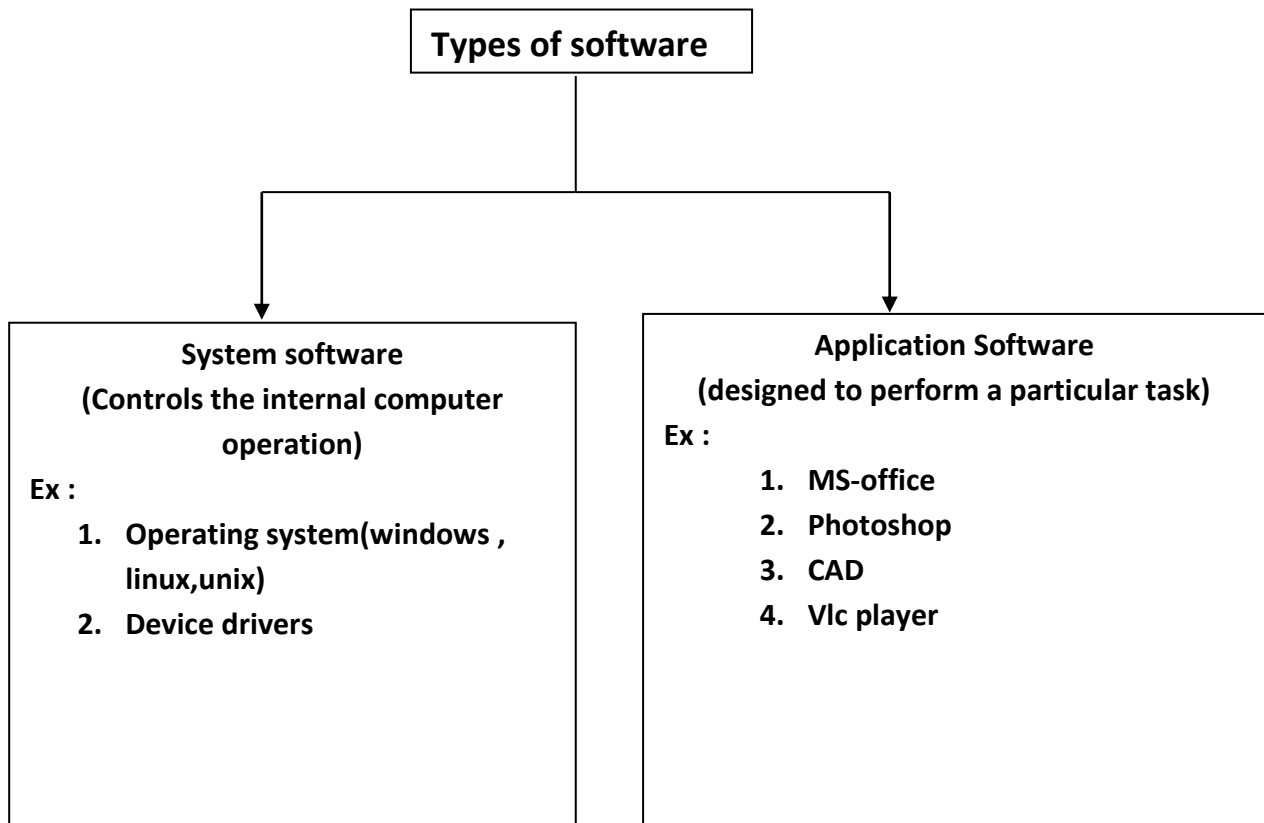
Example for software



- In above picture is a example of software
- This software consist of **play,pause,next,previous** buttons.
- We can play the song with the help of **play** button
- We can also pause the song while playing with the help of **pause** button

- we can also move the next song with the help of **next** button
- These types of **play, pause, next** buttons is also known as **programs**.
- When we click on **pause** buttons, the software shows the **pause** time on the software screen is called as **digitalized**.
- When the songs **getting** ends the next song also will start **automatically** is called **Automated**.
- That's why we are calling the software is called as collection of **programs, digitalized, and automated**.

Types of software



WHAT IS TRANSLATOR?

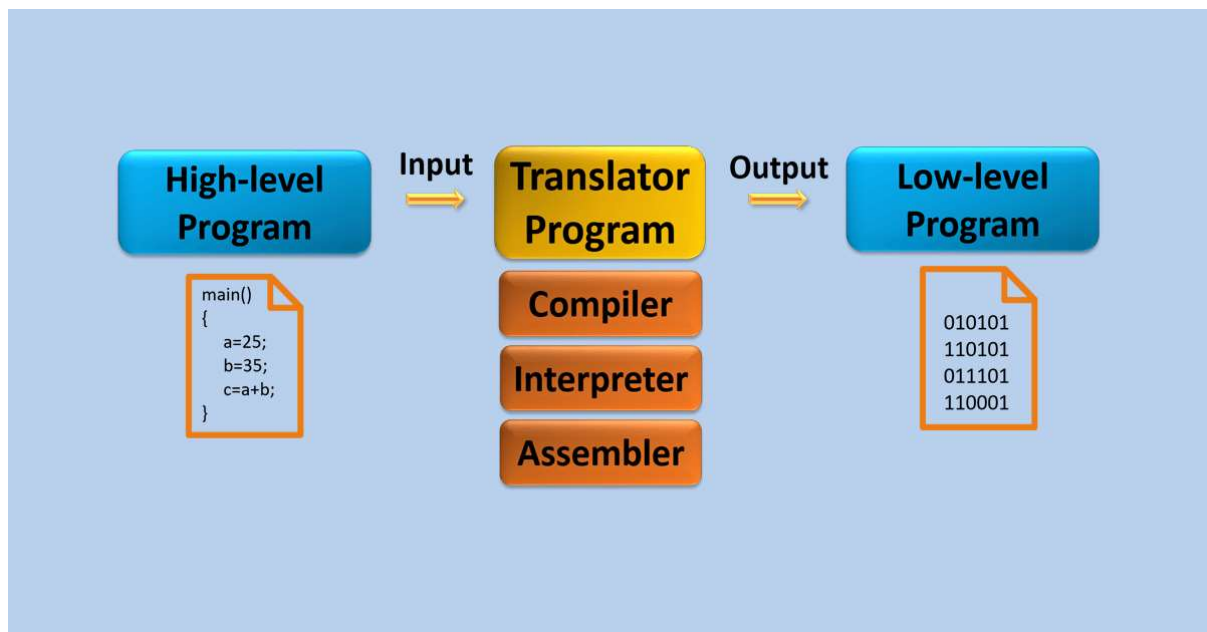
- A translator in a computer is a program that converts **source code** into **machine code** so that a **computer can understand** and **execute the instructions**.

WHAT IS SOURCE CODE?

- **Source code** is a set of **instructions written** by a **programmer** to **create a computer program**.

WHAT IS MACHINE CODE?

- **Machine code**, also known as **machine language** or **native code**, is a **low-level** programming language that instructs a computer's central processing unit (CPU) on how to execute programs.



- Translator converts **source code** into **Object code(Machine code)**

There are three types of Translators

1. Compiler
2. Interpreter
3. Assembler

What is compiler?

- A compiler is a computer program that **translates source code** written in a programming language into **machine code** or **another programming language**.
- C and C++ having only **compiler**.

WHAT IS INTERPRETER?

- An **interpreter** is a computer program that **executes instructions** in a programming language **without first compiling them into machine code**.
- It reads and executes the **code line by line**, translating each line **into machine instructions** before executing the next line.

Difference between Compiler and Interpreter

No	Compiler	Interpreter
1	Compiler Takes Entire program as input	Interpreter Takes Single instruction as input .
2	Intermediate Object Code is Generated	No Intermediate Object Code is Generated
3	Conditional Control Statements are Executes faster	Conditional Control Statements are Executes slower
4	Memory Requirement : More (Since Object Code is Generated)	Memory Requirement is Less
5	Program need not be compiled every time	Every time higher level program is converted into lower level program
6	Errors are displayed after entire program is checked	Errors are displayed for every instruction interpreted (if any)
7	Example : C Compiler	Example : BASIC

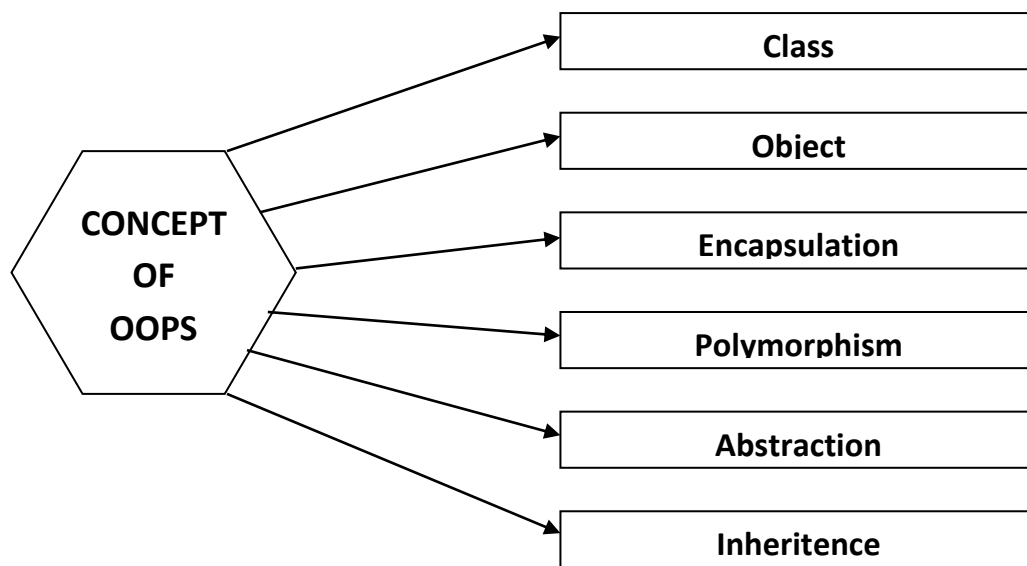
- Compiler and interpreter both are Translate **high level** language in to **machine language**.

WHAT IS ASSEMBLER?

- An assembler is a computer program that translates **assembly language** code into **machine code**.
- This allows a computer's hardware to communicate directly with the assembly language code.

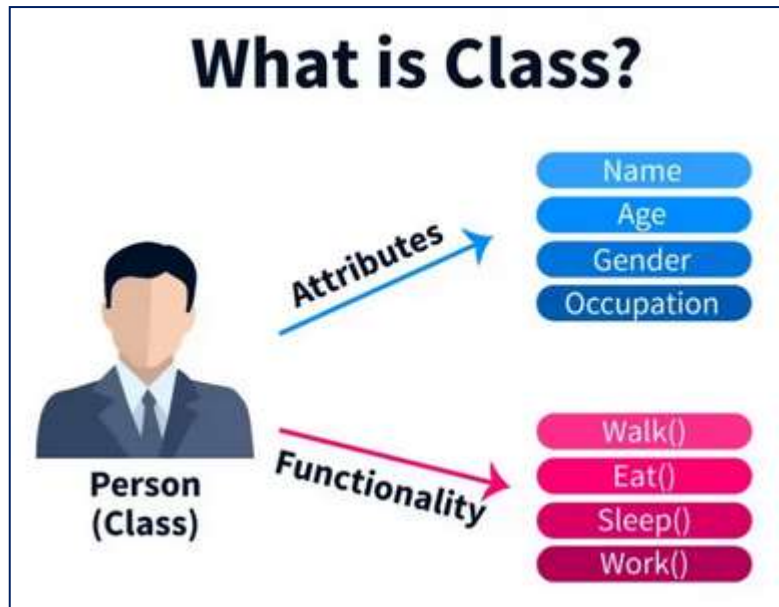
KEY OR BASIC CONCEPT OF OOP

- Object-oriented programming aims to implement **real-world entities** like **inheritance, hiding, polymorphism**, etc. in programming
- The main aim of **OOP** is to bind **together** the **data** and the **functions** that operate on them so that no other part of the code can access this data except that function.
- Object means a real word entity such as **pen, chair, table** etc.
- **Object-Oriented** Programming is a **methodology** or **paradigm** to design a program using **classes and objects**.
- It simplifies the software development and maintenance by providing some concepts



CLASS

- A class is **grouping of objects** that have the **identical properties, common behavior** and **shared relationship**.
- A class binds the **data** and its **related functions** together.
- A Class is a **user-defined** data type that has **data members** and **member functions**.
- **Data members** are the **data variables** and **member functions** are the functions used to **manipulate these variables together these data members** and member functions define the **properties and behavior** of the objects in a Class.
- The **Entire group of data** and **code of an object** can be built as a **user-defined** data type using class.
- Objects are nothing but **variable of type class**.
- Once a class has been declared, the programmer can create a **number of objects associated with that class**.
- A class is also called a **model**.
- Every object has its own value for each of its member variables.



OBJECT

- Objects are primary **run-time entities** in OOP.
- It is also instance of a class
- Every object has its **own properties** of features that illustrate what the **object can do**.
- It can be **singly recognized** by its name.
- It declares the state that is shown by the data values of its characteristic at a specific time. The state of the object varies according to the procedure used with it. It is called as the action of the object.
- **No memory is allocated** but when an object is defined, **memory is allocated**.
- The action of the object depends up on the member function defined within its class.

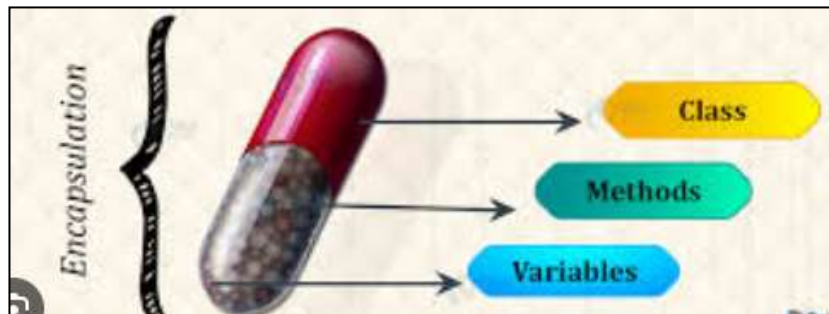
Object : signup
Data : First_Name_of_user Last_name_of_user Password Confirm_password Emai_id
Functions : register login

- The name of the object is **signup**. its data members are **fist_name_of_user**, **password** , **email_id**.

- The various functions associated with the **signup** are **register** and **login**.

ENCAPSULATION

- The packing of **data and functions** into a single component is known as encapsulation.
- It can be achieved by making the **data members' scope private** and **the member function's scope public** to access these data members.
- Encapsulation makes the data **non-accessible** to the **outside world**. The benefits of encapsulation include.



- **Data protection:** Encapsulation hides the internal implementation details of an object, protecting the data from unauthorized access or modification.

Ex: **When we turn on the Bluetooth we can assess only the shared content of the connected phone but not access to call or send message from that phone.** That is hidden from that unit and hence Encapsulation

- **Code Organization:** Encapsulation helps organize code by grouping related data and functions together within a class. This improves code readability and maintainability by providing a clear structure and reducing code clutter.
- **Code Reusability:** Encapsulation facilitates code reusability by encapsulating functionality within objects. Objects can be easily reused in different parts of the code or in different projects, promoting efficient development and reducing redundant code.

Real world Example of Encapsulation

Automatic teller machines (ATMs)

- ATMs are a class that takes in data from the user, such as their PIN and money amount, and displays options and icons. They also process the request.

Social networking websites

- Social networking websites, like Face book, make passwords private to keep accounts safe.

Medical capsules

- Medical capsules combine multiple types of medicine into one capsule

Large organizations

- Organizations are made up of several departments, such as sales, production, accounts, and purchase.

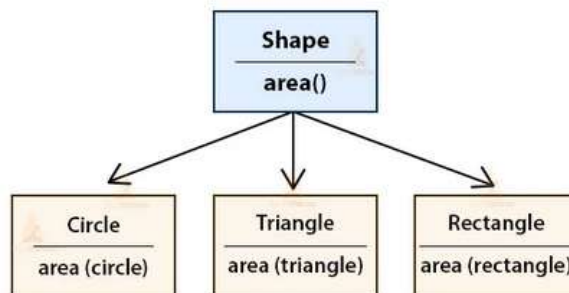
POLYMORPHISM

- Polymorphism makes it possible for the **same function** to act in a **different way** in **different classes**.
- It holds an ability to take more than one form.
- Polymorphism accomplishes an important part in allowing object of different classes to share the same external interface.

Example-1:

- **A person** at the same time can **have different characteristics**. **A man** at the same time is **a father, a husband, and an employee**. So the same **person possesses** different behavior in **different situations**. This is called polymorphism.

Example of Polymorphism in Java

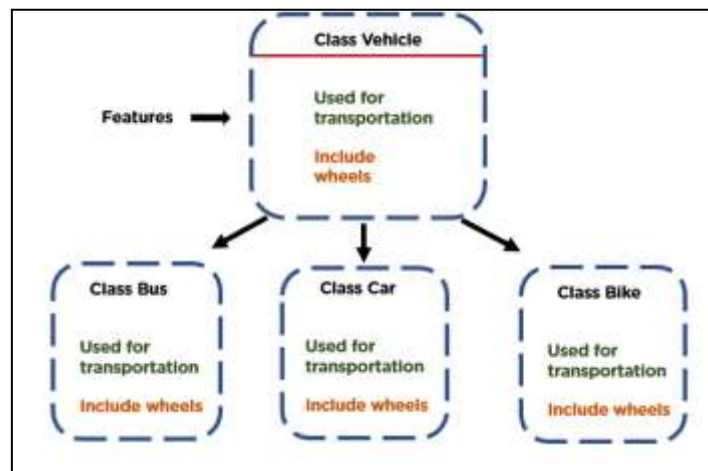


Example-2:

- In above example suppose we have to write a function to **area()** for all shapes , sometimes there **are 2 properties**, and **sometimes there are 3 properties** . We can write the **area ()** Method with the **same name** having **different formulas**; the concerned method will be called according to parameters.

INHERITANCE

- Inheritance is the method by which objects of one class get the properties of objects of another class.
- The class which inherits the features is known as the **child class**, and the class whose features it inherited is called the **parent class**. The **benefits** of Inheritance include.
 - **Code Reusability:** Inheritance allows classes to inherit properties and behavior from a base class, promoting code reuse. Derived classes can extend and specialize the functionality of the base class, eliminating the need to rewrite common code.
 - **Polymorphism:** Inheritance enables polymorphism, where objects of derived classes can be treated as objects of their base class. This allows for more flexible and dynamic programming, as different derived classes can be used interchangeably through a common interface.
 - **Hierarchical Organization:** Inheritance establishes a hierarchical relationship among classes, reflecting real-world or conceptual hierarchies. This improves code organization, understandability, and maintainability by representing relationships and dependencies in a structured manner.



- In above example, Class **Vehicle** is the **parent** class, and Class **Bus**, **Car**, and **Bike** are child classes and they are reusing the common feature (**include wheels**) this is called inheritance.

ABSTRACTION

- Abstraction means **displaying** only **essential information** and **hiding the details**.
- Data abstraction refers to providing only **essential information** about the data to the outside world(private variables), **hiding the background** details or **implementation**.
- Abstraction is the idea of using a simple interface to access functionality without needing to know the underlying details.

Here are some examples of abstraction in everyday life:

Driving a car

- You can drive a car **without needing** to know how the **engine works**. You only need to know how to turn the key or **press the start button**. What all components your car has. The car internal **implementation** and **complex logic** is completely **hidden** from the user.

Making coffee

- You only need to know how to **add water** and **coffee beans**, **turn on** the **coffee maker**, and select the type of coffee you want. You don't need to know how the **coffee maker** works internally.

Using a mobile phone

- You can make **calls**, **send** messages, or **browse** the internet without needing to know how **these functions work**.

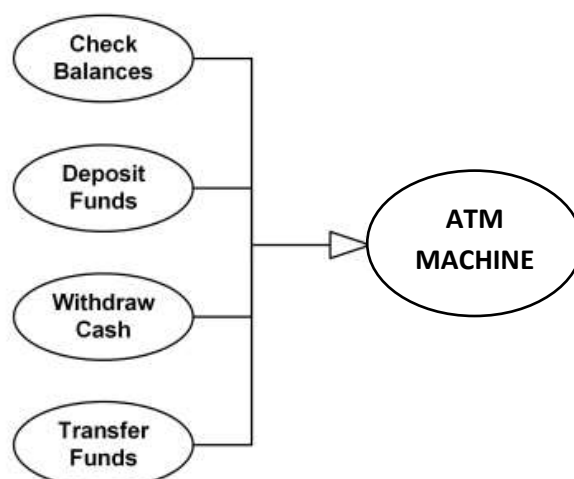
Taking a picture with a Smartphone

- You can use the camera app to switch between **front and rear-facing cameras**, apply filters, or blur the background. You don't need to see **how the camera works**.

Using an ATM

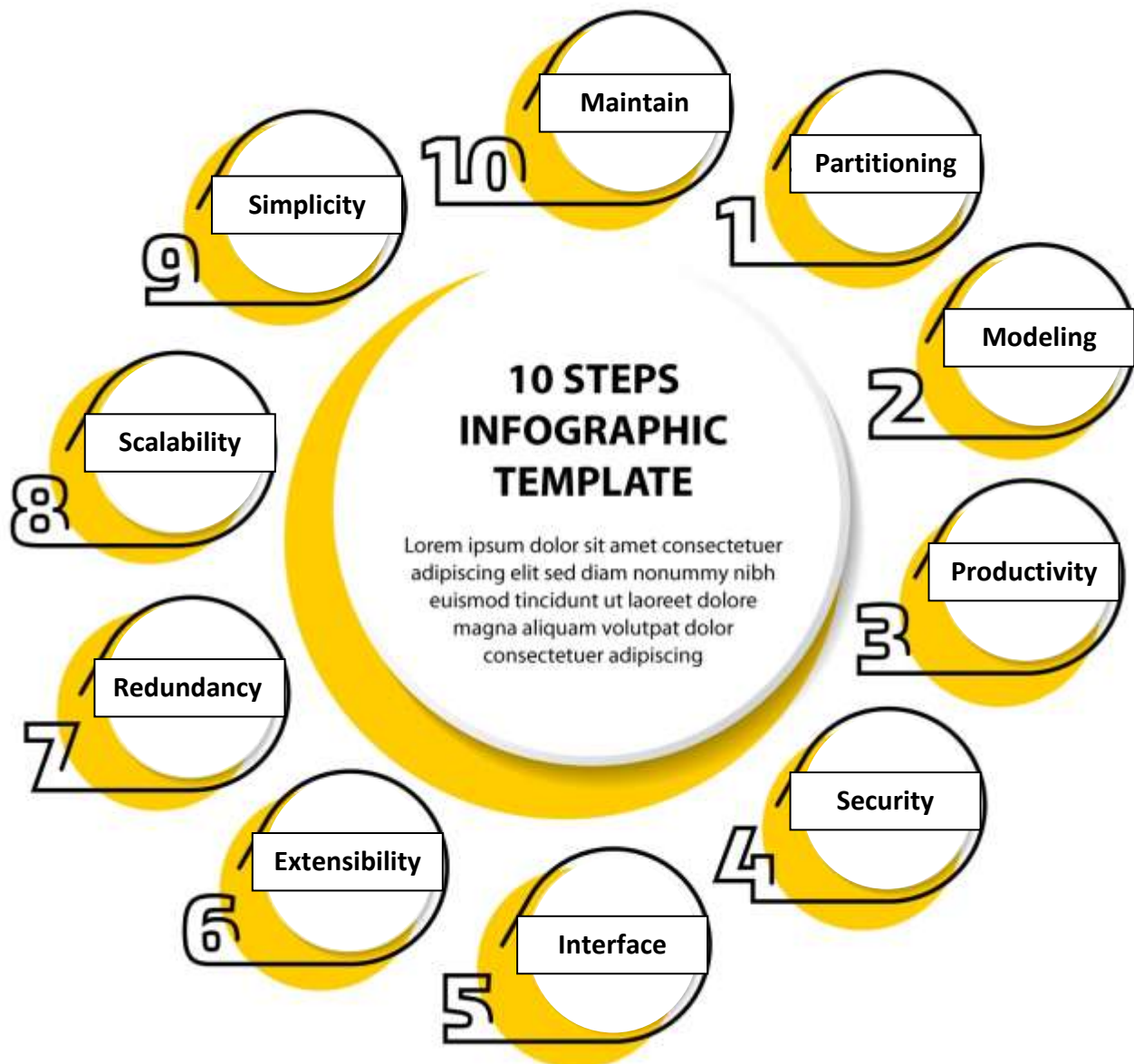
- You can perform functions like **withdrawing cash**, **depositing cash**, **checking your balance**, or **printing bills**. You don't need to see how the ATM performs these actions.

In programming, abstraction is used to **hide complex details behind** simple procedures. This makes it easier for other developers to use the code.



- In above example we all use an ATM machine for **cash withdrawal , cash deposit , money transfer , balance info , mini-statement,etc.,** in our daily life.
- But we don't know internally what things are happening inside **ATM** machine when you insert an ATM card for performing any kind of operation.

Advantages (or) Benefits of OOPS



1. Partitioning

- The objects in OOP have the **state** and **behavior** that is similar to the **real-world objects**. Program divided into objects it means a large program can divide into small, small objects. It is easy to partition the work in project based on object.

2. Modeling

- Objects represent **real-world objects**. In OOP easier to map real-world problem on to a solution.

3. Productivity

- Object can be developed. While developing object the **development time** will be reduced while development time **reduce the productivity** will increased with the help of OOP concept.

4. Security

- **Abstraction** techniques are used to hide the unnecessary details and focus is only on the relevant part.
- **Encapsulation** is also used to hide the information and to allow **strictly controlled** access to the structure as well as the behavior of the object. The principle of **data hiding** the programmer to build **secure programs**.

5. Interface

- **Message passing** techniques for **communication between** objects makes the interface **descriptions** with external systems much **simpler**.

6. Extensibility

- The concepts of OOP also enhance the **reusability** of the code written. Through the **inheritance**, we can eliminate code and extend the use of existing classes.

7. Redundancy

- **Duplication** of code is avoided by using inheritance.

8. Scalability

- OOP systems are **easier to modify**. OOP can be easily **upgraded** from the small to large system.

9. Simplicity

- Software **complexity can be better** managed. Flexibility of code using polymorphism.

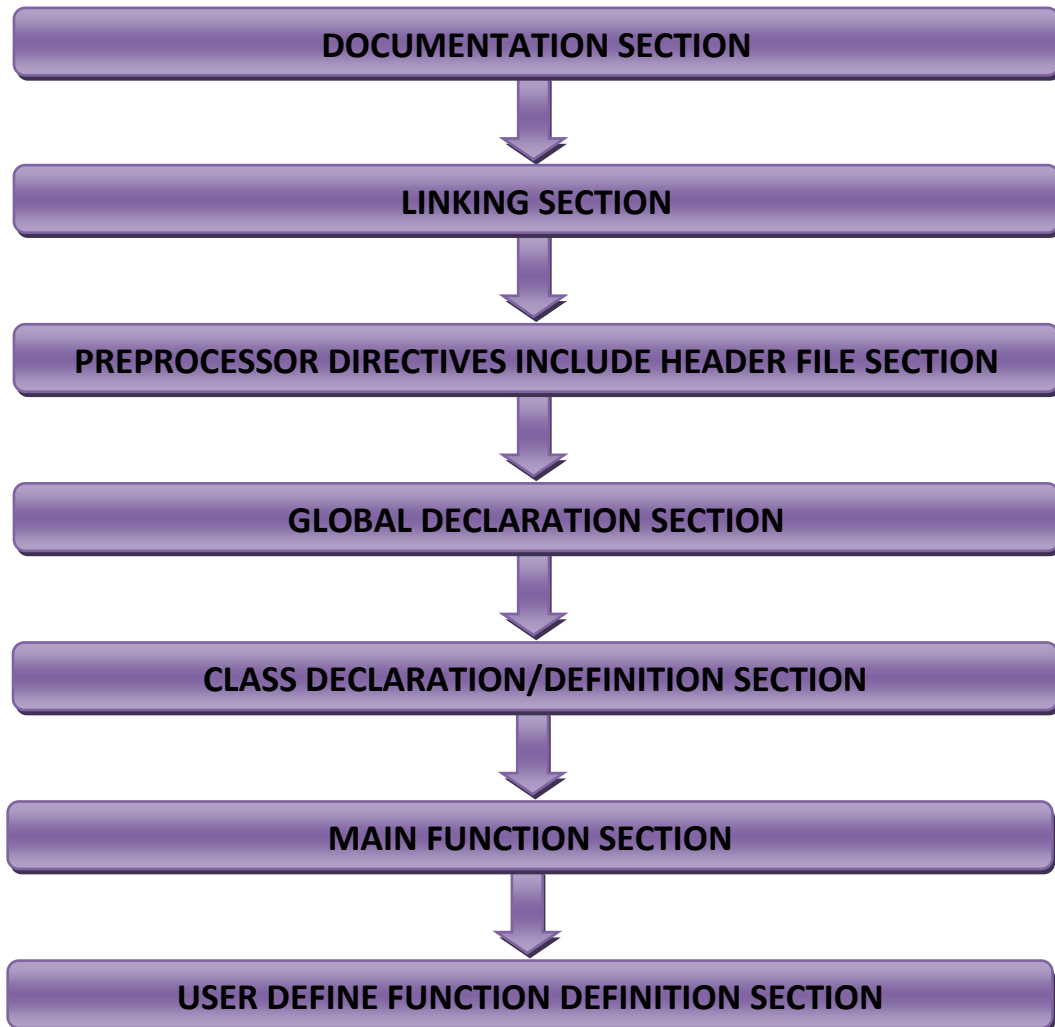
10. Maintainability

- OOP is more suitable for large projects and easy to maintain.

Difference Between C & C++		
S.no	C-language	C++ Language
1	C is a structural / Procedural / Functional oriented programming Language	C++ is a object oriented programming (OOPS) and also procedural oriented programming language
2	C is developed by donnish Ritchie in 1972	C++ is developed by Bjarne Stroustrup in 1980
3	C uses printf() and scanf() functions to write and read the data respectively	C++ uses cout and cin objects for output and input operations, The cout uses << (insertion operator) and cin uses >> (extraction operator).
4	C uses stdio.h for input/ouput functions	C++ uses iostream.h for input/output functions
5	Data is not secured because in c all the variables act as public by default .	Data is more secure because in c++ , we are using public,private,protected as access specifier.
6	C uses a top down approach .The program is prepared step by step in C.	C++ uses a bottom up approach . In C++ base elements are prepared first.
7	We can't use function in structure . We can only use data types and variable in a structure in C.	We can use data types and variables and functions in a class in C++.
8	We can't give the same name to function In a program , whereas due to the function overloading feature .	We can give the same name to two functions in a program . This concept is possible in C++. Once can initialize a number of functions with the same name but with different arguments . The polymorphism features in built in C++. Which support this concept
9	Constructor and Destructor are absent in C	Constructor and destructor are present in C++
10	Inline functions are absent in C.	Inline functions are supported in C++
11	C does n't support function overloading	C++ support function overloading
12	Operator overloading is not possible in C	Operator overloading is possible in C++
13	C does n't support for reference variables	C++ support reference variables
14	Function and data are write separately .	Function and data are written in a single unit . It supported for interface concept.
15	Inheritance is not supported in C	It support for inheritance concept.
16	Malloc(), calloc() to allocate memory	"new" operator to allocated memory
17	Allocated memory can be released using free()	Allocated memory can be released using "delete" function
18	Exception handling are not support	Exception handling is supported

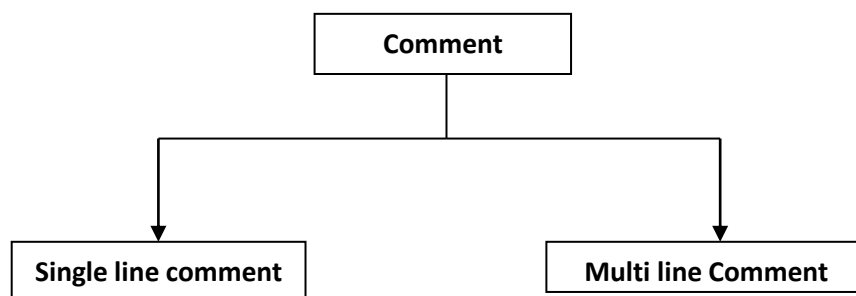
19	C program are divided into functions	C++ program are divided into class and object (or) functions .
20	C is a middle level language	C++ is a high level language

BASIC STRCUTURE OF CPP PROGRAM



1. Documentation Section

- We can write the **comment** in terms of text
- It is optional. It do not executed by the compiler.
- For this purpose a text can be written as **comment**.
- There are **2-types** of comments are available



Single line comment:

Single line comment represented by `//` .It write the comment in single line.

Multi line comment:

Multi line represented by `/* */`. It writes the comment in multiple lines.

Usage of comment line?

Write the name of programmer, time, date, description, meaning of keywords about the specified program. The comment line do not take it as source code in the program.

2. Linking Section

- All the header files should be included in this section.
- If you not link the header file in this section, the compiler will leads the error.

Example:

- The function `clrscr()` under comes by **conio.h (header file)**
- The functions `cin` and `cout` under comes by **iostream.h (header file)**
- The above both header file (**conio.h & iostream.h**) should be included in linking section.

3. Preprocessor directives include header file / Definition Section

- We can create the **constants**. We can use the **constants** in the program.
- Preprocessing refers to the first step compiling a c++ file from a source language in to machine language.
- The preprocessor directives are to be included at the beginning of the program before the **main ()** function.
- It begins with symbol **# (hash)**.

Example:

```
#define PI 3.14
```

4. Global Declaration Section

- We can create **global variables** and **user defined functions** are declared in this section.

5. Class Definition (or) Declaration Section

- Declaration of class is done in this section.
- In class **member functions** and **member variables** are defined in this section.
- Class definition part contains **definition of functions**. The definition of functions can also be written **outside the class** but before **main()** function.
- The **outside definition** of function should need **class name** and **scope** access operator before the **function name**.

6. Main Function Section:

- Like C , C++ programs also start with the **main()** function.
- The execution of every program starts with the **main()** function.
- Everything should be given in main function.

7. User Define Function definition Section:

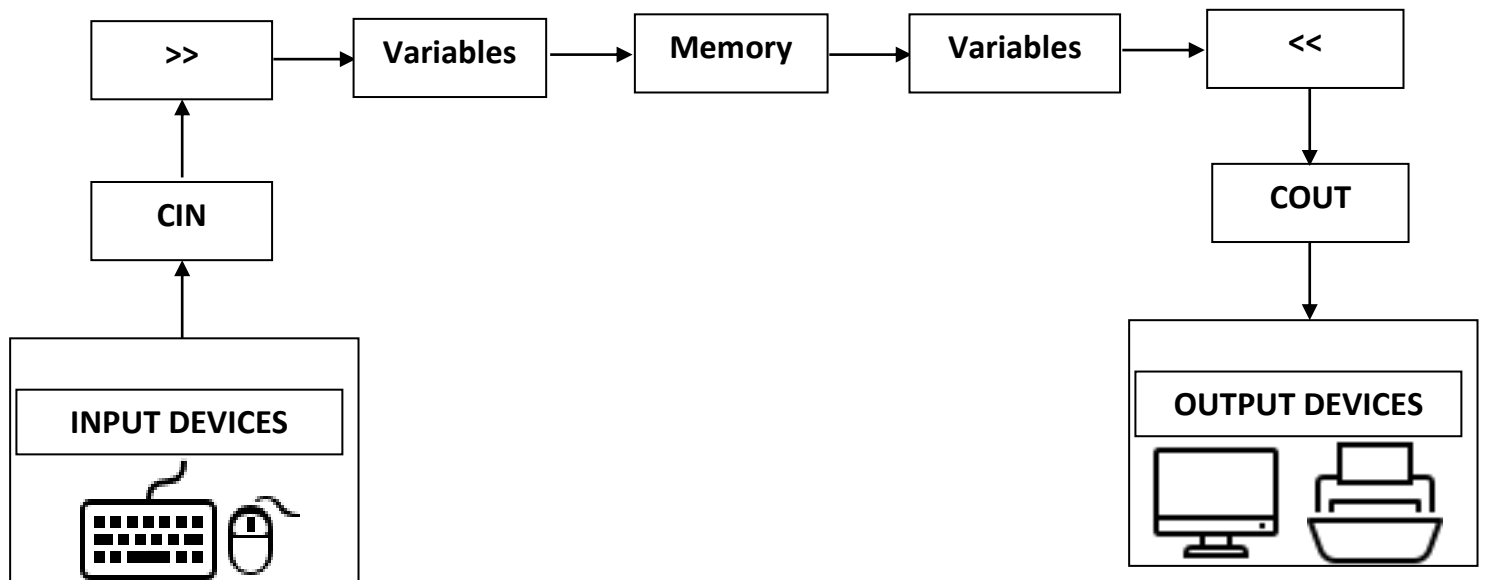
- Defines the **user-defined** function in this section

INPUT / OUTPUT IN C++

I/O Stream

- **Input stream** uses **cin** object to **read data** and **output stream** uses **cout** object to **display the data** on the screen.
- The **cin** and **cout** are **pre-defined** streams for **input** and **output** data.
- The data type is identified by these functions using operator overloading of the operators << - **insertion operator** and >> - **Extraction operator**
- The operator << is overloaded in “**ostream**” class and the operator >> is overloaded in “**istream**” class.

Working flow of cin and cout statements



INPUT STREAM

- The input stream reads operation through keyboard.
- It uses **cin** as object .The **cin** statement use >> (Extraction Operator) before a variable name
- The **cin** statement is used to read data through the input device.

Syntax :

cin>>variable_name;

Example :

```
int main()
{
    int v1;
    float v2;
    char v3;
    cin>>v1>>v2>>v3;
    return 0;
}
```

- Where **v1,v2 and v3** are variable names .
- The response of the user to this statement would be as shown below

Input Data:

2 5.4 A

- If user enter data in the manner **2 5.4 A**, then the operator will assign **2** to **v1**, **5.4** to **v2** and **A** to **v3**.
- If the entered data is greater than the variable.
- It remains in the input stream while entering **String, blank spaced** are not allowed.
- More than one variable can be used in **cin** statement to input data.
- The operator **>>** accepts the data and assigns it to the memory location of the variables.
- Each variables requires **>>** operator
- The enter data is separated by **space , tab (or) enter**.
- Similar to **scanf()** statement , **cin** does not requires control strings such as **%d** for integer and **%f** for float and **%c** for character,

OUTPUT STREAM

- The output stream manage output of the stream, That is display contents of variables on the screen.
- It uses **<<** - **insertion operator** before the variable name.
- It uses the **cout** object to perform console write operation.

Syntax :

cout<<variable;

Example :

cout<<v1<<v2<<v3;

- Where **v1,v2 and v3** are variables . The above statement displays the contents of these variables on the screen.
- Here the **cout** statement uses the **<<** (**insertion operator**).
- The **cout** statement does not use the format specification such as **%d,%f** as **%c** used in **C** .

- The **cout** statement allows us to use all “c” escape sequence such as “\n”, “\t” and etc.,

PRG1	PRG2
<pre>#include<iostream.h> int main() { cout<<" c plus plus "; return 0; }</pre> O/p C plus plus	<pre>#include<iostream.h> int main() { char n="CS and IT"; cout<<n; return 0; }</pre> o/p CS and IT

Example Programs for Input/output Stream

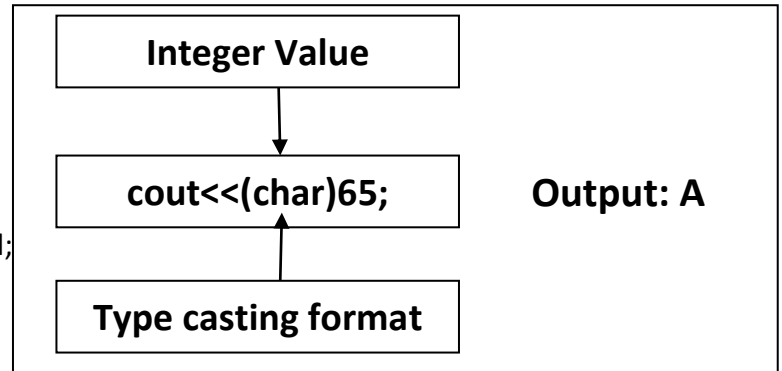
1. Write a c++ program to accept two integers through the keyboard and display it on the screen use **cin** and **cout** statements.
2. Write a c++ program to accept a string through the keyboard and display with welcome message on the screen.
3. Write a c++ program to display **int, float, char and string** using **cout** statement only .
4. Write a c++ program find area of the circle using float data type.
5. Write a c++ program find arithmetic operation using cin and cout statements.
6. Write a c++ program reverse a two digit number only using remainder(or) module operator.
7. Write a c+ program find 5% percentage of given grand total and less 5% amount from grand total using cin and cout statements.
8. Write a c++ program get **stu_id, stu_name, regno, dept, sec** and get 5 subjects marks for **tam, eng, mat, sci, ss** and find total marks and average marks using float data type.
9. Write a c++ program get personal details of employee and find **present, absent, total_ot, hra, pf, net_salary** using float data type.
10. Write a c++ program to get **your name, id, dept, section, year and mobile_no** using all data types.

TYPE CASTING

- Type casting refers to conversion data of one basic type to another by applying external use of data type keywords.

Example

```
#include<iostream.h>
int main()
{
int a=66;
float f=2.5;
double d=85.22;
char c='k';
cout<<"\n int in char format:"<<(char)a;
cout<<"\n float in int format:"<<(int)f;
cout<<"\n double in char format:"<<(char)d;
cout<<"\n char in int format:"<<(int)c;
return 0;
}
```



Output

int in char format: B

float in int format: 2

double in char format: U

char in int format: 75

Explanation

- In the above program , the variables of **int, float, double and char** type are declared and **initialized**.
- The variable **a** is initialized with **"65"** , **f** with **2.5** , **d** with **85.22** and **c** with **character 'k'**
- First **cout** statement convert integer value to the corresponding character according to the **ASCII** character set and the character **B** is displayed .
- Second **cout** statement converts **float** value to **integer**.
- The value displayed is **2** and not **2.5** when type cast format (**int**) is used the decimal portion of float value is removed and only the integer part is considered.
- Third **cout** statement , the double value is converted to character.
- The number **85.22** is converted to integer and then to character. The char data type is nothing else than **int** data type.
- The last statement convert character to int the value of **'k'** is **75**. When printed as an integer. The format (int) convert char to int.

Type casting format	Output	Conversion
cout<<(char)65 ;	A	int to char
cout<<(int) 'a' ;	65	char to int
cout<<(int)5.22;	5	float to int
cout<<(char)78.33	N	float to char

getline() and write() function

getline()- function

- These two functions are useful in **string input and output**.
- The **getline ()** function to read the string including white space.
- The **cin** statement does not allow to enter the string with blank space.
- The input reading is terminated when a user press the **enter** key .
- The new line character is accepted but not saved and replaced with the **null** character

Syntax:

cin.getline(variable, size);

- Where the variable names maybe a character type and **size** it the size of the character arrays.

write () – function

- The write() function is used to displaying the string on the screen. Its format is similar to **getline()** function.

Syntax:

cout.write(variable,size);

- Where the variable name maybe any character type and size is the size of the character arrays.
- **cout.write()** function display only a specified number of characters given in the second argument, through the actual string maybe more in length.

Example:

```
int main()
{
cout.write("india",6);
cout.write("is",3);
cout.write("great",6);
cout.write("country",7);
return 0;
}
```

OUTPUT

India is great country

- In this program the first statement displays **"india"** followed by one blank space. This is because the argument value is greater by one than the actual string length.
- Similarly the second statement displays **"is"** followed by one blank space. The reason same as above.
- Third statement displays **"greate"** followed by one blank space. The reason is same.
- The last statement displays **"country"**. Here the argument value and the string lengths are same no blank space are displayed.

- In case the argument value is less than the actual string length , the complete string will not be displayed.
- The number of character of the string displayed depends on the value of the given arguments.

getline() – function

- The getline() function reads the string through the keyboard .
- The getline() function accepts the string including spaces.

Example

```
int main()
{
char x[30],int nc;
cout<<" enter any string:";
cin.getline(x,30);
cout<<"\n actual string is:"<<x;
cout<<"\n enter no of character to display:";
cin>>nc;
cout.write(x,nc);
return 0;
}
```

OUTPUT

Enter any string: Indian cricketers

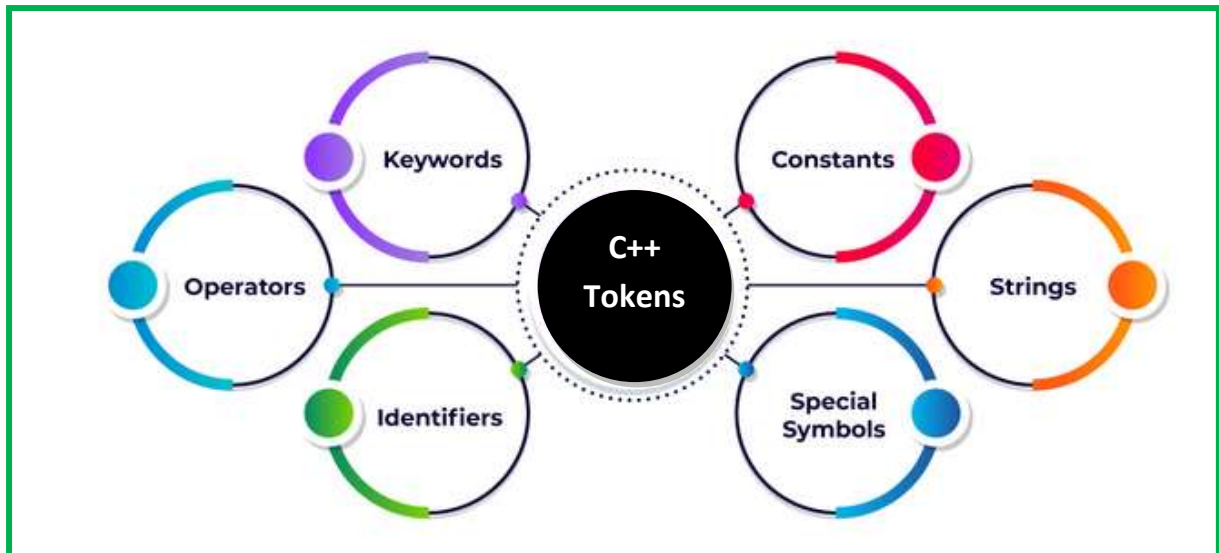
Enter no of character to display: 6

Indian

- The **cout** statement display the string including white space
- The **cin** statement cannot accept a **string including white space**. It accepts only a **single word**.
- The cout statement display the string read through **cin** and **getlines()** functions that is, It can display the **string with(or) without blank space**
- The **write()** function displays the string according to the specified size . It displays the **string with or without blank space**.
- The **write ()** function **does not support** for **escape sequence**.

TOKENS

- The smallest individual unit in a program is known as “**token**”.
- Every word in a C++ source code can be considered a token.
- Tokens can be one of several things, including **keywords, identifiers, constants, operators, string and punctuation marks**.



Keywords

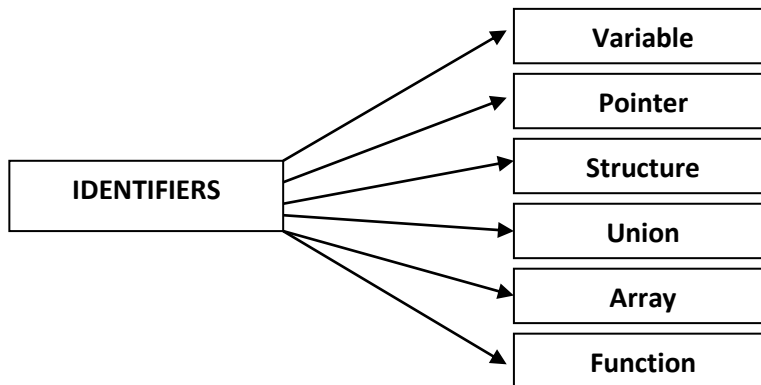
- The keywords are reserved set of words with fixed meanings.
- It cannot be used as identifier (or) variable.
- It is used to perform specific tasks or operations.

There are **95** keywords reserved in C++. Some of the main Keywords are:

break	try	catch	char	class	const	continue
default	delete	auto	else	friend	for	float
long	new	operator	private	protected	public	return
short	sizeof	static	this	typedef	enum	throw
mutable	struct	case	register	switch	and	or
namespace	static_cast	goto	not	xor	bool	do
double	int	unsigned	void	virtual	union	While
Inline	template					

Identifier(or) variables

- The variables are used to hold data temporarily.
- Identifiers are used to refer to these entities in the program, and they can consist of **letters, digits, and underscores**.
- Identifier always starts with an **alphabet**.
- Identifier does not allow blank spaces , punctuations, signs , etc.,
- Identifiers are user-defined and are generally defined in **lower case letters**. However , the upper case letters are also permitted.
- The underscore _ symbol can be used as an identifier.
- Identifiers are names of **variables , functions , arrays , etc.,**
- In other words identifier refers to variety of entities such as **structure , union,enumeration,constants , typedef names ,functions and objects**.



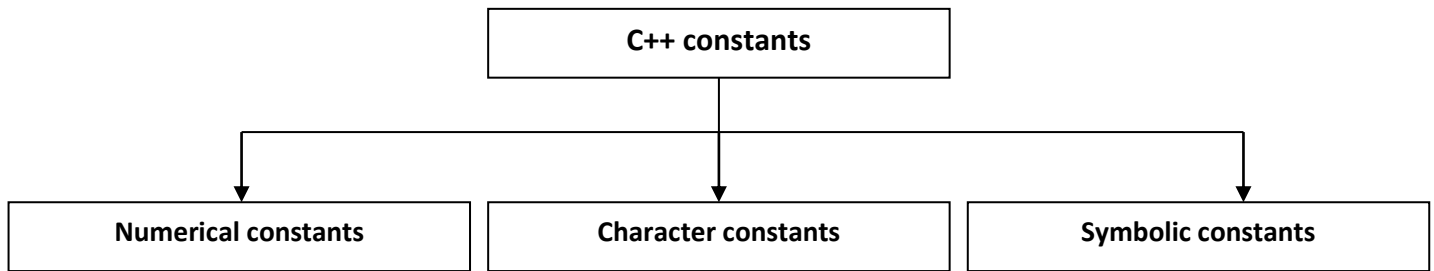
Rules for defining identifiers are follows:

- The first character must be a **letter or an underscore**.
- **Identifiers cannot** be the **same as a keyword**.
- Identifiers **cannot contain any spaces** or **special characters** except for the underscore.
- Identifiers are **case-sensitive**, meaning that a variable named "**myVariable**" is different from a variable named "**myvariable**".

Valid Identifiers	Invalid Identifiers
_name	#name (Cannot start with special character except ' _ ')
Number89	2num (Cannot start with a digit)
first_name	first name (Cannot include space)
_last_name_	string (Cannot be same as a keyword)

Constants

- The constants in **c++** are applicable to the values that do not change during execution of a program.
- Constants often **represent fixed values** used frequently in the code, such as the value of **pi** or **the maximum size of an array**.
- To define a **constant** in C++, you use the **const** keyword followed by the **data type** and the **constant's name** and **then initialize it with a value**.
- The value of a **constant cannot be changed** once it **has been defined**.



1. Symbolic constants

- A symbolic constants is defined in the same way as a variable. However, once the constant is initialized, the assigned value cannot be altered.
- We can define the symbolic constants in C++ in three ways.
 1. Using the '**const**' keyword
 2. Using '**#define**' preprocessor directive.
 3. Using "**enum**" keyword

Defining constant using the "const" keyword in C++

- Here is an example:
const double PI = 3.14159;
- In this example, the **constant PI** is of type **double** and is **initialized** with the value of **pi**. You can then use the constant **PI** in your code wherever you need to use the **value of pi**.
- Using constants **instead of regular variables** is important when you need to represent **fixed values** in your code.

Define Constants using the '#define' preprocessor directive in C++

- **#define** preprocessor directive can be used for defining constant.
- The symbol defined using **#define** are called **macros**.
- Here is an example

#define MAX 100

2. Numerical Constants

2.1: Integer constants

- Integer constants are represented with whole numbers.
- It requires minimum two bytes and maximum four bytes of memory.
- The numerical constants are represented with numerical numbers. At least one digit is needed for representing the numerical number.
- The decimal points or fractional part or any other symbols are not permitted. Neither even blank spaces nor commas are permitted.
- Integer constant could be either positive or may be negative zero.

Example:

Valid example: 10,20,+30,-15,etc,.

Invalid example: 2.3,.235,\$76

2.2: Real constants

- Real constants are often known as floating point constants and can be represented in exponential or fractional form.
- Integer constants are unfit to represent many quantities.
- Many parameters or quantities are defined not only in integer but it also in **real number**.

Example:

Length,height,prize

Example for real constants:

2.5 , 5.521 , 3.14

3. Character constants

3.1: single character constants

- A character constants in c++ represented with a single character and it is enclosed within single quotes.
- Each character constant has a particular integer value associated it.
- Each character is represented with different **ASCII** value.
- A character can also be represented with single digit or single special symbol or white space, all enclosed within a pair of single quote marks.

Example:

'a' , '8' , '&'

3.2: String Constants

- String constants are sequence of characters enclosed within **double** quote marks.

- The string maybe may be combination of all types of symbols.

Example: "hello" , "india","444"

OPERATORS

- An operator is a symbol that operates on a value to perform specific mathematical or logical computations
- An operator operates the operands.

For example,

int c = a + b;

- Here, '+' is the addition operator. 'a' and 'b' are the operands that are being 'added'.
- The new operators are in C++
 - >> (insertion operator)
 - << (Extraction operator)
 - :: (Scope access (or resolution) operator)
 - ::* (pointer to member decelerator)
 - ->* (deference pointers to pointers to class members)
 - .* (deference pointers to class members)
 - Delete (memory release operator)
 - New (memory allocation operator)

STRING

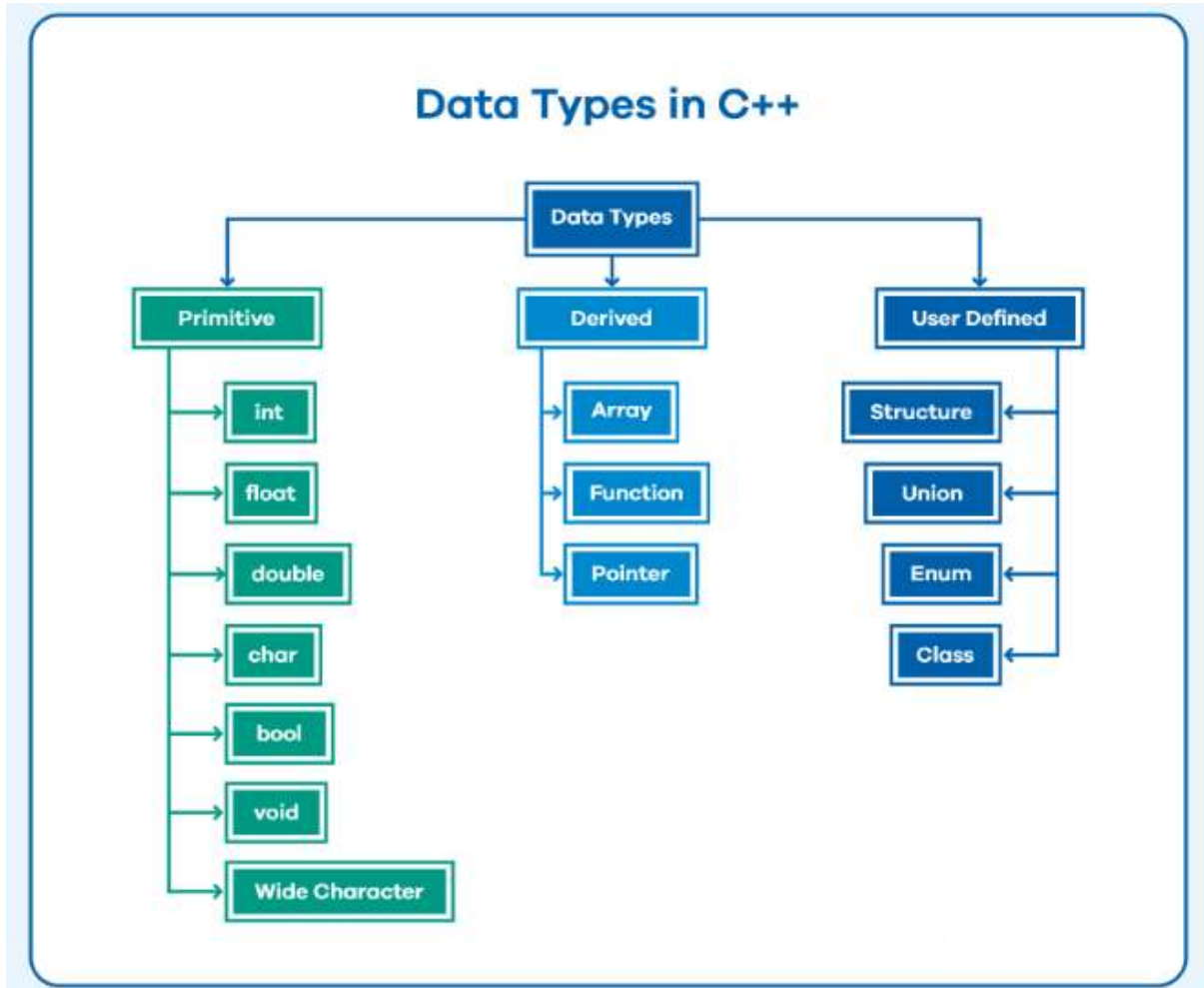
- String represented by an array of characters. String constants are enclosed in double quotes.
- Following are the valid examples of string constants.

Example

"person" , Tamilnadu" , "india"

DATA TYPES

- Data are collection of characters , digits , symbols , etc.,
- The data are used to represent information. The data are classified into various types.
- It can be classified as **primitive data type**, **derived data type**, and **user defined data type**.



1. Primitive Data Types:

- These data types are **built-in or predefined data types** and can be used directly by the user to declare variables.

Example: int, char, float, bool, etc.

Primitive data types available in C++ are:

- Integer
- Character
- Boolean
- Floating Point
- Double Floating Point
- Valueless or Void

- **Integer:**

- The keyword used for integer data types is int.
- Integers typically require 4 bytes of memory space and range from -**2147483648** to **2147483647**.

- **Character:**

- Character data type is used for storing characters. The keyword used for the character data type is char.
- Characters typically require **1 byte** of memory space and range from **-128 to 127 or 0 to 255**.

- **Boolean:**

- Boolean data type is used for storing Boolean or logical values. A Boolean variable can store **either true or false**.
- The keyword used for the Boolean data type is bool.

- **Floating Point:**

- Floating Point data type is used for storing single-precision floating-point values or decimal values.
- The keyword used for the floating-point data type is float. Float variables typically require **4 bytes** of memory space.

- **Double Floating Point:**

- Double Floating Point data type is used for storing double-precision floating-point values or decimal values.
- The keyword used for the double floating-point data type is double. Double variables typically require **8 bytes** of memory space.

- **void:**

- Void means without any value.
- void data type represents a valueless entity.
- A void data type is used for those function which does not return a value.

- **sizeof () operator:**
 - [sizeof\(\) operator](#) is used to find the number of bytes occupied by a variable/data type in computer memory.

Data type	Size(bytes)	Range
char	1	-128 to 127
unsigned char	1	0 to 255
short	2	-32,768 to 32,767
unsigned short	2	0 to 65535
int	4	-2147483648 to +2147483647
unsigned int	4	0 to 4294967295
long	4	-2147483648 to +2147483647
Unsigned long	4	0 to 4294967295
float	4	-3.4e-38 to +3.4e-38
double	8	1.7 e-308 to 1.7 e+308
long double	8	1.7 e-308 to 1.7 e+308
bool	1 bit	
void	-	-
wchar_t	2 or 4	1 wide character

2. Derived Data type

- The derived data types are
 - Pointers
 - Functions
 - Arrays
 - References
- **Pointers**
 - A pointer is a memory variables that stores a memory address.
 - Pointer cab have any name that is legal for other variable and I is declared in the same fashion like other variable but it is always denoted by “*” operators
 - **Example : int *x ; float *y ; char *y;**
 - In the first statement ‘x’ is an integer pointer and it tells the compiler that it holds address of any integer variable.
 - In the same way ‘f’ is a float pointer that stores the address of any float variable and ‘y’ is a character pointer that stores the address of any character variable.
- **Arrays**
 - Array is a collection of elements of similar data type in which each element in separate memory location
 - **Example : int b[4];**
 - The above statement declares an array b[] which can hold four integer values

- **Function**

- A function is a self-contained block or a sub-program of one or more statements that perform a special task when called.
- It is possible to use the same name with multiple definition known as function overloading.
- **Example : void personal() ; void display()**

3. User defined Data types

- User defined data types are

- **Structure**
- **Classes**
- **Union**
- **Enumerated**

- **Structure**

- Structure is a combination of variables with different data types into a single record.
- It constrains variables and data types
- It saved (or) keep the data in separate memory.
- **Example :**

```
struct my_friend
{
    char fname[30], phone[20];
    int age;
    float height;
}friend;
```

- **Union**

- A union same as compared to a structure , but the only difference is that is allows the user to declare variable that share the same memory space.

- **Example :**

```
union charorint
{
    char c;
    int i;
}num;
```

- **Class**

- Class is a group of objects
- It is consist of member data and member function
- Each member function having a own behavior .

- **Example :**

```
Class staff
{
```

```
Public:
Int st_id;
Char sname[30];
Void personal()
{
St_id=100;
Sname="Dev";
}
}hm;
```

- **Enumerate**

- It is used for declaring **enumeration** data types.
- The programmer can declare new data type and define the variable of these data types that can hold.
- **Example :**
enum logical { false , true };

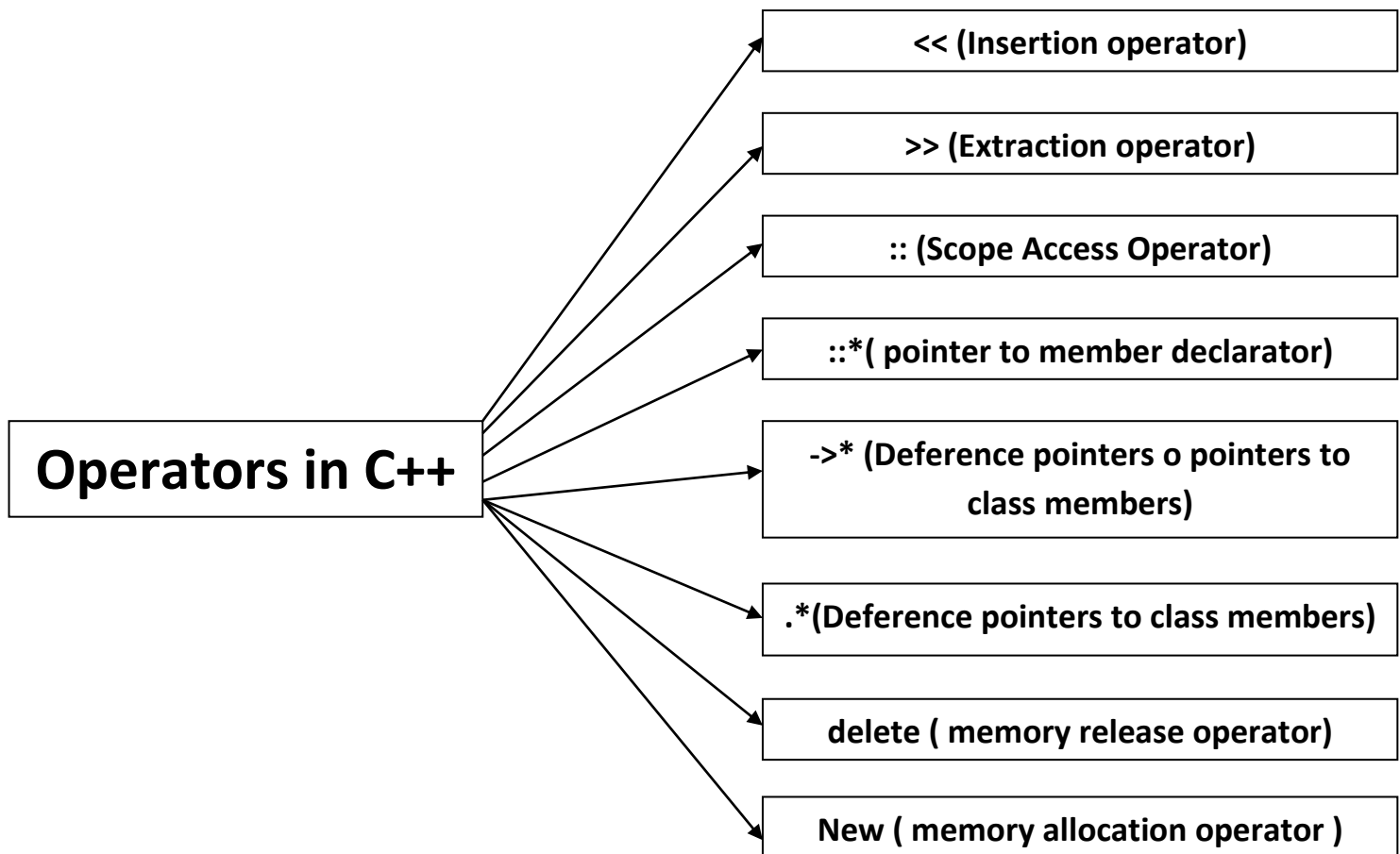
- **Operators in C++**

- Operators are used to perform operations on variables and values.
- An operator operates the operands.

For example,

```
int c = a + b;
```

- Here, '+' is the addition operator. 'a' and 'b' are the operands that are being 'added'.



1. Insertion Operator

- In C++, the **insertion operator** (<<) is typically used with output streams (such as std::cout) to send data to the output. It is commonly used for printing or displaying values.

2. Extraction operator

- In C++, the **extraction operator** (>>) is primarily used with input streams (such as std::cin) to read data from the input. It extracts data from the input stream and stores it in a variable.

Example :

```
int main()
{
int a;
cout<<"Enter the value of a : ";
cin>>a;
cout << "The value of a is: " << a <<;
return 0;
}
```

Output :

Enter the value of a : 100

The value of a is :100

3. Scope access operator

- The **scope access(or resolution)** operator :: (two colons) allows programmer to access a global (or file duration) name even if it is hidden by a local re-declaration of that name.
- We can use the same variable names in the **c++** program in separate blocks.
- The two declarations of the same variable refer different memory locations
- When we declare a variable , it is available only to specific part or block of the program.
- Remaining block or other functions cannot access the variables.
- The area or block of the **c++** program from where the variable can be accessed is known as the **scope of variables** .

Example :

```
int a=10;
int main()
{
int a=20;
cout<<" local variable a is : "<<a;
cout<<" global variable a is: "<<::a;
return 0;
}
```

OUTPUT:

Local variable a is : 20

Global variable a is:10

4. ::* (pointer-to-member declarator)

- The **::* (pointer-to-member declarator)** in C++ is used to declare a pointer to a **member function** or **member variable** of a class.
- It allows accessing class members (data members or member functions) indirectly through an object or pointer to an object.

```
class ClassName
{
// Member variable
MemberType ClassName::* pointer_to_member_variable;
// Member function
ReturnType (ClassName::*pointer_to_member_function)(Parameters);
};
```

5. ->* (Deference pointers o pointers to class members)

- The **->*** operator in C++ is used to dereference a pointer to a pointer-to-member in a class.
- It allows you to access a member (variable or function) of a class object via both:

A pointer to an object, and

A pointer to a member of the class.

- This is typically used in advanced object-oriented scenarios where you need to dynamically access members of a class.

(object_pointer->*pointer_to_member)

6. .*(Deference pointers to class members)

- The **.*** operator in C++ is used to dereference a pointer to a class member (either a variable or a function) when you are accessing the member through an object directly (not a pointer to an object).
- It is paired with the **::*** operator, which is used to declare and initialize pointers to class members.

(object.*pointer_to_member)

7. new

- The **new** operator in C++ is a memory allocation operator used to dynamically allocate memory for an object or an array during runtime.
- It returns a pointer to the allocated memory and calls the constructor if the object is a class type.

Type* pointer_name = new Type[array_size];

8. Delete

- The delete operator in C++ is a memory release operator used to deallocate memory that was previously allocated using the new operator.
- It ensures that the memory is returned to the heap and can be reused by the program. Using delete prevents memory leaks caused by unreleased dynamic memory.

delete[] pointer_name;

DECISION MAKING (or) CONTROL STRUCTURE

- Decision making statements in a program language help the programmer to transfer the control from one part of the program to another part.
- Thus, these decision-making statements enable the programmer to determine the flow of control
- One basic of applications it is essential to
 - **Alter the flow of a program**
 - **Test the logical conditions and**
 - **Control the flow of execution as per the selection**
- These conditions can be placed in the program using decision making statements.

Decision Statements



SIMPLE IF

- The **if** statements is the simplest one.
- C++ uses the keyword **if** to execute a set of command lines or a command line when the logical condition is true.
- It has only one option.

Syntax

```
if (expression)
{
    statement-1;
    statement-2;
    .....
    .....
}
Statement-3;
```

- The is statement contains an expression.
- The expression is always enclosed within a pair of parentheses.
- The condition statement should not be terminated with semicolons(;;)
- In above syntax , if the expression is evaluated , if the expression is true , the **statement-1 and statement-2** will execute otherwise it moves for **statement-3**

Example :

```
int main()
{
    int std=10;
    if(std==10)
    {
        cout<<"\n welcome to sslc";
    }
    cout<<"\n end";
    return 0;
}
```

OUTPUT:

Welcome to SSLC

END

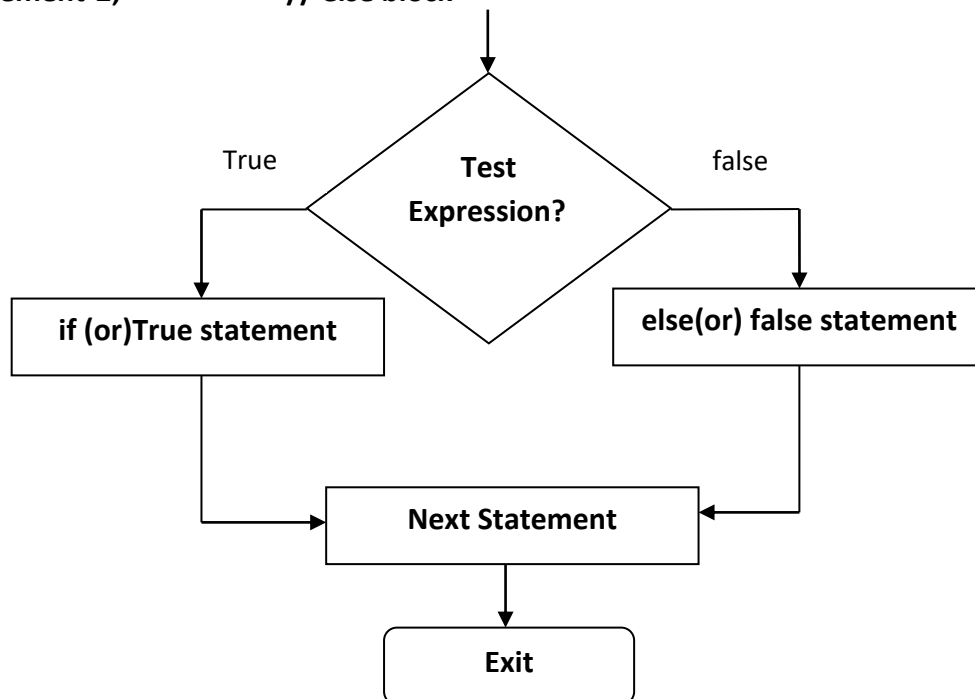
- In above example we have assign value **std=10**
- Then **if(std==10)** is evaluated first
- Here the if condition become true so we got the message(**welcome to SSLC**)

IF_ELSE_STATEMENT

- In the **if_else** statement , if the expression/condition is true, the body of the **if** statement is executed; otherwise , the body of the **else** statement is executed.
- The **else** keyword is used when the expression is not true.

Syntax :

```
if(expression_is_true)
{
Statement-1;           // if block
}
else
{
Statement-2;           // else block
}
```



Explanation:

- in above flow chart , if the given **test expression** is evaluated first , if the **test expression is true means** , the **if(or) true block will be executed**
- If the given **test expression** is false means, the **else (or) false statement** will be executed.
- After execute **if (or) else** block , the **next statement will be executed** finally

```

#include<iostream.h>
#include<conio.h>
int main()
{
clrscr();
int no=5;
if(no>0)
{
cout<<"\n given number is positive number";
}
else
{
cout<<"\n given number is negative number";
}
cout<<"thanks for using";
return 0;
}

```

OUTPUT

Given number is positive number

Thanks for using

Explanation

- In above example if the given condition (**no>0**) is true means the **if** block i.e(**given number is positive**) is executed .
- If the given condition false means **else** block i.e (**given number is negative**) is executed
- After executed both if and else block the next statement i.e (**thanks for using**) is executed finally

Example program for IF else:

1. Write a c++ program find greatest of two numbers using IF_Else
2. Write a c++ program find the given number is odd or even number using If_Else
3. Write a c++ program find the given letter is vowels or consonant using If_Else
4. Write a c++ program find the given user is eligible vote or not using If_Else

Nested If_Else

- In this kind of statement , a number of logical conditions are tested for taking decisions.
- Here , the **if** keyword followed by an expression is evaluated.
- If it is true , the compiler executes the block following the **if** condition; otherwise , it keeps this block and executes the **else** block.

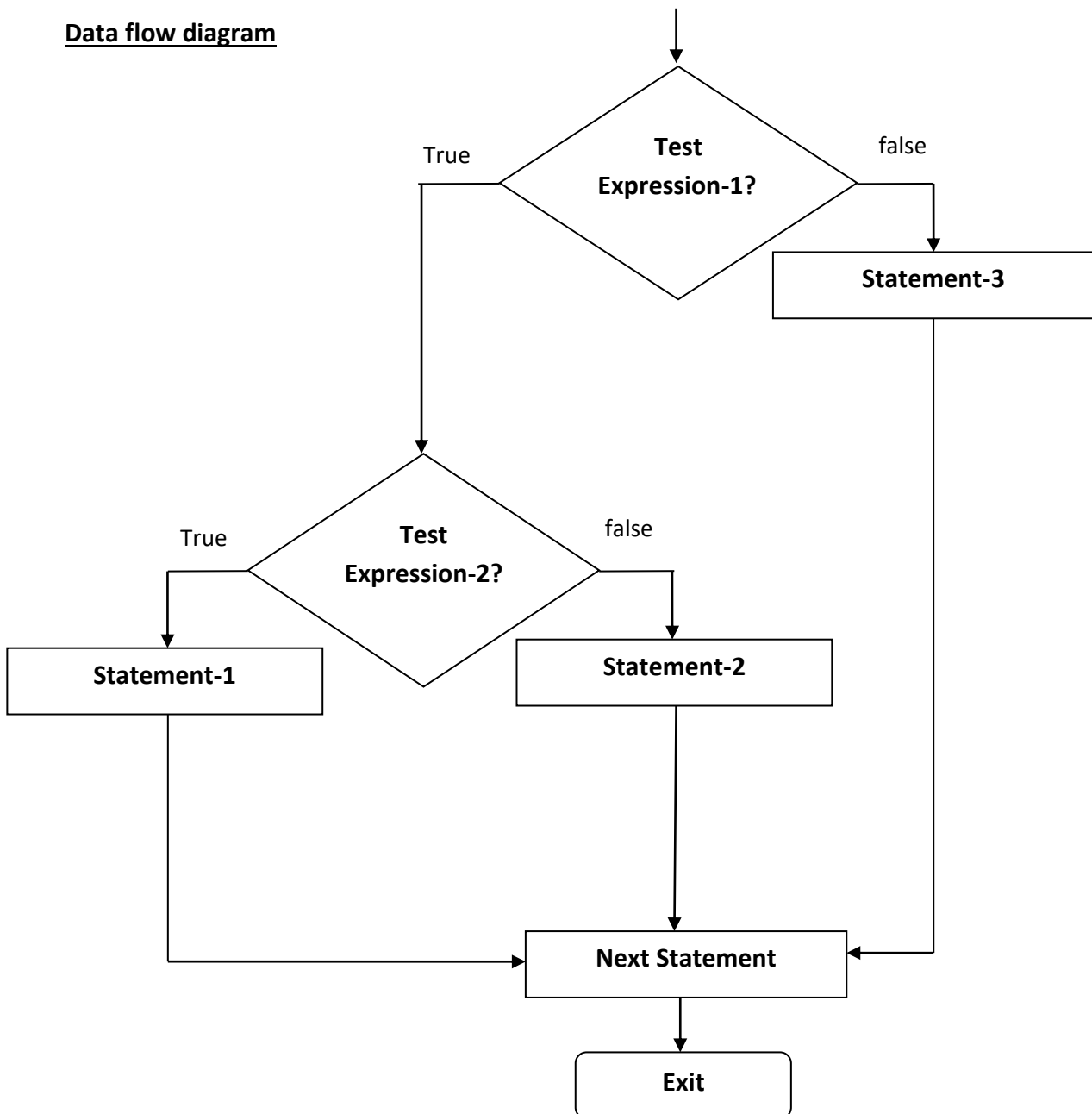
Usage of nested if else :

- It uses the **if** statement nested **inside** an **if-else** statement, which is nested inside another **if_else** statement.
- This kind of nesting can be limitless

Syntax :

```
if (expression-1)
{
    if (expression-2)
    {
        Statement-1;
    }
    else
    {
        Statement-2;
    }
}
else
{
    Statement-3;
}
next statement ;
```

Data flow diagram



- In above data flow diagram the **test expression-1** is evaluated **first**
- If **test expression-1** is **true** means the process will move to **test expression-2**
- If **test expression-2** is **true** means it execute the true statement (i.e: **statement-1**) will be executed
- If **test-expression-2** is false means it execute the false statement(i.e: **statement-2**) will be executed
- If **test expression-1** is false means it execute the false statement (i.e: **statement-3**) will be executed
- After execute all the statements finally it executes **next statement**.

Example program for Nested if else

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
void main()
{
clrscr();
char fn[30],sn[30],pwd[3],cpwd[30];
int ln;
cout<<"\n Enter first name:";
cin>>fn;
cout<<"\n Enter second name:";
cin>>sn;
cout<<"\n Enter pwd:";
cin>>pwd;
cout<<"\n Enter cpwd:";
cin>>cpwd;
ln=strlen(pwd);
cout<<"\n Total length is:"<<ln;
if(ln>=8)
{
cout<<"\n Password length is strong";
if(strcmp(pwd,cpwd)==0)
{
cout<<"\n Welcomes you";
}
else
{
cout<<"\n password not matched";
}
}
else
{
cout<<"\n password length is too weak";

}
getch();
}
```

OUTPUT

Enter first name : Sarah

Enter second name : Jacob

Enter password : Krishnan@90

Enter Confirm password : Krishnan@90

Password length is strong

Welcomes you

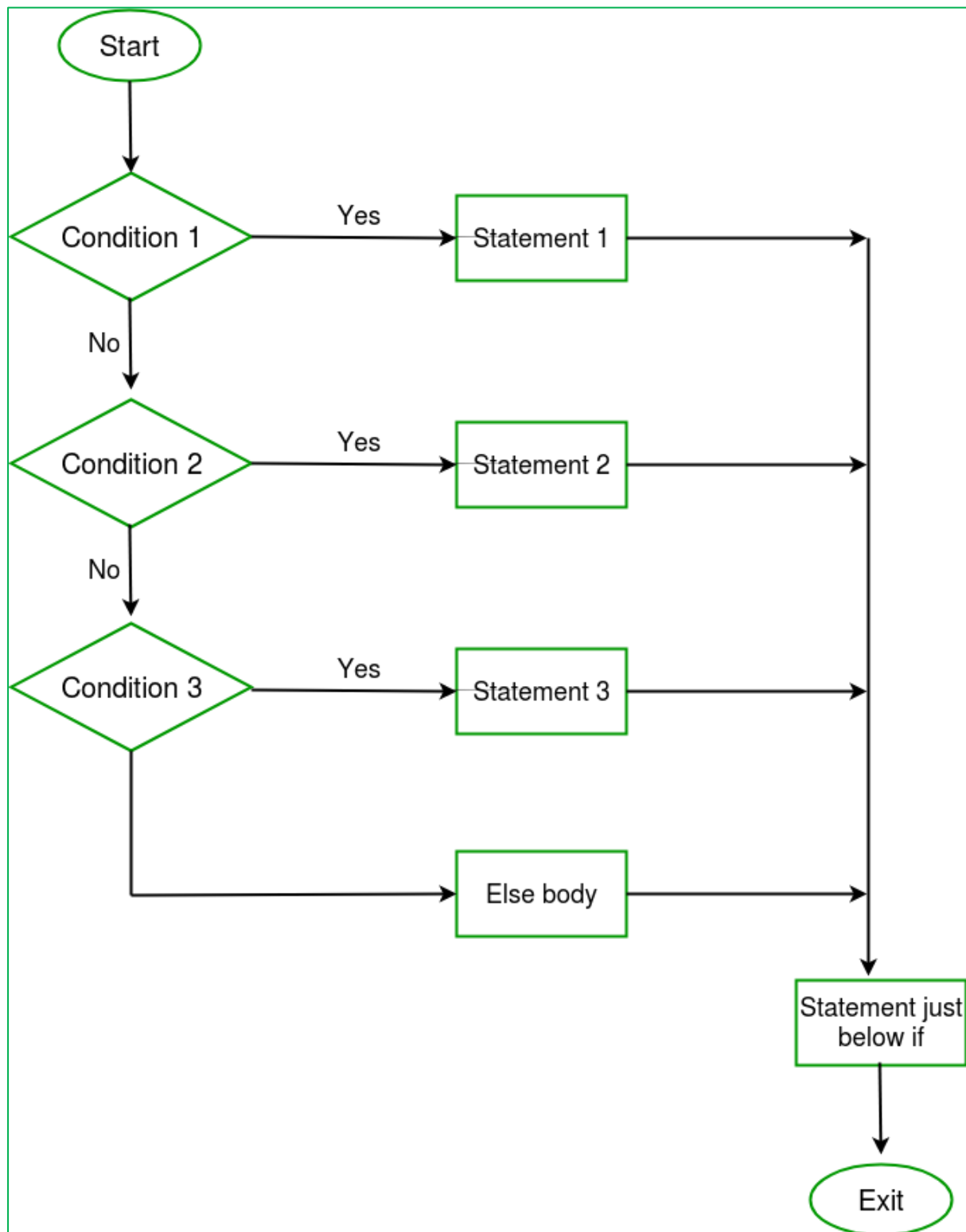
ELSE IF LADDER

- The **if-else-if ladder** helps the user decide from **among multiple options**.
- The C++ **if statements** are executed from the **top down**.
- As soon as one of the conditions controlling the if is true, the statement associated with that **if is executed**, and the rest of the C++ **else-if ladder is bypassed**.
- If none of the **conditions is true**, then the final statement will be executed.

Syntax:

```
if (condition-1)
{
True statement -1 ;
}
else if (condition-2)
{
True statement-2;
}
else if (condition-n)
{
True statement-n;
}
else
{
false statement
}
```

DATA FLOW DIAGRAM



Explanation

- In above diagram
- The **condition-1** is evaluated in the beginning
- If **condition-1** is true means the **statement-1** will be executed
- If **condition-1** is false means , the process will move to check another **if condition** (i.e: **condition-2**) is evaluated along with **else**
- If no one is true means **the** false statement will be executed.

Write a c++ program find greatest of three numbers using else if ladder

```
#include<iostream.h>
#include<conio.h>
void main()
{
clrscr();
int a,b,c;
cout<<"\n Enter 3 numbers:";
cin>>a>>b>>c;
if(a>b && a>c)
{
cout<<"\n A is greater than B and C";
}
else if(b>a && b>c)
{
cout<<"\n B is greater than A and C";
}
else if(c>a && c>b)
{
cout<<"\n C is greater than A and B";
}
else
{
cout<<"\n All numbers are same ";
}
getch();
}
```

OUTPUT

Enter 3 Numbers : 5 2 1

A is greater than B and C

Explanation

- in above program if **condition-1 ($a > b$ and $a > c$)** is satisfied means the result will (**A is greater than B and C**)
- if the condition-1 false means the process will move to check the **condition-2 (i.e: $b > a$ and $b > c$)** is true means the result will come like (**b is greater than A and C**)
- if condition-2 is false means the process will move to check the **condition-3 (i.e: $c > a$ and $c > b$)** is true means the result will come like (**c is greater than B and C**)
- if all the three conditions are false means the process will move to **final** else statement (i.e: **all numbers are same**)

Unconditional Control Transfer Statement(or) Jump Statement

- C++ has **four** statements that perform an unconditional control transfer
- They are
 - **return**
 - **go to**
 - **break**
 - **continue**
- **return**
 - it is used only in **functions**
- The **goto** and **return** may be used anywhere in the program.
- But **continue** and **break** statements may be used only in conjunction with a **loop** statement.
- The **break** is used most frequently in **switch case**.

GO TO STATEMENT

- This statement does not require any condition.
- This statement passes control anywhere in the program without considering any condition.
- In simple terms, the **goto** is a **jump statement** transfers the control flow of a program to a **labeled statement** within the scope of the **same function**, breaking the normal flow of a program.

Syntax:

goto label_name;

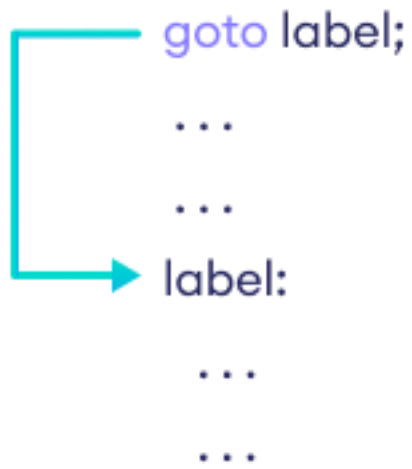
// skipped code

label_name:

// Execution continues here

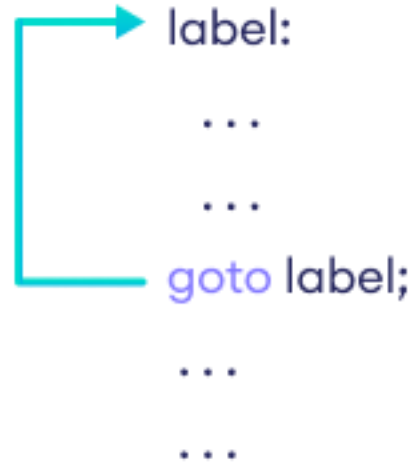
- Here a **label** is any valid label either before or after goto.
- The '**label**' must start with any character and can be constructed with rules used for forming identifiers.

Working flow of goto statement :



A diagram illustrating a forward jump. It shows a code block with a `goto label;` statement, followed by three lines of code represented by ellipses (`...`), and then a `label:` statement followed by two more lines of code represented by ellipses (`...`). A red arrow originates from the `goto` statement and points to the `label:` statement, indicating the jump.

```
goto label;  
...  
...  
label:  
...  
...
```



A diagram illustrating a backward jump. It shows a `label:` statement followed by three lines of code represented by ellipses (`...`), and then a `goto label;` statement followed by two more lines of code represented by ellipses (`...`). A red arrow originates from the `goto` statement and points back to the `label:` statement, indicating the jump.

```
label:  
...  
...  
goto label;  
...  
...
```

Write a c++ program find the given number is positive number. if user entered negative number means the program ask the message (positive number only) again and again using goto statement .

```
#include<iostream.h>
#include<conio.h>
void main()
{
clrscr();
int no;
begin:
cout<<"\n Enter Positive number only:";
cin>>no;
if(no>0)
{
cout<<"\n You have entered Positive Number thank you for using";
}
else
{
goto begin;
}
getch();
}
```

OUTPUT

Enter Positive number only : -5

Enter Positive number only: -2

Enter Positive number only: 5

You have entered Positive Number Thank you for using

Explanation

- In above program the condition-1 **no>0** is evaluated at the beginning
- if the condition (**n>0**) is true means the true statement will be executed (i.e : **You have entered positive number thank you for using**
- if the condition (no>0) **false** means the else statement will be executed .
- The else will transfer the process to **label (begin)** with the help of **goto** statement . this is what jump one statement to another statement without any condition.

Break Statement

- The **break** statement allows the programmer to terminate the loop.
- The **break** skips from the **loop** or the **block** in which it is defined.
- The control then automatically passes on to the first statement after the **loop** or the **block**.
- The **break** can be associated with all the conditional statements (**especially switch case**).
- We can also use **break** statements in the **nested loops**.
- If we use **break** statement in the innermost loop.
- The widest use of this statement is in the **switch case** where it is to avoid flow of control from one “**case**” to the other programming examples on **break** are given in **switch** topic.

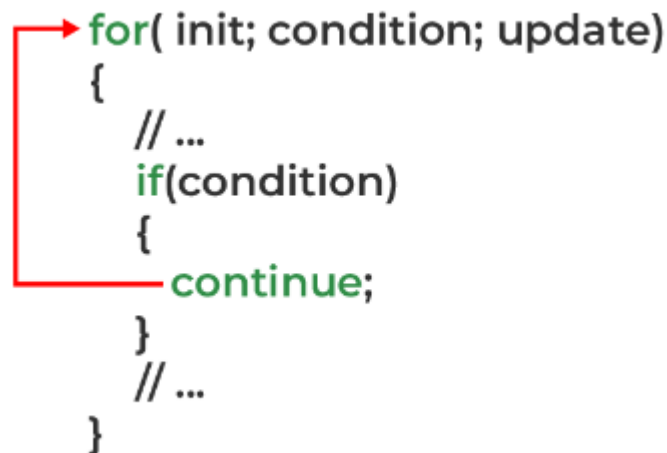
```
int i = 0;
while (i < 10) {
    cout << i << "\n";
    i++;
    if (i == 4) {
        break;
    }
}
```

Continue Statement

- The **continue** statement works somewhat like the **break** statement.
- Instead of forcing the control to the end of the loop (as it is in case of break),
- The **continue** case causes the control to pass on to the beginning of the block/loop .
- In the case of **for** loop, The **continue** case initiate the testing condition and increment on steps has to be executed (while rest of the statement case causes control to pass on to conditional tests.
- It is useful in a programming situation where it is required that particular iteration occur only up to some extent or when some part of the code has to be neglected.
- C++ **continue** statement is a loop control statement that forces the program control to execute the next iteration of the loop.
- As a result, the code inside the loop following the continue statement **will be skipped** and the **next iteration** of the loop will begin.

Syntax

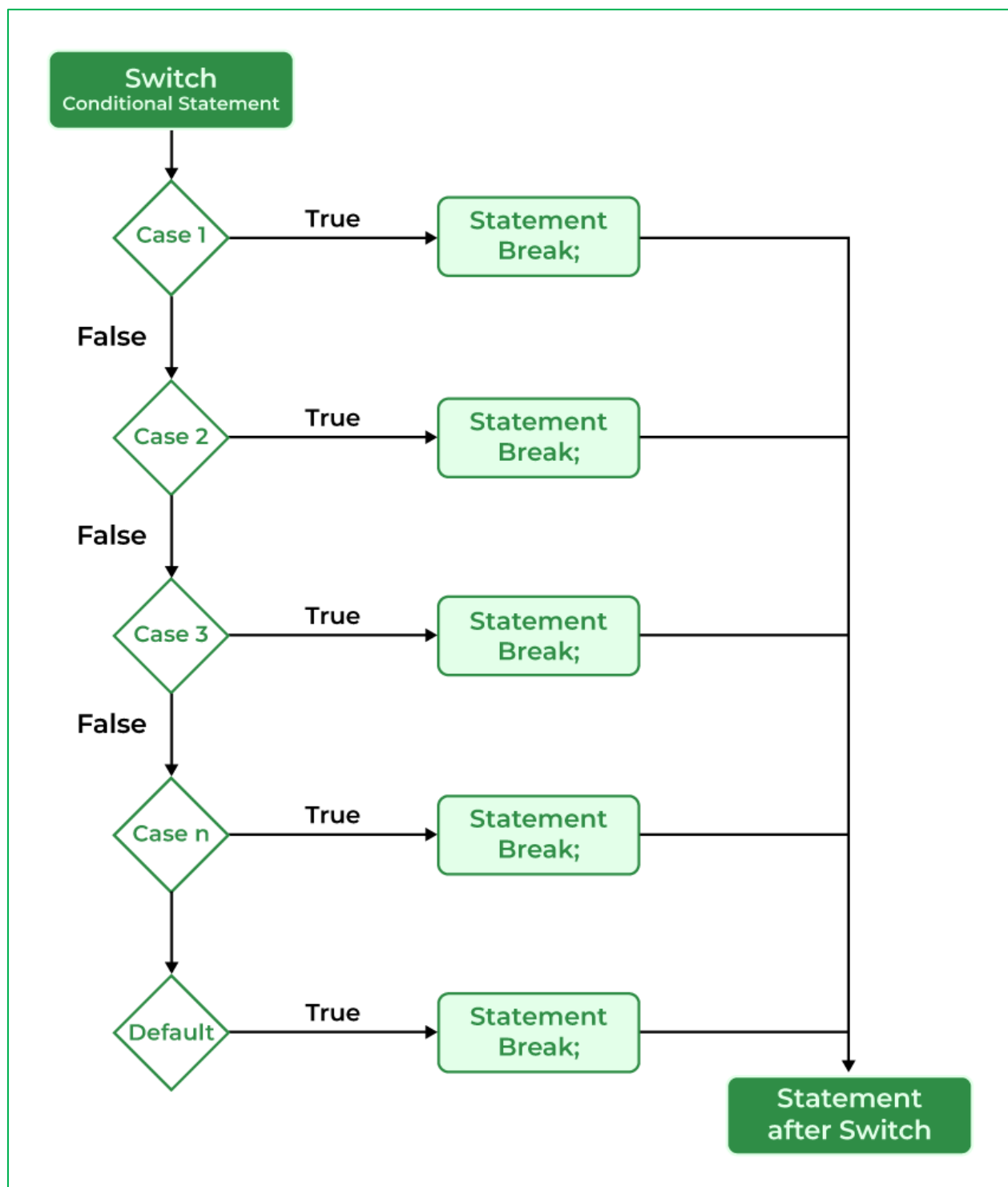
continue;



```
for( init; condition; update)
{
    // ...
    if(condition)
    {
        continue;
    }
    // ...
}
```


Switch Case Statement

- The **switch** case statement is a **multi-way** branch statement and an alternative to **if_else_if** ladder in many situations.
- The expression of switch contains only one argument, which is then checked with a number of **switch cases**.
- The **switch** statement evaluates the expression and then looks for its value among the case constants.
- If the value is matched with a particular case constant, then those case statements are executed until a **break** statement is found or until the end of **switch** block is reached.
- If not, then simply the **default** (if present) is executed (if default is not present , then default is normally present at the bottom of the **switch case** structure.
- The **default** must be not being empty.
- Every case statement terminates with a “:” (**colon**)
- The **break** statement is used to stop the execution of succeeding case and pass the control to the end of the **switch** block.



Explanation

- **Step 1:** The switch expression is evaluated.
- **Step 2:** The evaluated value is then matched against the present case values.
- **Step 3A:** If the matching case value is found, that case block is executed.
- **Step 3B:** If the matching code is not found, then the default case block is executed if present.
- **Step 4A:** If the break keyword is present in the block, then program control comes out of the switch statement.
- **Step 4B:** If the break keyword is not present, then all the cases after the matching case are executed.
- **Step 5:** Statements after the switch statement is executed.

Write a c++ program to display ON/OFF based on given choice like (if 1 present the ON statement will be displayed),(if 0 present the OFF statement will be displayed) using switch case

```
#include <iostream.h>
#include<conio.h>
void main()
{
clrscr();
int num;
cout<<"Enter a number 0 or 1:";
cin>>num;
switch(num)
{
case 1:
cout<<"\n ON";
break;
case 0:
cout<<"\n OFF";
break;
default:
cout<<"Must give 0 or 1 dont use other numbers";
break;
}
getch();
}
```

OUTPUT

Enter a number 0 or 1:1

ON

Explanation

- The program prompts the user to input 1 or 0.
- A switch statement evaluates the user's input:
- If the user enters 1, the program displays ON.
- If the user enters 0, the program displays OFF.
- For any other input, the default case displays an error message.

C++ Loops

- In Programming, sometimes there is a need to perform some operation **more than once** or (say) **n number of times**.
- Loops come into use when we need to **repeatedly execute a block of statements**.
- **For example:**
 - Suppose we want to print **"Hello World"** 10 times.
 - This can be done in two ways as shown below:

Manual Method (Iterative Method)

- Manually we have to write **cout** for the C++ statement 10 times.
- Let's say you have to write it **20 times** (it would surely take more time to write 20 statements) now imagine you have to write it 100 times,
- It would be really hectic to re-write the same statement again and again.
- So, here loops have their role

// C++ program to Demonstrate the need of loops

```
#include <iostream.h>
#include<conio.h>
void main()
{
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    getch();
}
```

Output

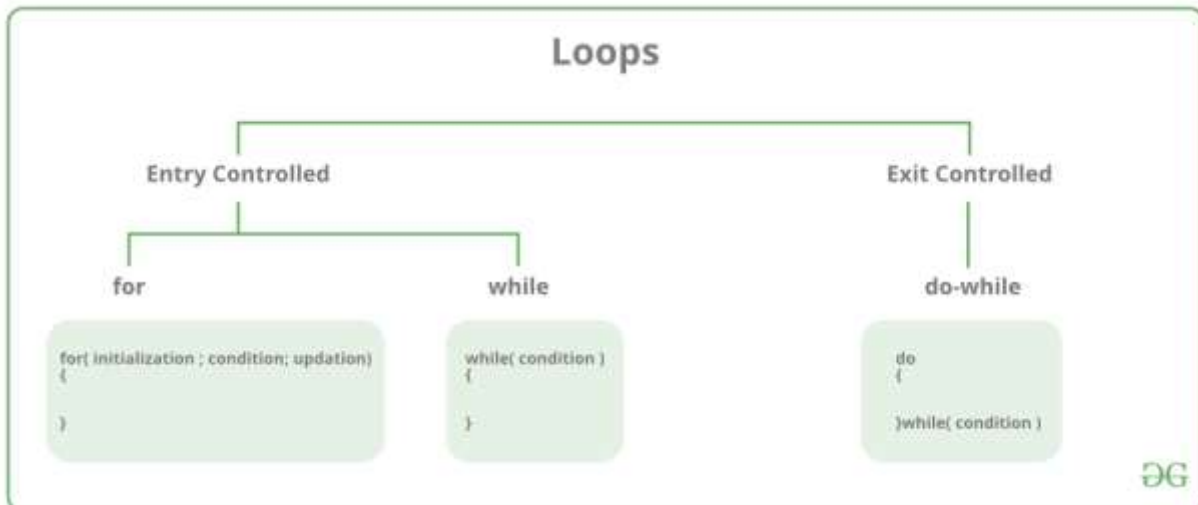
```
Hello World
Hello World
Hello World
Hello World
Hello World
```

Using Loops

- In Loop, the statement needs to be written only once and the loop will be executed 10 times as shown below.
- In computer programming, a loop is a sequence of instructions that is repeated until a certain condition is reached.

There are mainly two types of loops:

- **Entry Controlled loops:** In this type of loop, the test condition is tested before entering the loop body. **For Loop** and **While Loop** are **entry-controlled loops**.
- **Exit Controlled Loops:** In this type of loop the test condition is tested or evaluated at the end of the loop body.
- Therefore, the loop body will execute at least once, irrespective of whether the test condition is true or false. The do-while loop is **exit controlled loop**.



FOR-LOOP

- For loop is an **entry-controlled** loop that is used to **execute a block of code repeatedly** for the specified range of values.
- Basically, for loop allows you to repeat a set of instructions for a specific number of iterations.

SYNTAX

for (initialization; test condition; updation)

```
{  
    // body of for loop  
}
```

The various parts of the for loop are:

1. Initialization Expression in for Loop

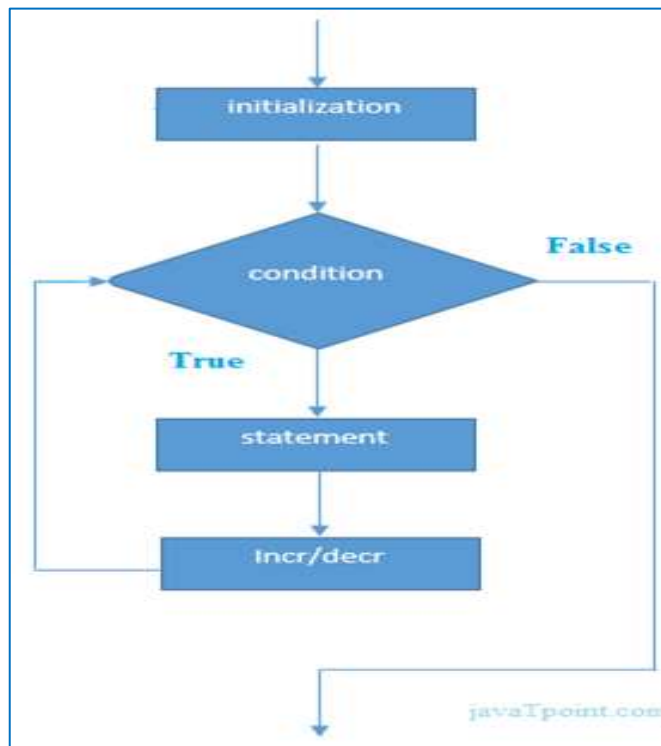
- We have to initialize the loop variable to some value in this expression.

2. Test Condition in for Loop

- In this expression, we have to test the condition. If the condition evaluates to true then we will execute the body of the loop and go to the update expression. Otherwise, we will exit from the for loop.

3. Update Expression in for Loop

- After executing the loop body, this expression increments/decrements the loop variable by some value.



Execution Flow of a for Loop

1. Control falls into the for loop. Initialization is done.
2. The flow jumps to Condition.
3. Condition is tested.
4. If the Condition yields true, the flow goes into the Body.
5. If the Condition yields false, the flow goes outside the loop.
6. The statements inside the body of the loop get executed.
7. The flow goes to the update.
8. Updation takes place and the flow goes to Step 3 again.
9. The for loop has ended and the flow has gone outside.

```
// C++ program to illustrate for loop to print numbers from  
// 1 to n
```

```
#include <iostream.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int n,i;
```

```
    cout<<"Enter the value for n:";
```

```
    cin>>n;
```

```
    for (i = 1; i <= n; i++)
```

```
    {
```

```
        cout << i << " ";
```

```
    }
```

```
    Getch();
```

```
}
```

OUTPUT

Enter the value for n: 5

1 2 3 4 5

Explanation

- The above program uses a **for loop** to **print numbers** from **1 to n** (here n is user given limit).
- The loop variable '**i**' iterates from **1 to n** and in each iteration condition is checked (**is i<=n i.e i<=5**),
- if **true** then **it** prints the value of **i** followed by a space and increment **i**.
- When the condition is **false loop terminates**.

While – Loop

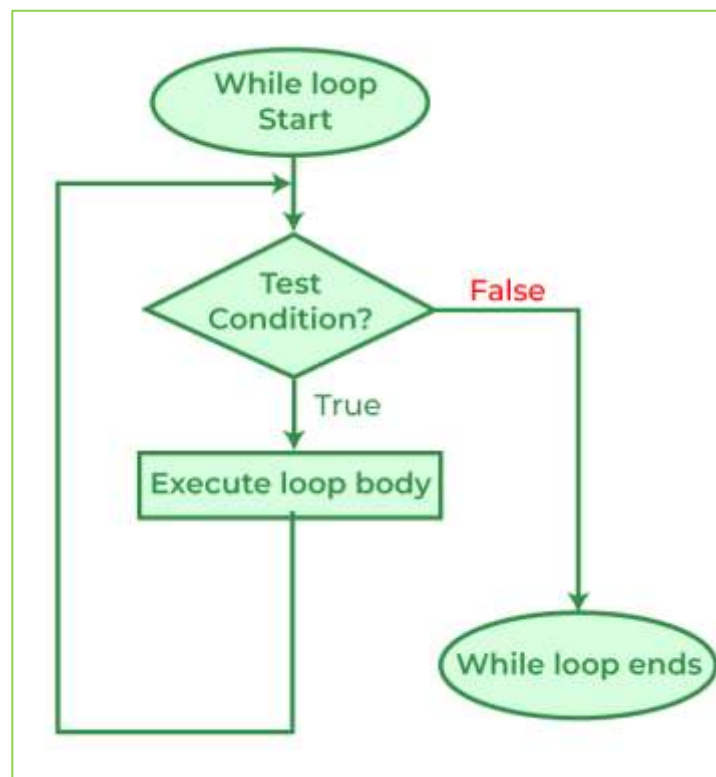
- In C++, **while loop** is used to iterate a part of the program **several times**.
- If the **number of iteration is not fixed**, it is recommended to use while loop than **for loop**.

Syntax:

```
while (test_expression)
{
    // statements
    update_expression;
}
```

The various parts of the While loop are:

1. **Test Expression:** In this expression, we have to test the condition. If the condition **evaluates to true** then we will execute the **body of the loop** and go to update expression. Otherwise, we **will exit from the while loop**.
2. **Update Expression:** After executing the loop body, these expression **increments/decrements** the loop variable by some value.
3. **Body:** This is a group of statements that include **variables, functions**, and so on. With the **while loop**, code, and simple names can be **printed, complex algorithms** can be executed, or **functional operations** can be performed.



How does a While loop execute?

1. Control falls into the while loop.
2. The flow jumps to Condition
3. Condition is tested.
4. If the Condition yields true, the flow goes into the Body.
5. If the Condition yields false, the flow goes outside the loop
6. The statements inside the body of the loop get executed.
7. Updation takes place.
8. Control flows back to Step 2.
9. The while loop has ended and the flow has gone outside.

// C++ program to illustrate while loop

```
#include <iostream.h>
#include <conio.h>
void main()
{
    clrscr();
    int i = 1;
    while (i < 6)
    {
        cout << "Hello World\n";
        i++;
    }
    getch();
}
```

OUTPUT

```
Hello World
Hello World
Hello World
Hello World
Hello World
```

DO-While – LOOP

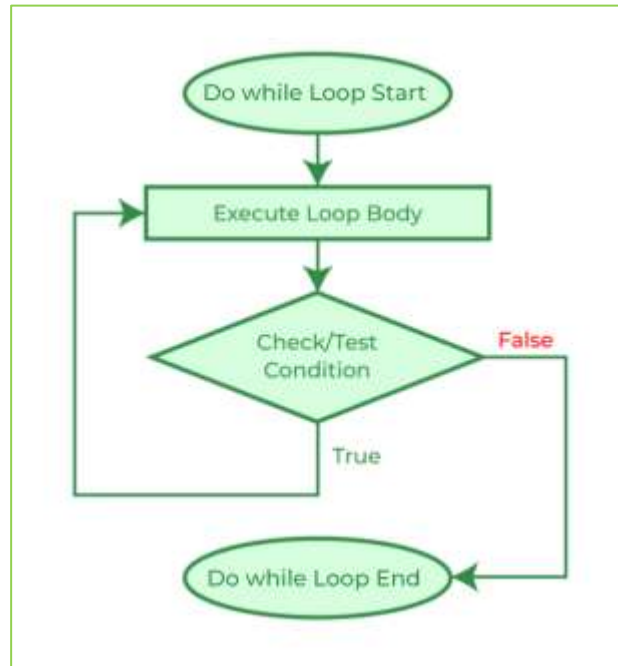
- The C++ **do-while** loop is used to **iterate a part** of the program several times.
- If the **number of iteration** is **not fixed** and you must have to execute the loop at least once, it is recommended to use **do-while loop**.
- The C++ **do-while** loop is executed at least once because condition is checked after loop body.

Syntax:

```
do
{
    // loop body
    update_expression;
}
while (test_expression);
```

The various parts of the do-while loop are:

1. **Test Expression:** In this expression, we have to test the condition. If the condition evaluates to true then we will execute the body of the loop and go to the update expression. Otherwise, we will exit from the while loop.
2. **Update Expression:** After executing the loop body, these expression increments/decrements the loop variable by some value.
3. **Body:** It is the Collection of statements i.e, variables and functions, etc. The condition is not satisfied until the condition is executed automatically after a successful iteration. do-while loop, code can be used to print simple names, execute complex algorithms, or perform functional operations.



How does a do-While loop execute?

1. Control falls into the do-while loop.
2. The statements inside the body of the loop get executed.
3. Updation takes place.
4. The flow jumps to Condition
5. Condition is tested.
6. If the Condition yields true, go to Step 6.
7. If the Condition yields false, the flow goes outside the loop
8. The flow goes back to Step 2.
9. The do-while loop has been ended and flow has gone outside the loop.

// C++ program to illustrate do-while loop

```
#include <iostream.h>
#include <conio.h>
void main()
{
int i = 1;
do
{
// Loop body
cout << i << endl;
// Update expression
i++;
}
// Test expression
while (i <= 5);
getch();
}
```

Output:

```
1
2
3
4
5
```

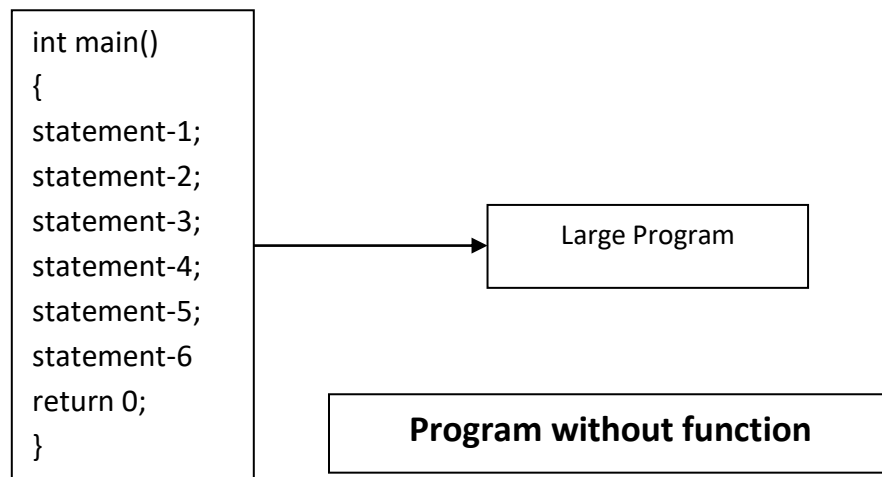
Functions in C++

Introduction

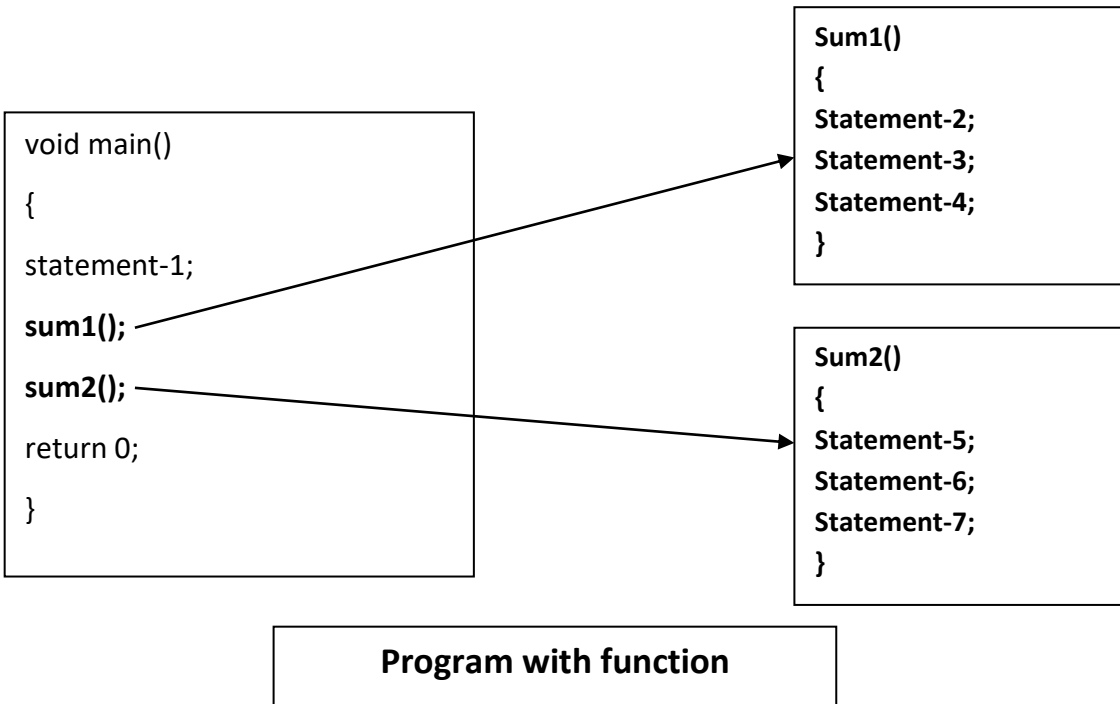
- In our daily life , most of the complex works are done by a group of people.
- This results in better productivity and quality.
- Work is done at a great speed.
- Normally, big work is distributed among many people.

For Example:

- To declare the result of examination in an autonomous institute, **marks/score of all subjects, practical, seminar, projects etc.,** are collected from different departments through professors and the result is compiled, declared, and displayed.
- The same concept is applied while developing complex program/software applications using programming languages.
- In case the **size of the program also hard to identify the flow of data.**
- It **preferred to divide the large program into small modules called functions.**
- Each **function** takes the data that are provided by the **main ()** function and carries out operations as per the **requirement, and results** can be written to the calling function. Thus, programs are made more efficient.
- When the concept of function or sub-program was not introduced, the program **sizes were large** and **few codes get repeated** as shown in the below figure



- The above concept was very difficult to debug and update these large programs because many bugs get encountered.
- When the functions and sub-programs are introduced , the programs are divided into a number of segments and code duplication is avoided as shown in the below **figure**



- Bugs in small programs can be searched easily and this step leads to the development of complex programs with functions.

FUNCTION

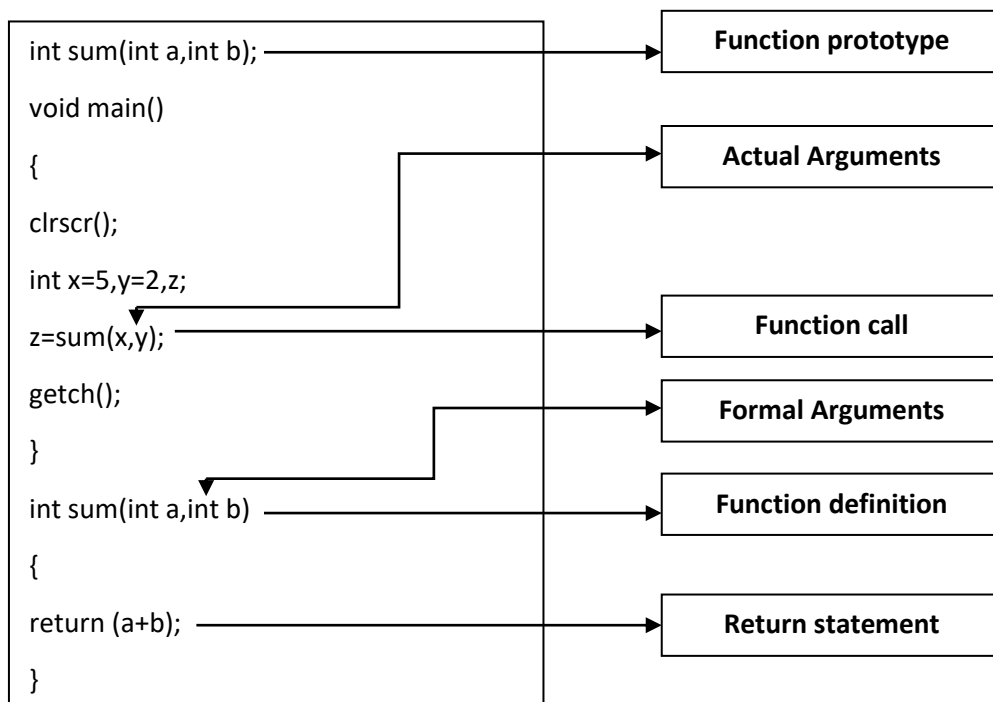
- A **function** is a **block of code** which **only runs when it is called**.
- You can pass data, **known as parameters**, into a function.
- **Functions** are used to perform **certain actions**, and they are important for **reusing code**: Define the code once, and use it many times.
- A **function** is a **set of statements** that takes input, **does some specific computation**, and produces output.
- The idea is to **put some** commonly or **repeatedly done tasks together** to make a function so that instead of writing the same code again and again for different inputs, we can call this function.
- In simple terms, a function is a **block of code that runs only when it is called**.
- The function in C++ language is also known as **procedure** or **subroutine** in other programming languages.

Advantages of Functions

1. **Reusability:** A function once written can be invoked again and again , thus helping us to reuse the code and removing data redundancy.
2. **Modularity:** Function can help us in breaking a large , hard to manage problem into smaller manageable sub-problems. It is easier to understand the logic of sub-programs.
3. **Reduced program size:** function can reduce the size of the program by removing data redundancy.
4. **Easy Debugging:** using functions , debugging of a program becomes very easy , as it is easier to locate and rectify the bug in the program if functions are used.
5. **Easy updating:** if we need to update some code in the program , then it is much more easier in case we have used functions , as the changes need to be made in one place only.

Part of functions

1. **Function prototype declaration**
2. **Function call**
3. **Definition of a function**
4. **Actual and formal arguments**
5. **Return statement.**



Function prototype

- We use many **built-in-functions**. The prototype of these functions are given in the respective header files, and we include them using **#include** directive.
- In C++ , while defining **user-define function** it is unavoidable to declare its **prototype**.
- A prototype statement helps the compiler to check the **return** and **argument** type of the function
- A function prototype declaration consists of function's **return** type , **name** and **arguments list**. It tells the compiler.
 - **Name of the function**
 - **Type of value returned**
 - **The type and number of arguments**

Function call

- It gets activated only when a call to a function is invoked.
- A function must be called by its name followed by argument, or without argument , list enclosed in parenthesis and terminated by semicolon.

Syntax :

Function-name (with/without argument-list);

Function definition

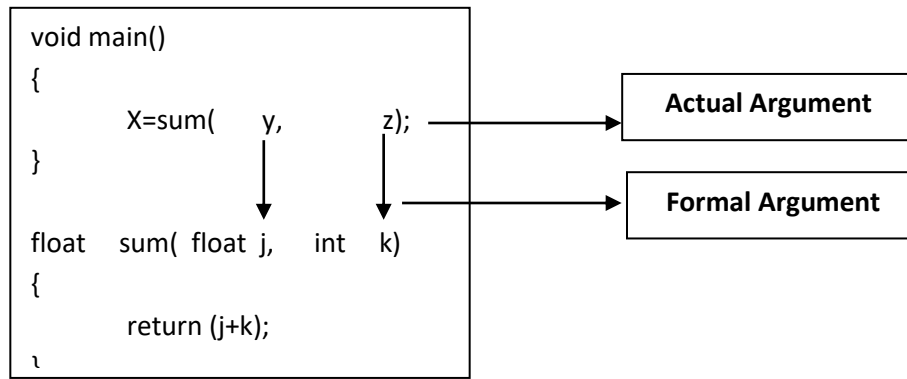
- The first line is called **function definition** and **function** body follows it
- The function definition and function prototype should match with each other.
- The function body is enclosed within curly braces.
- The function can be defined anywhere.
- If the function is defined before its caller, the its prototype declaration is optional.

Syntax :

```
Return_data_Type function-name(argument/parameter-list)
{
    Variable_declaration;
    Function_statement;
}
```

Actual and formal argument

- The arguments declared in caller function and given in the function call are called **actual arguments**.
- The arguments declared in the function definition are known as **formal arguments**.



- In above example variables **y** and **z** are actual arguments and variables **j** and **k** are formal arguments.
- The value of **y** and **z** are stored in **j** and **k** respectively.
- The values of **actual arguments** are **assigned** to **formal** arguments. The function uses **formal arguments for computing**.

Return statement

- The **return** statement is used to **return** value to the **caller** function.
- The **return** statement returns only one value at a time.
- When return statements is encountered , compiler transfers the control of the program to caller function,

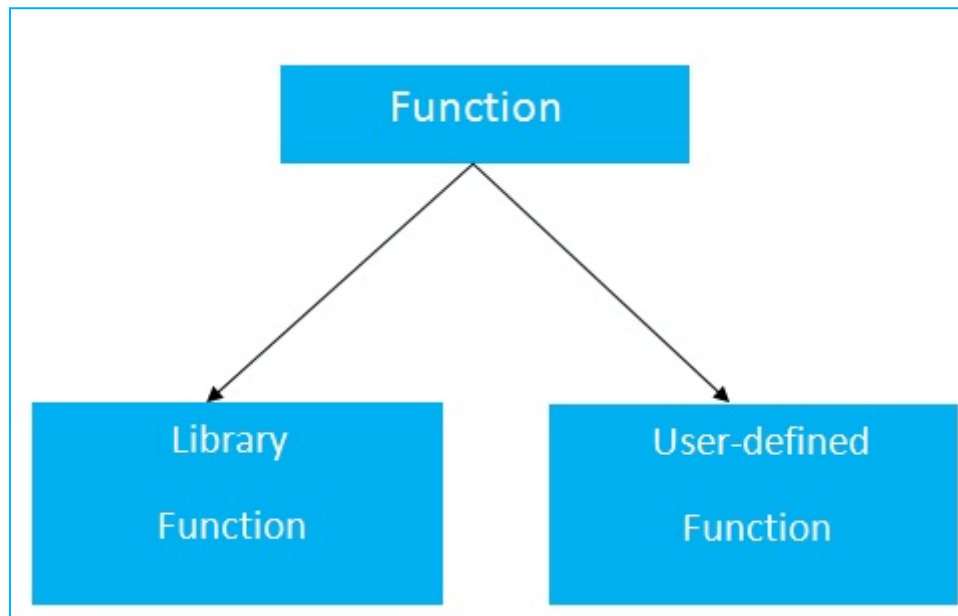
Syntax

Return (variable name); (or) return variable name;

Types of Functions

There are **two types** of functions in C programming:

1. **Library Functions**: are the functions which are declared in the C++ header files such as `print()`, `scanf()`, `clrscr()`, `getch()`, etc.
2. **User-defined functions**: are the functions which are created by the C++ programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code.



Function Categories

1. Without arguments, without return
2. With arguments , without return
3. Without arguments , with return
4. With arguments , with return

Type-1: without arguments , without return

```
#include<iostream.h>
#include<conio.h>
void summation() // function definition area
{
int a,b,c;
cout<<"\n Enter A and B for Addition:";
cin>>a>>b;
c=a+b;
cout<<"\n Addition is:"<<c;
}
void subtraction()
{
float x,y,z;
cout<<"\n Enter X and Y for subtraction:";
cin>>x>>y;
z=x-y;
cout<<"\n subtraction is:"<<z;
}
void main()
{
clrscr();
summation(); // calling user define function
subtraction();
getch();
}
```

OUTPUT

Enter A and B for Addition: 5 2

Addition is : 7

Enter X and Y for subtraction: 5 2

Subtraction is : 3

Type-2: with arguments , without return

```
#include<iostream.h>
#include<conio.h>
void transfer(int x) // here variable 'x' will receive the value from 's'
{
    int y;
    y=4*x;
    cout<<"\n Area of square is :"<<y;
}
void main()
{
    clrscr();
    int s;
    cout<<"\n Enter Number of sides:";
    cin>>s;
    transfer(s); // calling with 1-argument
    getch();
}
```

OUTPUT

Enter Number of sides: 5

Area of square is: 20

Type-3: without argument, with return

// without argument, with return

```
#include<iostream.h>
#include<conio.h>
float area()
{
float b,h,ans;
cout<<"\n Enter breadth of the triangle:";
cin>>b;
cout<<"\n Enter height of the triangle:";
cin>>h;
ans=0.5*b*h;
return ans; // here we are return the final result to main function
}
void main()
{
clrscr();
float a;
a=area(); // calling without argument
// in above line variable 'a' will receive the final result from variable 'ans'
cout<<"\n Area of the triangle is:"<<a;
getch();
}
```

OUTPUT

Enter breadth of the triangle: 5

Enter height of the triangle: 3

Area of the triangle is: 7.5

Type-4: with Argument, with return

```
#include<iostream.h>
#include<conio.h>
int eligible(int e) //here variable 'res' will receive the value from 'uage'
{
    if(e>=18)
        return 1; // true statement
    else
        return 0; // false statement
}
void main()
{
    clrscr();
    char uname[40];
    int uage,res;
    cout<<"\n Enter User name:";
    cin>>uname;
    cout<<"\n Enter User age:";
    cin>>uage;
    res=eligible(uage); // calling with one argument
//in above line variable 'res' will receive the value 1(or) 0 from 'e'
    if(res==1)
    {
        cout<<uname<<" U are Eligible to vote";
    }
    else
    {
        cout<<uname<<" U are not eligible to vote";
    }
    getch();
}
```

OUTPUT

Enter User name: Krishnan

Enter User age: 34

Krishnans U are Eligible to vote

Inline Function

What is an Inline Function in C++

- C++ inline function is powerful concept that is commonly used with classes.
- If a function is **inline**, the compiler places a **copy of the code of that** function at each point **where the function** is called at **compile time**.
- Any change to an **inline function** could require all clients of the function to be recompiled because compiler would need to replace all the code once again otherwise it will continue with old functionality.
- One of the main objectives of using **inline** function in a program is to **save some memory space**.

Normal Function

- Every time a function is called , it takes **a lot of extra time in executing a series of instructions** for tasks such as **jumping to the functions , saving registers , pushing arguments** into the **stack and returning** to the **calling function**.

Syntax of inline Function

```
inline void function_name()  
{  
Function body ;  
}
```

- “**inline**” – keyword sends a **request , not a command , to the compiler .**
- In large program “**inline**” is too long or complicated so it avoid to used there .
- **Inline-** function only used in **small program**

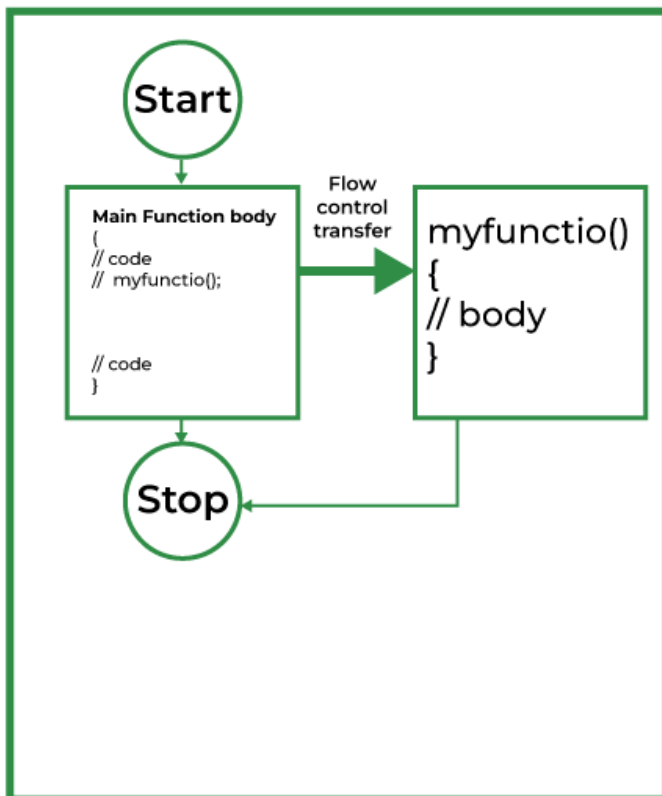
Some the situations where **inline expansion may not work** are:-

1. **For function returning values, if a loop , a switch , or a go to exists .**
2. **For function not returning values, if a return statement exists .**
3. **If function contain static variables**
4. **If inline function are recursive.**

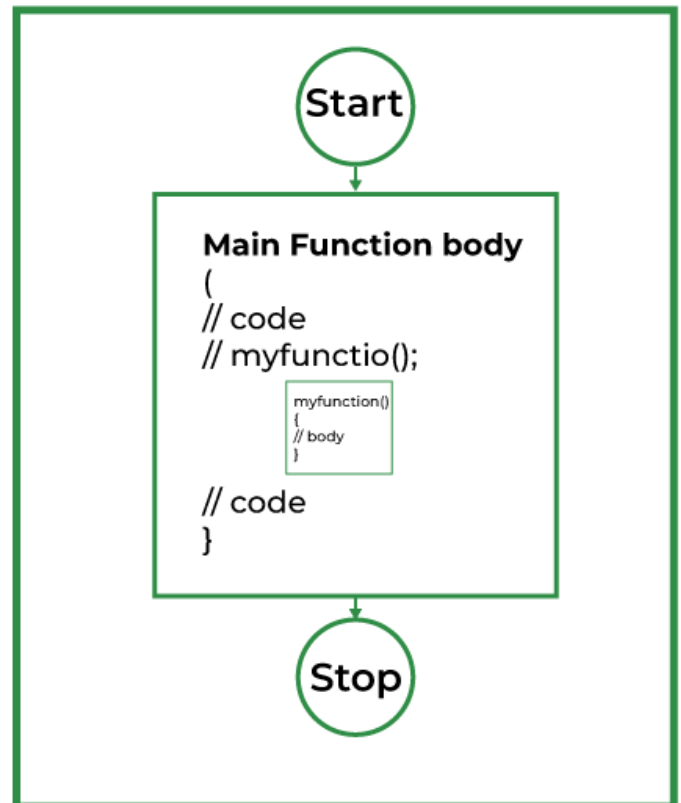
WITHOUT inline function	WITH inline function
<pre> #include <iostream.h> #include<conio.h> int add(int a, int b) { return(a+b); } int main() { cout<<"Addition of 'a' and 'b' is:"<<add(2,3); return 0; } </pre>	<pre> #include <iostream.h> #include<conio.h> inline int add(int a, int b) { return(a+b); } int main() { cout<<"Addition of 'a' and 'b' is:"<<add(2,3); return 0; } </pre>

WITHOUT inline function	WITH inline function
<pre>#include <iostream.h> #include <conio.h> int add(int a, int b) { return(a+b); } int main() { cout<<"Addition of 'a' and 'b' is:"<<add(2,3); return 0; }</pre> Handwritten annotations for 'WITHOUT inline function': - A red circle with the number '1' is placed above the <code>add</code> function definition. - A red arrow points from this circle down to the <code>return(a+b);</code> line. - A red circle with the number '2' is placed next to the <code>return(a+b);</code> line. - A red circle with the number '3' is placed to the left of the <code>cout</code> statement in <code>main</code>. - A red arrow points from this circle to the <code><<add(2,3);</code> part of the <code>cout</code> statement. - A large red curved arrow originates from the <code>return(a+b);</code> line and points towards the <code><<add(2,3);</code> part of the <code>cout</code> statement, indicating a function call.	<pre>#include <iostream.h> #include <conio.h> inline int add(int a, int b) { return(a+b); } int main() { cout<<"Addition of 'a' and 'b' is:"<<add(2,3); return 0; }</pre> Handwritten annotations for 'WITH inline function': - A red box encloses the <code>inline int add(int a, int b)</code> function definition. - A red arrow points from the bottom right corner of this box towards the <code><<add(2,3);</code> part of the <code>cout</code> statement in <code>main</code>. - Handwritten text "The code comes here" is written to the right of the box, with an arrow pointing to the <code><<add(2,3);</code> part of the <code>cout</code> statement.

Normal Function



Inline Function



Function Overloading

- It is possible in c++ to use the **same function name** for **number of times** and for **different intentions**.
- Defining multiple functions with **same name** is known as **function overloading** or **function polymorphism**.
- **Polymorphism** means one function having many forms.
- The **overloading function** must be different in **their argument** list and with **different data types**.
- The following are examples of **overloading functions**.
- All the functions defined should be equivalent to their prototypes.

Example

```
int square(int);  
float square(float);  
long square(long);
```

To write a program to calculate area of some shapes of an int, float numbers.

Define area (). Use function-overloading concept.

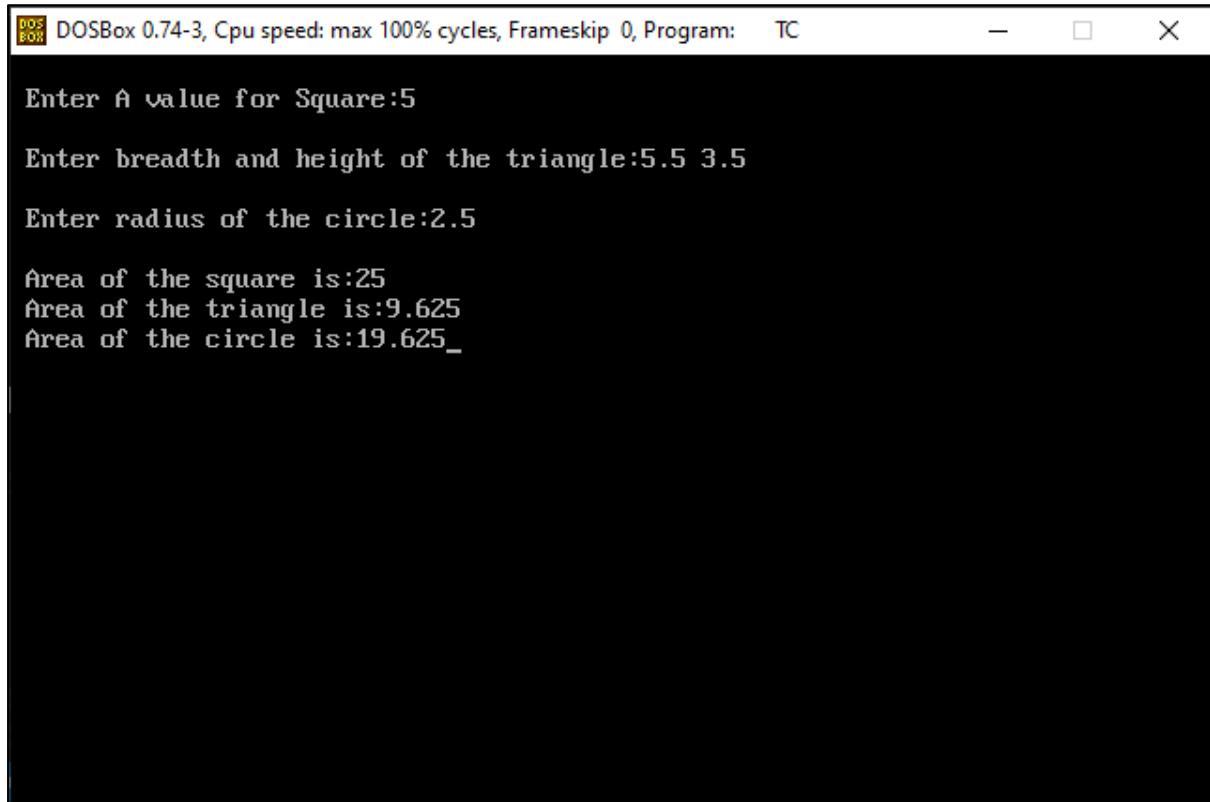
```
#include<iostream.h>  
#include<conio.h>  
int area(int a)  
{  
return a*a;  
}  
float area(float b,float h)  
{  
return 0.5*b*h;  
}  
double area(double r)  
{  
return 3.14*r*r;  
}  
void main()  
{  
clrscr();  
float b1,h1;  
double r1;  
int a1;  
cout<<"\n Enter A value for Square:";  
cin>>a1;  
cout<<"\n Enter breadth and height of the triangle:";
```

```

cin>>b1>>h1;
cout<<"\n Enter radius of the circle:";
cin>>r1;
cout<<"\n Area of the square is:"<<area(a1);
cout<<"\n Area of the triangle is:"<<area(b1,h1);
cout<<"\n Area of the circle is:"<<area(r1);
getch();
}

```

OUTPUT



```

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Enter A value for Square:5

Enter breadth and height of the triangle:5.5 3.5

Enter radius of the circle:2.5

Area of the square is:25
Area of the triangle is:9.625
Area of the circle is:19.625_

```

Explanation

- In the above program **area ()** is overloaded for **integer, float, and double**.
- In the **first call** of the function **area ()**, an integer **5** is passed.
- The compiler executes **integer** version of the function and **returns** result **25**.
- In the **second call** two **floats** values **5.5 and 3.5** is passed to function **area ()**.
- The compiler executes **float** version of the function and **returns** result **9.625**.
- In the **third call** of the function **area ()**, a **double 2.5** is passed.
- The compiler executed **double** version of the function and **returns** result **19.625**.
- The selection of which function to execute is decided at run time by the compiler according to the **data type** of variable passed.

Class and Objects

- In C++, **classes** and **objects** are fundamental concepts of **object-oriented programming (OOP)**.
- They allow you to create and work with **user-defined data types** that combine **data** and **functions** into a **single entity**.

CLASS:

- A **class** is a **blueprint** or **template** for creating objects.
- A **class** is a **user-defined data type**, which holds its own **data members** and **member functions**, which can be accessed and used by creating an **instance** of that class.
- **Data members** are the **data variables** and **member functions** are the **functions** used to manipulate these **variables together**, these **data members and member functions** define the properties and behavior of the **objects** in a Class.
- A **class** is declared by the keyword **class**.

OBJECTS

- An **object** is an **instance of a class**. When you **create an object**, the class's blueprint is used to allocate memory and **initialize data members**.
- Objects allow you to access the class's **data members and member functions**.
- But we **cannot** use the **class** as it is. We first have to create an object of the class to use its features. An Object is an instance of a Class.
- Note: **When a class is defined, no memory is allocated** but when it is instantiated (i.e. an object is created) memory is allocated.

Access Control:

- Access specifier or access modifiers are the labels that specify type of access given to members of a class. These are used for data hiding. These are also called as visibility modes. There are three types of access specifiers.

1. private

2. public

3. protected

1.Private:

- If the **data members** are declared as **private access** then they **cannot be accessed** from other functions **outside the class**.
- It can only be accessed by the functions declared within the class. It is declared by the key word **"private"**.

2.public:

- If the **data members** are declared **public access** then **they can be accessed** from other functions **outside** the class.
- It is declared by the key word "**public**".

3.protected:

- The access level of protected declaration lies between **public and private**. This access specifier is used at the time of inheritance.
- Note:- If **no access specifier** is specified then it is treated by default as **private**.

Syntax:

```
class ClassName  
  
{  
  
    Access Specifier: // can be private, public , protected  
  
    Data Members Declaration; // variable to be used  
  
    Member Function ()  
  
    {  
  
    } // methods to access data members  
  
}; // class Ends with semicolon
```

Defining Member Function

- The **member function** must be declared **inside the class**.
- They can be defined as a **private or public** section.
- They can be defined as **inside or outside** of the class.
- If function is defined **outside** the **class**, its **prototype declaration** must be **done inside** the class.
- While defining the function, **scope access operator** and class **name should precede** the function name.

MEMBER FUNCTION INSIDE THE CLASS:

- Member function inside the class can be declared in **public** or **private** section.

```
#include<iostream.h>
#include<conio.h>
class royal
{
public:
int pid,qty;
float rate,amt;
char pname[30];
void getproduct()
{
cout<<"\n Enter product id:";
cin>>pid;
cout<<"\n Enter product name:";
cin>>pname;
cout<<"\n Enter no of quantity:";
cin>>qty;
cout<<"\n Enter rate of product:";
cin>>rate;
amt=qty*rate;
}
void displayproduct()
{
cout<<"\n product id \t product name \t quantity \t rate \t amount";
cout<<"\n "<<pid<<"\t"<<pname<<"\t\t"<<qty<<"\t"<<rate<<"\t"<<amt;
}
};
void main()
{
clrscr();
royal ob;
ob.getproduct();
ob.displayproduct();
getch();
}
```

OUTPUT

Enter product id: 100

Enter product name: Apple

Enter no of quantity: 5

Enter rate of product: 250

Product id	product name	quantity	rate	amount
100	Apple	5	250	1250

MEMBER FUNCTION OUTSIDE THE CLASS:

- If a function is small , it should be defined inside the class and if large it must be defined outside the class.
- To define a function outside the class, following care must be taken.
 - o The prototype of function must be declared inside the class.
 - o The function name must be preceded by class name and its return type separated by scope access operator.

Scope:

- **Visibility** or **availability** of a variable in a program is called as scope.
- There are two types of scope.
 - o **Local scope:** visibility of a variable is local to the function in which it is declared.
 - o **Global scope:** visibility of a variable to all functions of a program Scope resolution operator in ::
 - o This is used to access global variables if same variables are declared as local and global.

```
#include<iostream.h>

int a=5;

void main()
{
    int a=1;
    cout<<"local variable:"<<a;
    cout<<"Global variable:"<<::a;
}
```

Class Scope:

Scope **resolution operator (::)** is used to define a function **outside a class**.

```
#include<iostream.h>
class rectangle
{
public:
    int L, B;
    void get_data();
    void area();
};
void rectangle::get_data()
{
    cout<<" Enter Length of rectangle";
    cin>>L;
    cout<<"Enter breadth of rectangle";
    cin>>B;
}
int rectangle::area()
{
    return L*B;
}
```

```
void main()
{
rectangle r;
r.get_data();
cout<<"Area of rectangle is" <<r.area();
getch();
}
```

STATIC

Static Variables in a Function

- In a function, when a variable is declared as **static**, space for it gets allocated **for the lifetime of the program**.
- Even if the function is called **multiple times**, space for the static variable is allocated **only once** and the value of the variable in the previous call gets carried through the **next function call**.

WITHOUT STATIC	WITH STATIC
<pre>#include <iostream.h> #include<conio.h> void f() { // without Static variable int i= 0; i++; cout << i << " "; } void main() { clrscr(); // Calling function f() 3 times f(); f(); f(); getch(); }</pre>	<pre>#include <iostream.h> #include<conio.h> void f() { // with Static variable static int count = 0; // The value will be updated and carried over // to the next function call count++; cout << count << " "; } void main() { clrscr(); // Calling function f() 5 times f(); f(); f(); getch(); }</pre>
<u>OUTPUT</u>	<u>OUTPUT</u>
1	1
1	2
1	3

Static Member variable and Function

- When a **static** keyword is used, **variables**, data members, and **functions can't be modified again**.
- It is allocated for the **lifetime of the program**. Static functions can be called directly by **using a class name**.

STATIC MEMBER VARIABLE

- **Static** variables are variables that are defined using **static** keywords, which consist of special behavior compared to regular variables.
- A field which is declared as **static** is called **static field**. Unlike instance field which **gets memory** each time whenever you create object, there is **only one copy of static field created** in the memory. It is **shared to all the objects**.
- **static variable** are accessed using only the **class name** and the **scope resolution** operator **::**

Here we will see its key points.

1. **Static variables** are initialized only once. The compiler persists the variable till the end of the program.
2. Static variables can be defined **inside** or **outside** the function. They are local to the block.
3. The default value of the static variable is **zero**.
4. The **static** variables are alive till the execution of the program.

Example

- It is used to refer the **common property** of **all objects** such as **rateOfInterest** in case of Account, **companyName** in case of **Employee** etc.

EXAMPLE PROGRAM FOR STATIC VARIABLE

```
#include <iostream.h>
#include<conio.h>
class Account
{
    public:
    int accno; //data member (also instance variable)
    char *name;
    static int count;
    Account(int accno, char *name)
    {
        this->accno = accno;
        this->name = name;
        count++;
    }
    void display()
    {
        cout<<accno<<" "<<name<<endl;
    }
};
```

```

int Account::count=0;
void main()
{
    clrscr();
    Account a1 =Account(201, "Sanjay"); //creating an object of Account
    Account a2=Account(202, "Nakul");
    Account a3=Account(203, "Ranjana");
    a1.display();
    a2.display();
    a3.display();
    cout<<"Total Objects are: "<<Account::count;
    getch();
}

```

OUTPUT

```

201 Sanjay
202 Nakul
203 Ranjana
Total Objects are: 3

```

STATIC MEMBER FUNCTION

- The **static member functions** are **special functions** used to access the **static data members** or other **static member functions**.
- A **member function** is defined using the **static** keyword.
- A **static member** function shares the **single copy** of the **member function** to **any number** of the class' objects.
- We can access the **static member function** using the **class name** or **class' objects**.
- If the **static member function** accesses any **non-static data member** or **non-static member function**, it throws an **error**.
- A **static member function** is **independent** of **any object** of the **class**.
- A **static member function** can be called even **if no objects of** the class exist.
- A **static member function** can also be accessed using the **class name** through the **scope resolution operator**.
- A static member function can access **static data members** and **static member functions inside** or **outside** of the class.
- You can also use a static member function to determine how many objects of the class have been created.

Array

- An array in any programming language is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array.
- They can be used to store the collection of primitive data types such as **int** , **float** , **double** , **char** , **etc.**, of any particular type.

Example: let's consider an example of taking random integer from the user.

34	67	78	32	78	98	89
0	1	2	3	4	5	6

----- Array Indexes -----

Array of object

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create object

Syntax

ClassName ObjectName[Number_of_Object];

The Array of object stores objects. An Array of a class type is also known as an Array of objects.

Example

Store more than one Employee data. Let's Assume there is an Array of object for storing employee data emp[50];

objects	Emp_ID	EMP_Name
emp[0]	1001	ABC
emp[1]	1002	DEF
emp[2]	1003	GHI
emp[49]	1049	rrrr

Program-1 : without Array of object

```
#include<iostream.h>

#include<conio.h>

class products
{
public:
int pid;

char pname[30];

float rate;

void get_product_details()
{
cout<<"\n Enter product Id and Name and Rate:";

cin>>pid>>pname>>rate;

}

void display_products()
{
cout<<"\n"<<pid<<"\t"<<pname<<"\t"<<rate;

}

};


void main()
{

clrscr();

products ps;

ps.get_product_details();
```

```
ps.display_products();  
  
getch();  
  
}
```

OUTPUT

Enter Product Id and Name and Rate: 100 Apple 50

100 Apple 50

in above example , a class names **Products** with **pid , pname, rate** being considered.

The two functions are declared-

get_product_details() --> taking user input for pid , pname ,rate

display_products() --> showing the data on the console screen

This program can take the data of only one Product. What if there is a requirement to add data of more than one Products.

Here comes the answer **Array of objects**. An array of objects can be used if there is a need to store data of more than one employee. Below is the C++ program to implement the above approach.

Program-2 : Array of object

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class products
```

```
{
```

```
public:
```

```
int pid;
```

```
char pname[30];
```

```
float rate;
```

```
void get_product_details()
```

```

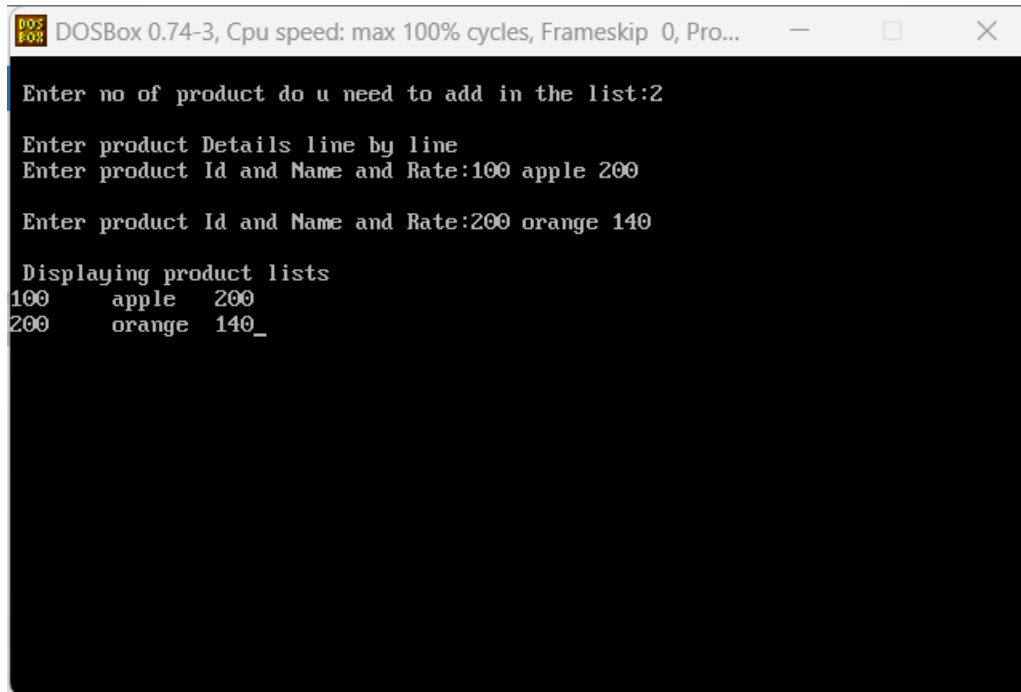
{
cout<<"\n Enter product Id and Name and Rate:";
cin>>pid>>pname>>rate;
}

void display_products()
{
cout<<"\n"<<pid<<"\t"<<pname<<"\t"<<rate;
}
};

void main()
{
clrscr();
int i,n;
products ps[15]; // this is an Array of object having max of 15 products
cout<<"\n Enter no of product do u need to add in the list:";
cin>>n;
cout<<"\n Enter product Details line by line";
for(i=1;i<=n;i++)
{
ps[i].get_product_details();
}
cout<<"\n Displaying product lists";
for(i=1;i<=n;i++)

```

```
{  
ps[i].display_products();  
}  
getch();  
}
```



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...  
Enter no of product do u need to add in the list:2  
Enter product Details line by line  
Enter product Id and Name and Rate:100 apple 200  
Enter product Id and Name and Rate:200 orange 140  
Displaying product lists  
100    apple    200  
200    orange   140_
```

Advantages of Array of objects

The array of objects represent storing multiple objects in a single name

The array of objects , the data can be accessed randomly by using the index number.

Reduce the time and memory by storing the data in a single variable.

FRIEND FUNCTIONS

- The private members cannot be accessed from outside the class. i.e.... a non member function cannot have an access to the private data of a class. In C++ a non member function can access private by making the function friendly to a class.

Definition:

- A friend function is a function which is declared within a class and is defined outside the class. It does not require any scope resolution operator for defining . It can access private members of a class. It is declared by using keyword “friend”

SYNTAX:

```
class <class name>
{
private:
.....
.....
public:
.....
friend <return type> <function name>(<arguments>);
};
*/
```

//EX11 :Friend Function for a Single Class

/*

```
#include<iostream.h>
#include<conio.h>
class a
{
private:
int x;
public:
a()
{
x=10;
}
friend void dis(a ob);
};
```

```

void dis(a ob)
{
cout<<"The Value of the Private member of A = "<<ob.x;
}

//-----
void dis1(a ob)
{
cout<<"Private member of A = "<<ob.x; //Leads to an Error
}

//-----
void main()
{
clrscr();
a ob;
dis(ob);
//dis1(ob);
getch();
}
*/

```

Friend Class:

- A class can also be declared to be the friend of some other class. When we create a friend class then all the member functions of the friend class also become the friend of the other class. This requires the condition that the friend becoming class must be first declared or defined (forward declaration).

```
#include <iostream.h>
#include<conio.h>
class A
{
    private :
    int x =500;
    friend class B;    // friend class.
};
class B:public A
{
    public:
    void display()
    {
        cout<<"value of x is : "<<x;
    }
};
void main()
{
    B b;
    b.display();
    getch();
}
```

OVERLOADING MEMBER FUNCTIONS

overloading member functions is defined as the process of having two or more function with the same name , but different in parameters is known as overloading member function in C++.

In overloading member function , the function is redefined by using either different types of arguments or a different compiler can differentiate between the member functions.

The advantage of overloading member function is that it increases the readability of the program because you dont need to use different names for the same action.

Example:

```
#include<iostream.h>

#include<conio.h>

class shapes
{
public:
int ans1;
float ans2;
void area(int a)
{
ans1=a*a;
cout<<"\n Area of the square is:"<<ans1;
}
void area(float b,float h)
{
ans2=0.5*b*h;
cout<<"\n Area of the triangle is:"<<ans2;
}
};

void main()
```

```
{  
clrscr();  
  
shapes sp;  
  
sp.area(5);  
  
sp.area(5,3);  
  
getch();  
}
```

In above program , the class **shapes** has a member function **area()**.

The **area()** function is overloaded for **integer** , **float** . In function **main()** the object **sp** invokes the member function area with **one value**. The compiler invokes the overloaded function according to the value.

BIT Fields & Classes

- When we declare the member variables in a class , a **pre-defined** size is allocated for the member variables.
- In C++ , we can **compress** that allocated size using **bit fields**.
- so we can compress the above bytes as our wish using **bit fields**
- Bit fields is a feature in C++ for specifying the size of **struct** and **class members** so that they **occupy a specific number of bits** , not the **default memory size**.
- Example: if u declare a variable as **int** data types means it allocated memory size is **4** bytes based on compiler
- An integer takes **4 bytes(4x8=32 bits)** of size but the size is large according to our requirements so we can change it to **2 bytes(2x8=16 bits)** using **bit fields**.

BIT Fields & Classes

Syntax

- We can define a **bit fields** by specifying the data type and then field name with the desired width in **bits**.

```
class ClassName
{
public:
    dataType var_name : width;
    // more bit-field members....
};
```

Without Bit-Fields

```
#include<iostream.h>

#include<conio.h>

class Loan
{
public:
```

```

int principle;

int int_rate;

};

void main()

{

clrscr();

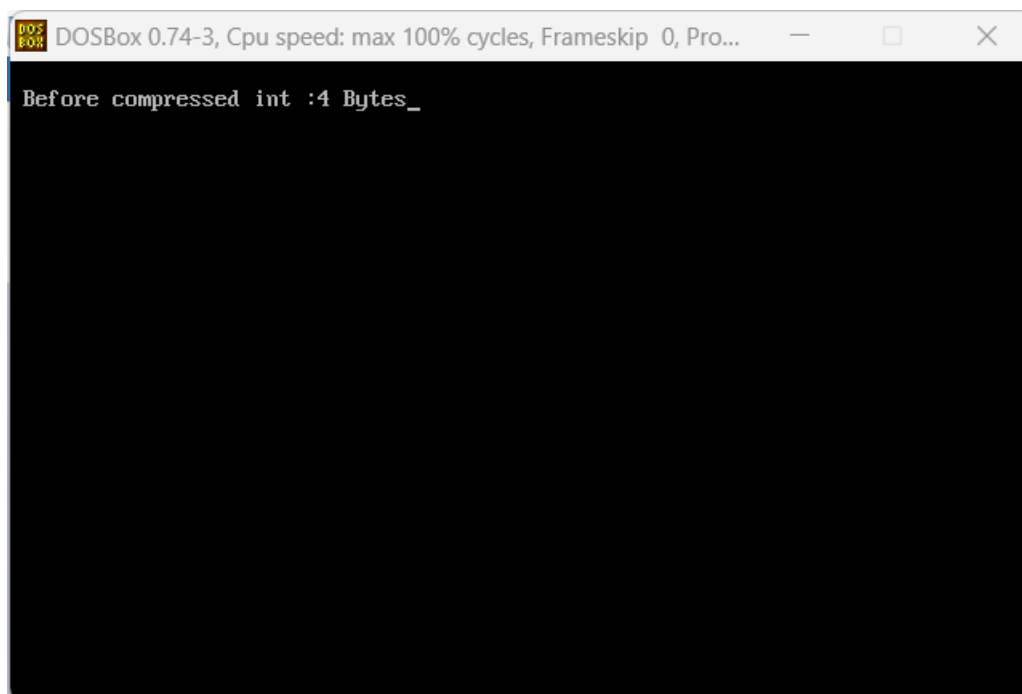
Loan ln;

cout<<"\n Before compressed int : "<<sizeof(Loan)<<" Bytes";

getch();

}

```



The screenshot shows a DOSBox window titled "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The window contains a black terminal area with the text "Before compressed int :4 Bytes_" displayed in a monospaced font. The text is aligned to the left and followed by a cursor underscore.

With Bit Fields

```

#include<iostream.h>

#include<conio.h>

class Loan

{

```

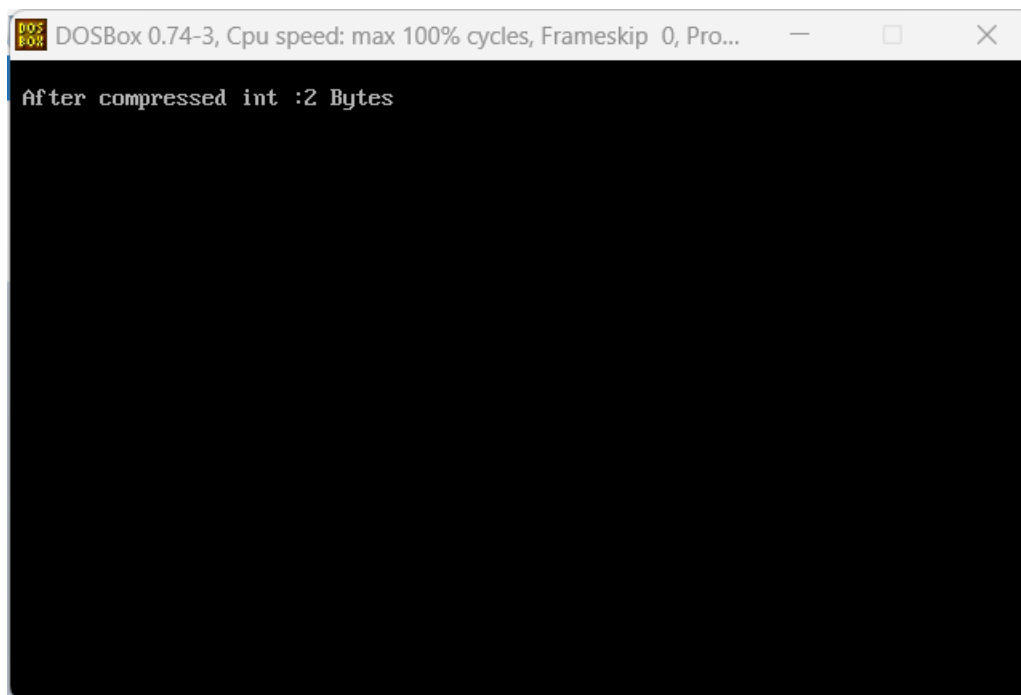
```
public:
int principle:8;
int int_rate:8;
};

void main()
{
clrscr();

Loan ln;

cout<<"\n After compressed int :"<<sizeof(Loan)<<" Bytes";

getch();
}
```



Advantages of Bit-fields in C++

- It is majorly for memory optimization as it allows us to use only the necessary number of bits to represent a member variable which reduces memory overhead.
- It can be used to compress the data so that it can take less time to transfer data from one point to another over the network.

- It can be used to design hardware registers that have specific bit structures.
- In graphics applications , color components , pixel formats and image data often have specific bit requirements which can be customized using bit fields.

Constructor

- In C++, **Constructor** is a **special** member function that is invoked(executed) automatically at the time of creating object for class.
- It is used to **initialize** the value for member variable(or) objects.
- The **constructor** has the **same name** as the **class**.

Characterstics of Constructor

1. constructor have the **same name** as that of the **class name**.
2. constructor are executed automatically at the time of creating object for class.
3. They **don't have** return types.
4. it is used to assign the value to the variable
5. They should be declared in the **public** section
6. A constructor that accepts **no parameter** is called the **default constructor**.

Types of constructor

7. **Default constructor**
8. **Parameterized constructor**
9. **copy constructor**
- 10.

1) Default Constructor

- A default constructor is a constructor that doesn't take any arguments.
- It has **no parameters**.
- It is also called a **zero-argument constructor**.
- The compiler automatically creates an implicit default constructor if the programmer does not define one.

Example

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class circle
{
public:
float pi,r,ans;

circle() // constructor
{
pi=3.14;// value intialized

cout<<"\n Pi value assigned using constructor";

}

void area()
{
cout<<"\n enter radius of circle:";

cin>>r;

ans=pi*r*r;

cout<<"\n area of the circle is:"<<ans;

}

};

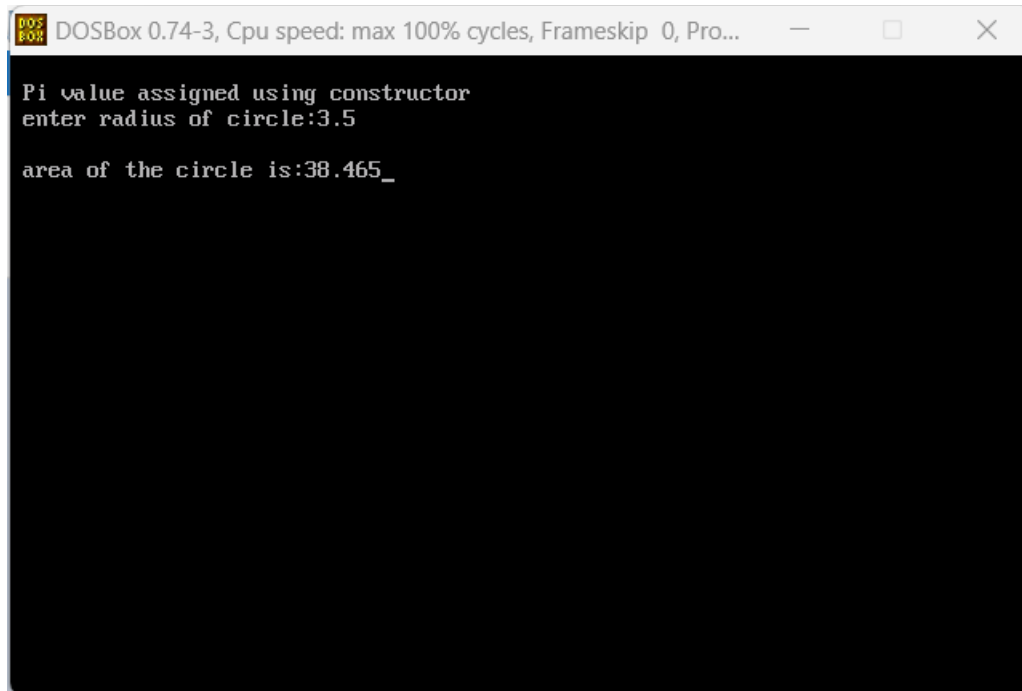
void main()
{
clrscr();

circle ob;//here constructor is executed

ob.area();

getch();

}
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The window contains a black terminal area with white text. The text shows the output of a C++ program: "Pi value assigned using constructor", "enter radius of circle:3.5", and "area of the circle is:38.465_".

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Pi value assigned using constructor
enter radius of circle:3.5
area of the circle is:38.465_
```

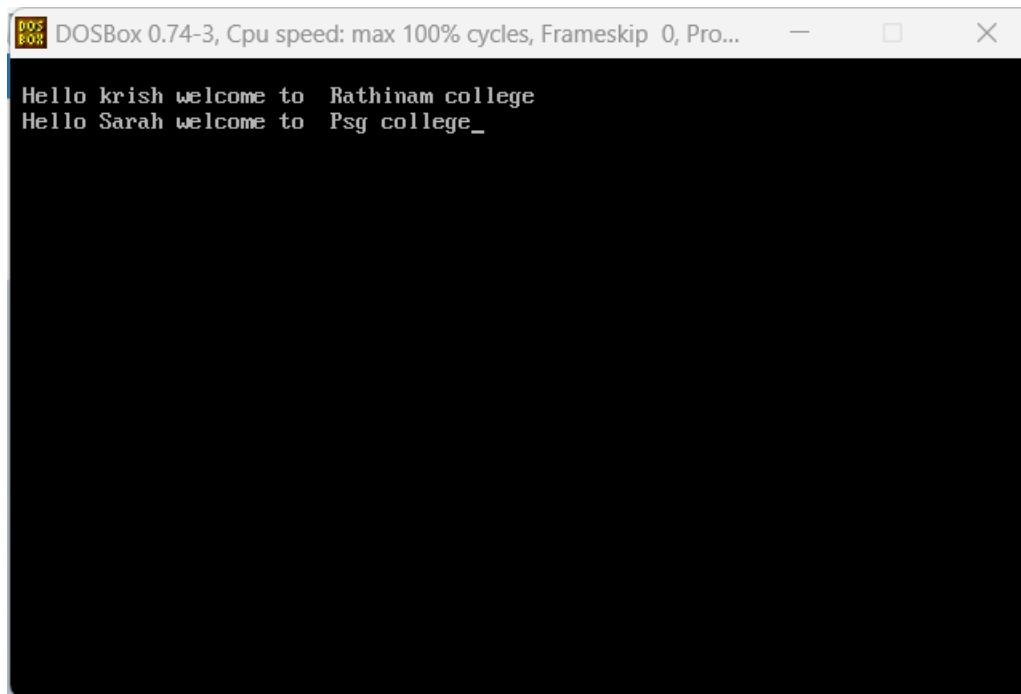
2. Parameterized constructor

- Parameterized constructor make it possible to pass arguments to constructors.
- For such constructor it is necessary to pass the values to the constructor when an object is created.

Example

```
#include<iostream.h>
#include<conio.h>
class college
{
public:
college(int rno,char sname[30],char clgname[30])//parameterized constructor
{
cout<<"\n Hello "<<sname<<" welcome to "<<clgname;
}
};
void main()
{
```

```
clrscr();  
college st1(100,"krish"," Rathinam college");  
college st2(200,"Sarah"," Psg college");  
getch();  
}
```



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...  
Hello krish welcome to Rathinam college  
Hello Sarah welcome to Psg college_
```

3. Copy Constructor

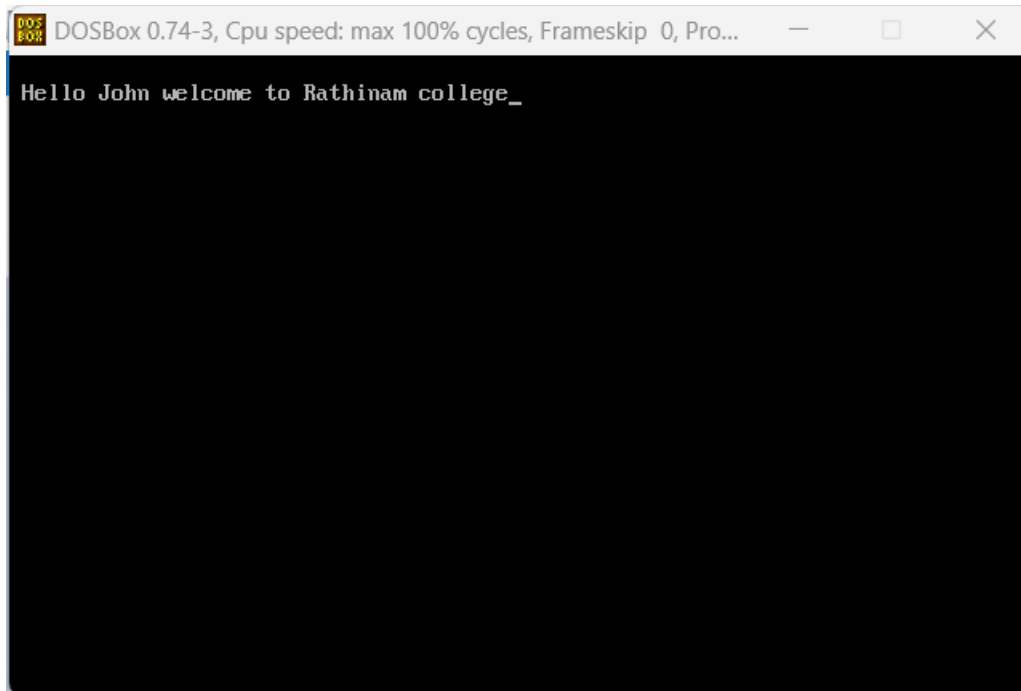
- A **copy constructor** is a member function that initializes an object using another object of the same class.
- copy constructor takes a reference to an object of the same class as an argument.

Example

```
#include<iostream.h>  
#include<conio.h>  
class college  
{  
public:  
char *name;  
char *clname;
```

```
college()
{
name="John";
clgname="Rathinam college";
}
college(college &os)
{
cout<<"\n Hello "<<os.name<<" welcome to "<<os.clgname;
}
};

void main()
{
clrscr();
college st1;
college st2(st1);
getch();
}
```



Destructor

- Destructor is an instance member function that is invoked automatically whenever an object is going to be destroyed.
- Meaning a destructor is the last function that is going to be called before an object is destroyed.
- Destroyed have the same name as that of the class they belong to preceded by ~ (tilde)
- The process of destroying the class objects created by **constructors** is done in destructor.
- A destructor is automatically executed when object goes out of scope.
- It is also invoked when delete operator is used to free the memory allocated with class pointer.

Constructor And Destructor with Static members

- Every object has its own set of data members.
- When a member function is invoked, only copy of data member of calling object is available to the function.
- Sometime, it is necessary for all the objects to share fields which are common for all the objects.
- If the member variable is declared as static, only one copy of such member is created for entire class.
- All objects access the same copy of static copy of static variable.

- The static member variable can be used to count the number of objects declared for a particular class.

Example

```
#include<iostream.h>

#include<conio.h>

class man

{

public:

static int no;

man()

{

no++;

cout<<"\n Number of object exists:"<<no;

}

~man()

{

--no;

cout<<"\n Number of objec exists:"<<no;

}

};

int man::no=0;

void main()

{

clrscr();

man A,B,C;

cout<<"\n Press Any key to Destroy object";

getch();

}
```

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...  
Number of object exists:1  
Number of object exists:2  
Number of object exists:3  
Press Any key to Destroy object  
Number of objec exists:2  
Number of objec exists:1  
Number of objec exists:0
```

Operator Overloading

- The capability to relate the existing operator with a member function and use the resulting operator with objects of its class as its operands is called operator overloading.
- We are familiar with **function overloading** in which multiple functions use the same name.
- The concept of operator overloading is somewhat similar to that of **function overloading**.
- A symbol that is used to perform an operation is called **operator**.
- It is used to perform an operation with constant and variables.
- The compiler knows how to perform various operations using **operator** for the **built-in** types ; however for the **object** those are instance of the class, the **operation** routine must be defined by the programmer.
- **For Example:** in traditional programming languages the **operator** such as **+, -, *, /** , etc, can be used only with basic data types such as **int , float**.
- The **operator** **+(plus)** can be used to perform **addition of two variables**, but the same is not applicable for **objects**.
- The **compiler** cannot perform **addition of two objects**.
- The compiler would throw an error if **addition of two objects** is carried out.
- we can solve the above problem with the help **operator overloading concept**.

Program-1: Adding two objects variable without operator overloading

```
#include<iostream.h>

#include<conio.h>

class numbers
{
public:
int x,y;

numbers() //default constructor
{
}
```

```
numbers(int j,int k) // constructor with 2-arguments
{
    x=j;
    y=k;
}

void show()
{
    cout<<"\n x="<<x<<" y="<<y;
}

};

void main()
{
    clrscr();

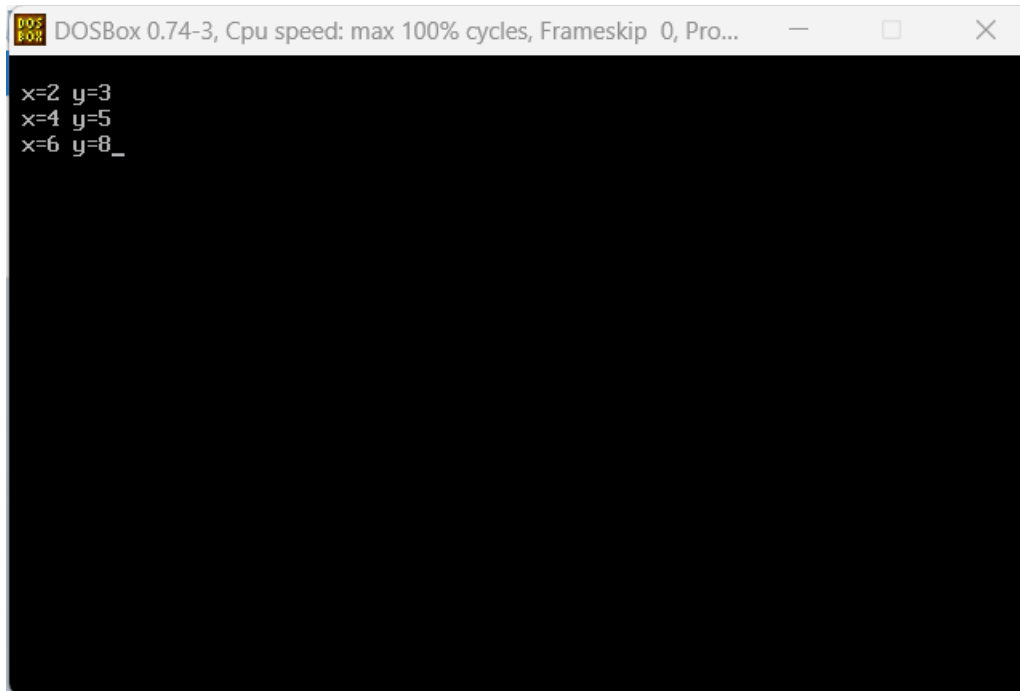
    numbers ob1(2,3);
    numbers ob2(4,5);
    numbers ob3;

    ob1.show();
    ob2.show();

    ob3.x=ob1.x+ob2.x;//Addition of two member variable using objects
    ob3.y=ob1.y+ob2.y;// using member variables directly

    ob3.show();

    getch();
}
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The main window area is black with white text in the top-left corner that reads:

```
x=2 y=3
x=4 y=5
x=6 y=8_
```

- In above program **ob1,ob2 and ob3** are objects of class **numbers** .
- using **constructor** , objects are intialized . consider the following statemetns;

ob3.x=ob1.x+ob2.x;

ob3.y=ob1.y+ob2.y;

- in above example , addition of members of objects **ob1 and ob2** is performed and stored in **ob3**.
- each member variables is accessed individually and stored in member variable of **ob3**.
- **For example:** member **x** of **ob1** and member **x** of **ob2** are added and stored in **x of ob3** . Similarly addition of the other members is carried out
- In this program we **cannot perform operation ob3=ob1+ob2** . The operation with object is complicated because it invloves operation of one or more data member variables which are part of objects.

Syntax :

return type **operator operator_symbol(parameters)**

```
{
    statement-1;
    statement-2;
}
```

EXAMPLE

numbers operator +(number D)

```
{  
    number T;  
    T.x=x+D.x;  
    T.y=y+D.y;  
    return T;  
}
```

Operator Overloading can be done by using 3-approaches

1. **overloading unary operator** (if only one operand having means is called unary operator)
2. **overloading binary operator** (if two operand having means is called binary opearator)
3. **overloading binary operator with friend function**

Overloading Unary Operator

4. In the Unary operator function , **no argument** should be passed.
5. It works only with one class object.
6. It is the overloading of an operator operating on a single operand.

Program for Overloading Unary operator

```
#include<iostream.h>  
  
#include<conio.h>  
  
class Distance  
{  
public:  
    int feet,inch;  
    Distance(int feet,int inch)  
{  
    this->feet=feet;  
    this->inch=inch;
```

```
cout<<"\n Before  unary operator";

cout<<"\n Feet:"<<this->feet<<" inches:"<<this->inch;

}

void operator -()

{

feet--;

inch--;

cout<<"\n After unary operator";

cout<<"\n Feet:"<<feet<<" inches:"<<inch;

}

};

void main()

{

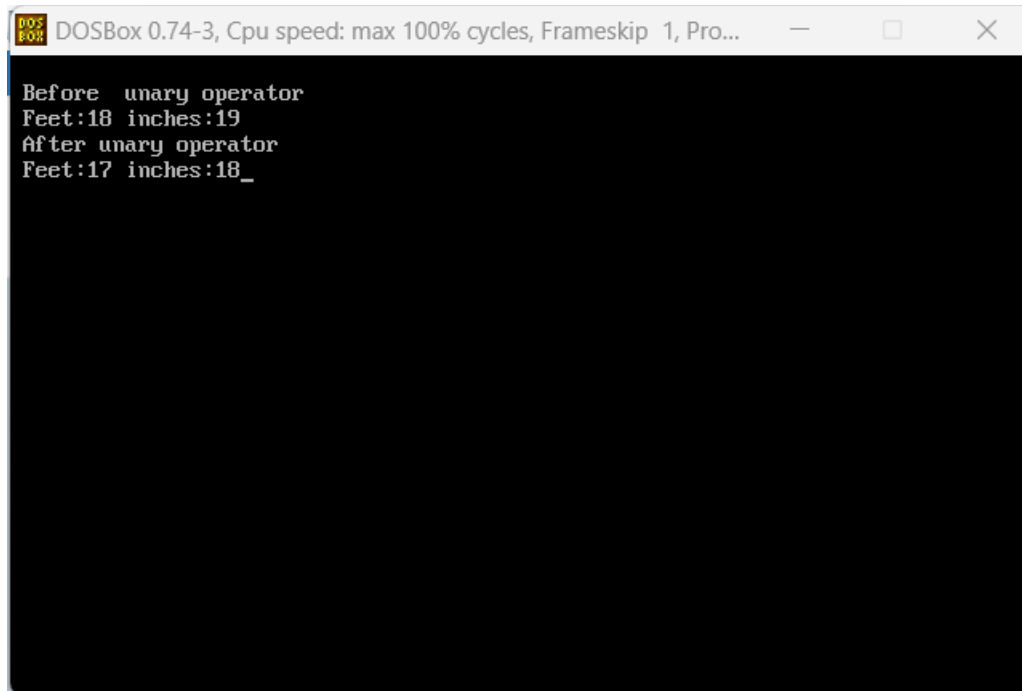
clrscr();

Distance d1(18,19);

-d1;

getch();

}
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 1, Pro...". The window contains a black terminal area with white text. The text shows the state of a program before and after a unary operator is applied to a Distance object. Before the operator, the object has 18 feet and 19 inches. After the operator, it has 17 feet and 18 inches.

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 1, Pro...
Before unary operator
Feet:18 inches:19
After unary operator
Feet:17 inches:18_
```

Overloading Binary operator

- In the **binary** operator overloading function , there should be one argument to be passed.
- It is the overloading of an operator operating on two operand.
- Below is the **C++** program to show the overloading of the binary operator (+) using a class **Distance** with two distant objects.

PROGRAM for Overloading Binary Operator

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class Distance
```

```
{
```

```
public:
```

```
int feet,inch;
```

```
Distance()
```

```
{
```

```
this->feet=0;
```

```
this->inch=0;
```

```

//cout<<"\n Before unary operator";

//cout<<"\n Feet:"<<this->feet<<" inches:"<<this->inch;

}

Distance(int f,int i)

{

this->feet=f;

this->inch=i;

}

Distance operator+(Distance &d2)

{

int ans1,ans2;

ans1=this->feet+d2.feet;

ans2=this->inch+d2.inch;

//return d3;

//cout<<"\n After unary operator";

cout<<"\n Feet:"<<ans1<<" inches:"<<ans2;

}

};

void main()

{

clrscr();

Distance d1(18,19);

Distance d2(20,12);

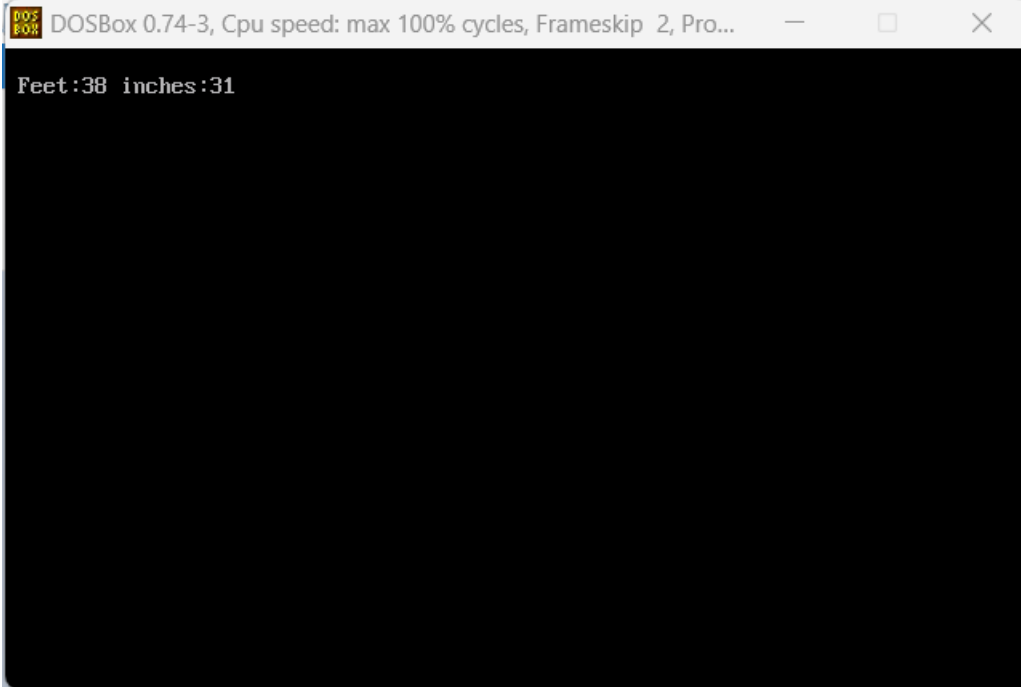
Distance d3;

d3=d1+d2;

getch();

}

```



TYPE Conversion

- A type (data type) cast is basically a conversion from one type to another.
- There are two types of type conversion.

1. Implicit Type Conversion

2. Explicit Type Conversion

Implicit Type Conversion

- It is also known as "**Automatic type conversion**"
- Done by the **compiler on its own**, without any **external trigger from** the user.
- Generally takes place when is an expressing , more than one data type is present.
- In such condition **type conversion** (type promotion) takes place to **avoid lose of data**.
- All the data types of the variables are **upgraded** to the data of the variable with largest data type.

TYPE CONVERSION

bool --> char --> short int --> int --> unsigned int --> long --> unsigned --> long long --> float --> double --> long double

- it is possible for **implicit** conversion to lose information , signs can be lost (when signed is implicitly converted to unsigned) and overflow and occur (when long long is implicitly converted to float).

Program for convert implicitly type conversion

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int x=10; //integer x
```

```
char y='a'; // character a
```

```
// y implicitly converted to int. ASCII value of 'a' is 97
```

```
x=x+y;
```

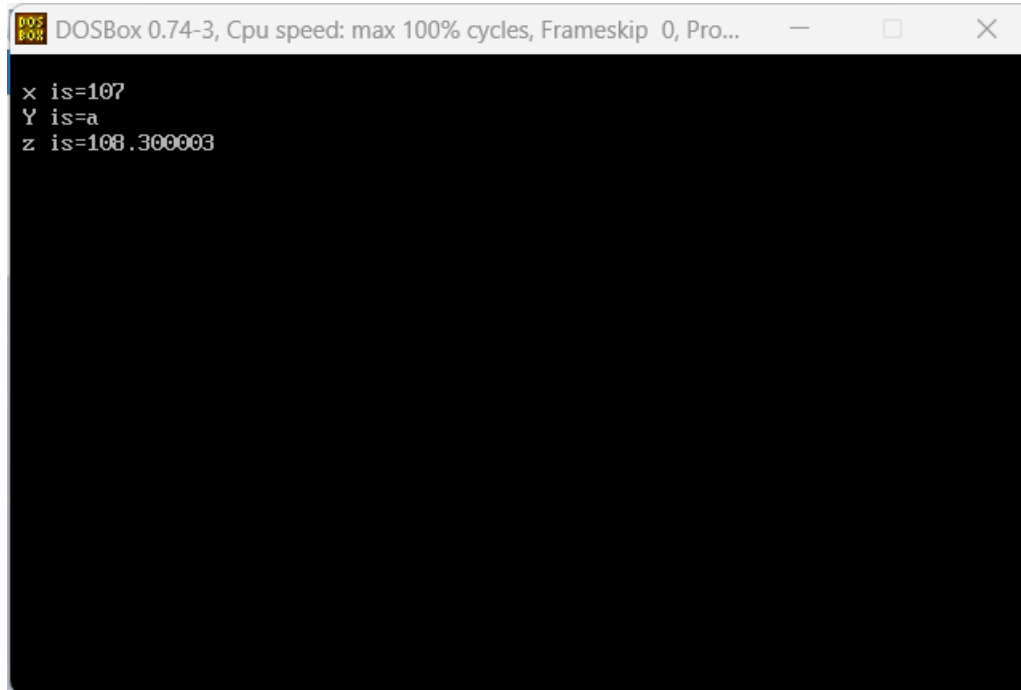
```
// x is implicitly converted as float

float z=x+1.3;

cout<<"x="<<x<<"\n Y="<<y<<"\n z="<<z;

getch();

}
```



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
x is=107
Y is=a
z is=108.300003
```

Explicit Type Conversion

- This process is also called as **type casting** and it is **user-defined**.
- Here the user can typecast the result to make it of a particular data type.

In C++ , It can be done by two ways

3. Converting by assignment
4. Conversion using Cast operator

Converting by Assignment

- This is done by explicitly defining the required type is front of the expression in parenthesis.
- This is can be also considered as forceful casting.

syntax

(type) expression

Program to convert higher data type to lower data type

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

float pi=3.2;

int a=5,b=3;

//Explicitly conversion from float to int

cout<<"\n Before convert:"<<pi+1;

int s=(int)pi+1;

cout<<"\n After convert float to int:"<<s;

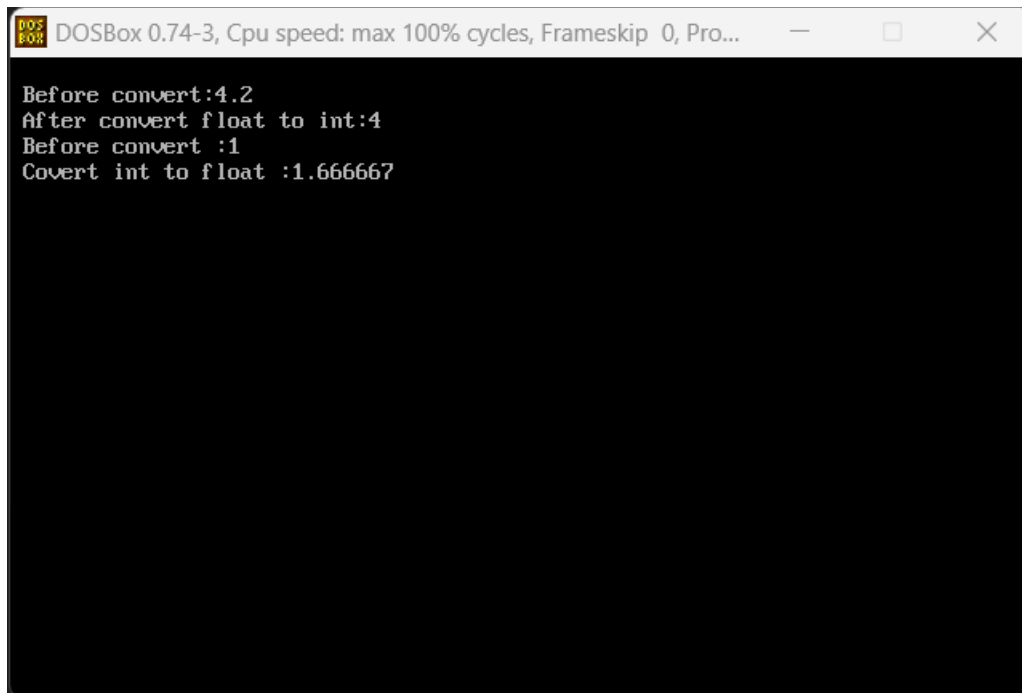
cout<<"\n Before convert : "<<a/b;

float c=(float)a/b;

cout<<"\n Covert int to float : "<<c;

getch();

}
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The window contains a black terminal area with white text showing the output of a C++ program. The text is: "Before convert:4.2", "After convert float to int:4", "Before convert :1", and "Covert int to float :1.666667".

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Before convert:4.2
After convert float to int:4
Before convert :1
Covert int to float :1.666667
```

Converting by using cast operator

- A cast operator is an unary operator with forces one data type to be converted into another data type.
- C++ supports **4-types of casting**
 5. **Static cast**
 6. **Dynamic Cast**
 7. **Const cast**
 8. **Reinterpret Cast**

```
#include<iostream.h>
#include<conio.h>
void main()
{
    clrscr();
    float pi=3.2;
    cout<<"\n Before convert:"<<pi+1;
    int s=static_cast<int>(pi);
```

```
cout<<"\n After convert float to int:"<<s;  
getch();  
}
```

Inheritance

- The capability of a class to **derive properties** and **characterstics** from **another class** is called **inheritance**.
- Inheritance is one of the mose important **feature** of **object-oriented programming**.
- Inheritance is a feature or a process in which , **new classes** are created from the **exisiting classes**.
- The **new class** created is called "**derived class**" or "**child class**" and the exisiting class is known as the "**base class**" or "**parent class**".
- The derived class now is said to be inherited from the base class.

Sub class(derived class) : The class that inherits properties from another class is called **subclass or derive class**.

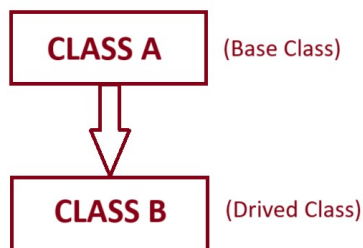
Super class(Base class): the class whose properties are inherited by a subclass is called **Base class (or) Super class**.

Types of inheittance

1. Single inheritance
2. Multilevel inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid Inheritance

Single inheritance

- In single inheritance , A class is allowed to inherit from only one class.
- i.e: one subclass is inherited by one base class only.



Syntax

```
class superclass_name
{
// body of super_class
}

class subclass_name : access_mode Superclass_name
{
//body of subclass and Super_class_properties
}
```

Example:

```
#include<iostream.h>

#include<conio.h>

class A
{
public:
float pi;
void pi_value()
{
pi=3.14;
}
};

class B:public A
{
public:
float ans,r;
void get()
{
cout<<"\n radius:";
```

```

cin>>r;

ans=pi*r*r; // here radius 'r' access from super_class(or) base class

cout<<"\n Area is:"<<ans;
}

};

void main()

{

clrscr();

B ob;

ob.pi_value();

ob.get();

getch();

}

```

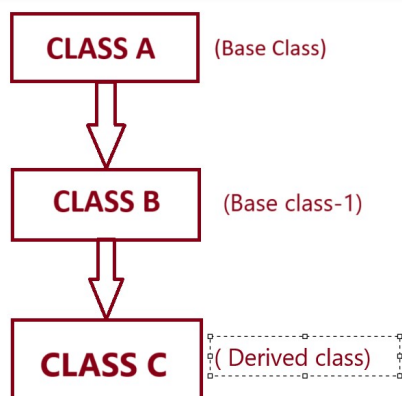
OUTPUT

Radius : 2

Area is : 12.56

MUTLILEVEL INHERITANCE

- In this type of inheritance , A derived class is created from another derived class.



Syntax

```
class A
{
// body of the class - A
}

class B: public A
{
// body of the class B and Class-A properties
}

class C:public B
{
// body of the class C and class-B properties
}
```

Example

```
#include<iostream.h>
#include<conio.h>

class A
{
public:
int side,area;
void side_of_square()
{
a=5;
area=4*a;
}
};

class B:public A
```

```

{
public:
int cir;
void circum()
{
cir=a*a; // here 'a' access from super_class(or) base class
}
};

class C: public B
{
public:
void display()
{
cout<<"\n area of square:"<<area; // here 'area' is acces from class 'A'
cout<<"\n circum of square:"<<cir; // here 'cir' is access from class 'B'
}
};

void main()
{
clrscr();

C ob;

ob.side_of_square();

ob.circum();

ob.display();

getch();
}

```

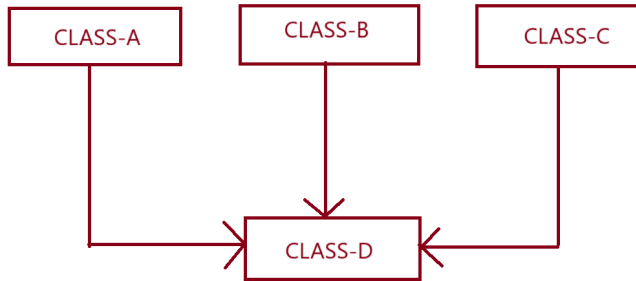
OUTPUT

Area of square is: 20

circum of square is: 25

MULTIPLE INHERITANCE

- Multiple inheritance is a feature of C++ where a class can inherit from **more than one class** i.e: one subclass is inherited from more than one base class.



Syntax

```
class A
```

```
{
```

```
// body of the class-A
```

```
}
```

```
class B
```

```
{
```

```
// body of the class-B
```

```
}
```

```
class C:public A, public B
```

```
{
```

```
// body of the class-C and properties of class A and B
```

```
}
```

Example

```
#include<iostream.h>
```

```
#include<conio.h>

class student
{
public:
char stname[30];
void get_student()
{
cout<<"\n Student name:";
cin>>stname;
}
};

class professor
{
public:
char pfname[30];
void get_prof()
{
cout<<"\n Staff name:";
cin>>pfname;
}
};

class college
{
public:
void welcome()
{
cout<<stname<<" and "<<pfname<<" welcome to our college";
// here stname and pfname access from class student and professor
```

```
}  
};  
  
void main()  
{  
clrscr();  
college clg;  
clg.get_student();  
clg.get_prof();  
clg.welcome();  
getch();  
}
```

OUTPUT

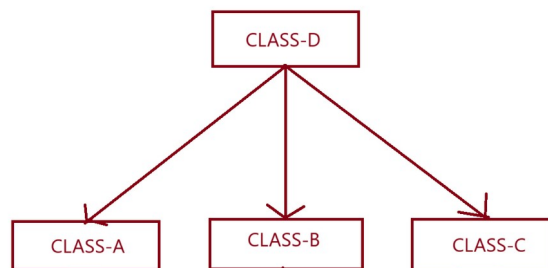
Student name: John

Staff name: David

john and David welcome to our college

Hierarchical inheritance

- In this type of inheritance , more than one subclass is inherited from a single class i.e: more than one derived class is created from a single class.



Syntax

class A

{

```
// body of class-A
}

class B: public A
{
// body of class-B and properties of class-A
}

class C: public A
{
// body of class-C and properties of class-A
}
```

Example

```
#include<iostream.h>
#include<conio.h>
class numbers
{
public:
int a,b;
numbers()
{
a=5;
b=2;
}
};

class Addition:public numbers
{
public:
int c;
```

```

void sum()
{
c=a+b; // here 'a' and 'b' access from class 'numbers'
cout<<"\n Addition of two numbers is:"<<c;
}
};

class subtraction:public numbers
{
public:
int c;
void sub()
{
c=a-b; // here 'a' and 'b' access from class 'numbers'
cout<<"\n Subtraction of two numbers is:"<<c;
}
};

void main()
{
clrscr();

Addition ad;

subtraction sb;

ad.sum();

sb.sub();

getch();
}

```

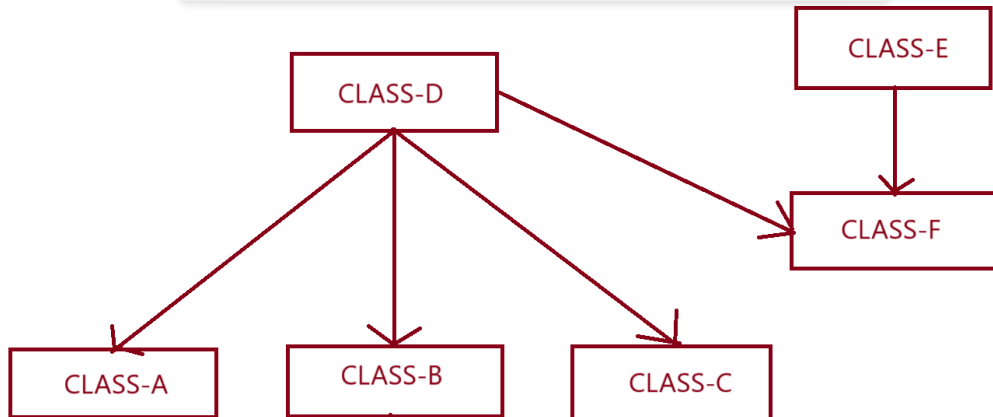
OUTPUT

Addition of two numbers is: 7

subtraction of two numbers is:3

Hybrid inheritance

- Hybrid inheritance is implemented by combining more than one type of inheritance.
- For Example: **Combining Hierarchical and multiple inheritance** is called **Hybrid inheritance**.



Syntax

```
class A
{
// body of class-A
}

class B: public A
{
// body of class-B and properties of class-A
}

class C: public A
{
// body of class-C and properties of class-A
}

class D
```

```
{  
// body of class-D  
}  
  
class E:public A , public D  
{  
// body of class-E and properties of class A and D  
}
```

VIRTUAL BASE CLASS

- Virtual base classes are used to **virtual inheritance** in a way of preventing multiple "**instance**" of a given class appearing in an inheritance hierarchy when using multiple inheritance.

Syntax

```
class B: public virtual A
```

```
{
```

```
};
```

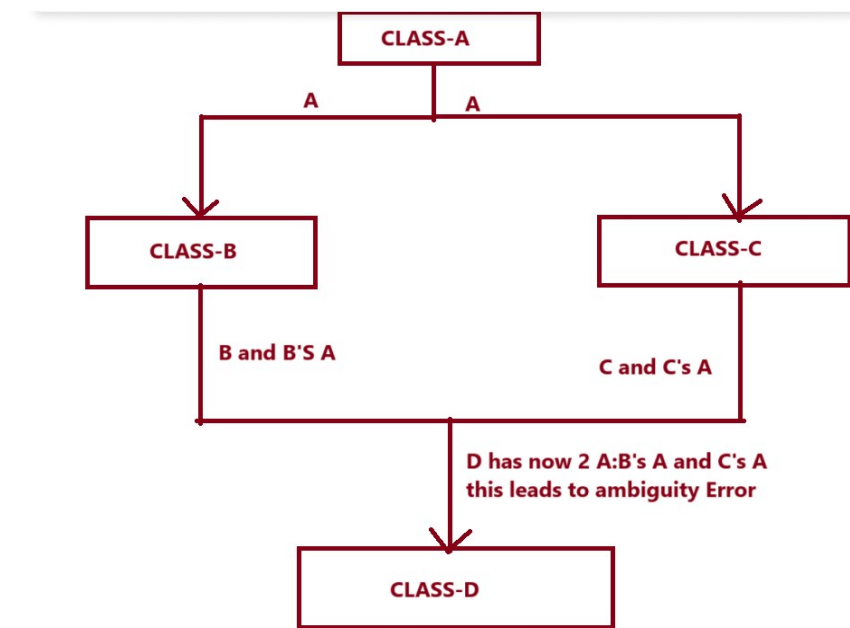
```
class C:public virtual A
```

```
{
```

```
};
```

Need for Virtual Base classes:

- Consider the situation where we have one class A.
- A is inherited by two other classes B and C.
- Both these classes are inherited into another in a new class D as shown in figure below.



- As we can see from the figure that data **member/functions** of class A are inherited twice to class D.
- One through class B and second through class C.

- When any data/function member of class A is accessed by an object of class D, Ambiguity arises as to which data/function member would be called? one inherited through B or the other inherited through C. This confuses compiler and it displays error.

Program-1: Twice copies for same base class

```
#include<iostream.h>

#include<conio.h>

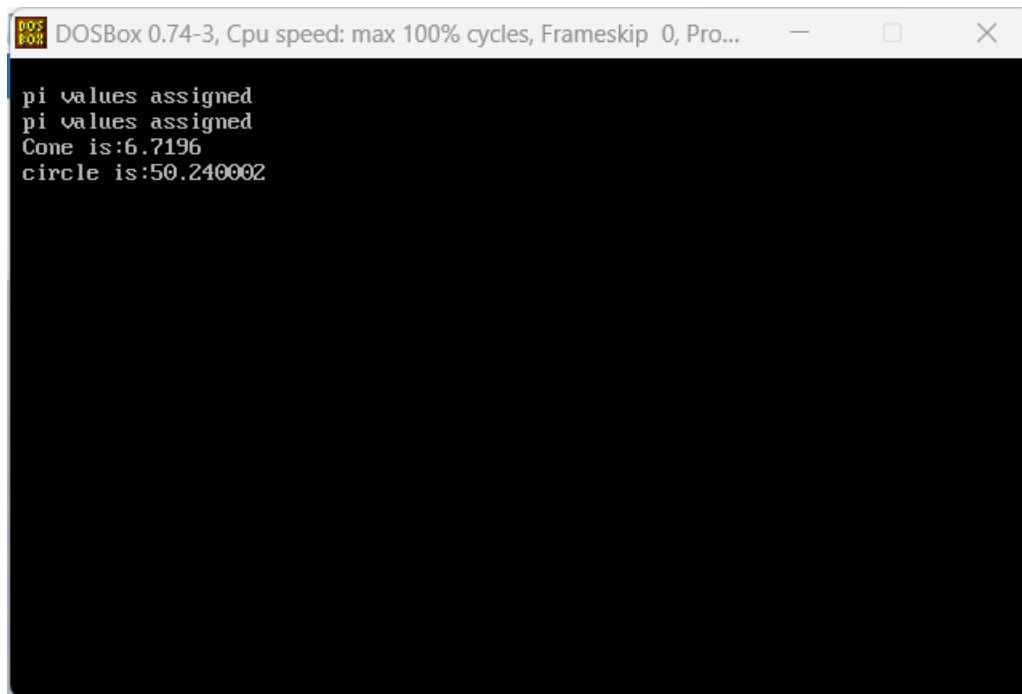
class shapes
{
public:
float pi;
shapes()
{
pi=3.14;
cout<<"\n pi values assigned";
}
};

class cone:public shapes
{
public:
float ans1;
void area()
{
ans1=pi*2.14;
}
};

class circle:public shapes
```

```
{  
public:  
float ans2;  
void area2()  
{  
ans2=pi*4*4;  
}  
};  
class result:public cone,public circle  
{  
public:  
void disp()  
{  
cout<<"\n Cone is:"<<ans1;  
cout<<"\n circle is:"<<ans2;  
  
}  
};  
void main()  
{  
clrscr();  
result cc;  
cc.area();  
cc.area2();  
cc.disp();
```

```
getch();  
}
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The window contains a black background with white text output from a program. The text is as follows:

```
pi values assigned  
pi values assigned  
Cone is:6.7196  
circle is:50.240002
```

in above output class **Shapes** are inherited twice to class **result**. one through class **circle** and another through **cone**

Program-2 : using Virtual keyword

```
#include<iostream.h>  
  
#include<conio.h>  
  
class shapes  
{  
  
public:  
  
float pi;  
  
shapes()  
{  
  
pi=3.14;  
  
cout<<"\n pi values assigned";
```

```
}  
};  
  
class cone:public virtual shapes  
{  
public:  
float ans1;  
void area()  
{  
ans1=pi*2.14;  
}  
};  
  
class circle:public virtual shapes  
{  
public:  
float ans2;  
void area2()  
{  
ans2=pi*4*4;  
}  
};  
  
class result:public cone,public circle  
{  
public:  
void disp()  
{
```

```
cout<<"\n Cone is:"<<ans1;

cout<<"\n circle is:"<<ans2;

}

};

void main()

{

clrscr();

result cc;

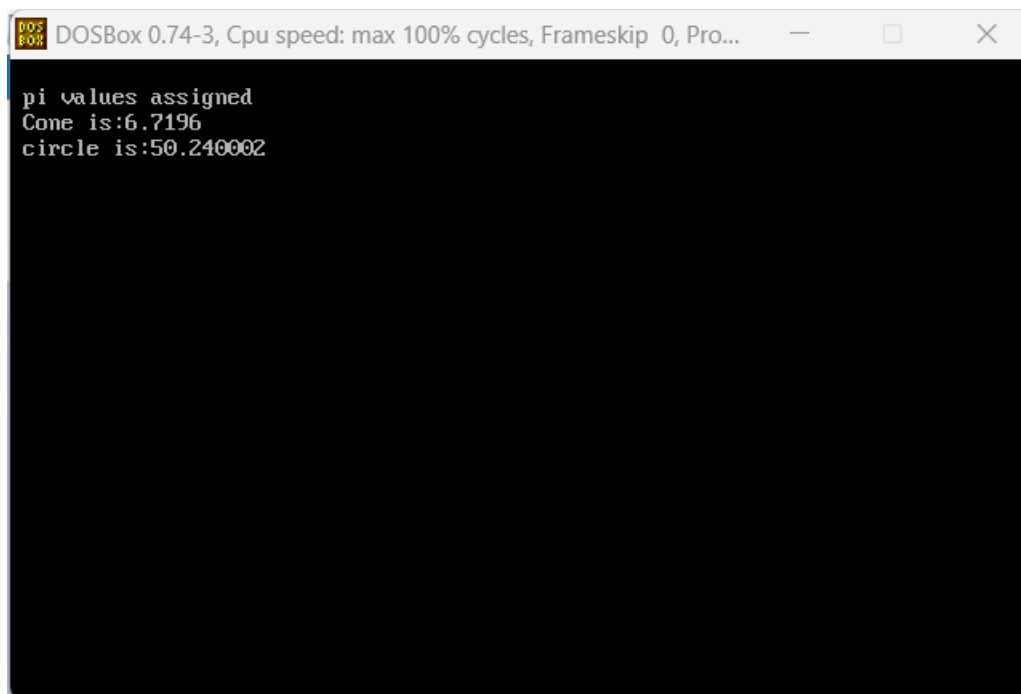
cc.area();

cc.area2();

cc.disp();

getch();

}
```



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...

pi values assigned
Cone is:6.7196
circle is:50.240002
```

- The class **Shapes** has just one data member a which is public.

- This class is virtually inherited in class **cone** and class **circle** .
- Now class **cone and circle** use the **virtual base class** , **Shapes** and no duplication of data member **a** is done.
- Classes **cone and circle** share a single copy of the members in the **virtual base class Shapes**.

ABSTRACT CLASS

- Sometime implementation of all function can't be provided in a base class because we don't know the implementation. Such a class is called an **abstract class**.
- **For Example:** A class **features** having two functions **fly()** and **takeoff()** that two functions doesn't have the implementation (Assume Flight and bird can have both features) but both classes know how to fly and take_off. we can't create object for Abstract classes.
- A **pure virtual function** (or **Abstract function**) in C++ is a virtual function for which we can have an implementation. But we must override that function in the **derived class**. otherwise the derived class will also become an **abstract class**.
- A pure virtual function is declared by assigning 0 in the declaration.

// An Abstract class

```
class Test
{
// Data member of class

public:

//pure virtual function

virtual void show()=0;

/* other member functions */

};
```

- A **pure virtual function** is implemented by classes that are derived from an Abstract class.
- if declare a **member function** as **virtual keyword** is called **pure virtual function**.
- if you not implementation a pure virtual function in **derived class**, Compiler leads error because every **pure virtual function** must be implemented in **derived class** is called **pure virtual function**.

Program for pure virtual function

```
#include<iostream.h>

#include<conio.h>

class features
{
```

```

public:

// pure virtual function

virtual void takeoff()=0;

virtual void fly()=0;

//Non-pure virtual function

void display()

{

cout<<"\n I am Optional";

}

};

class airplane:public features

{

public:

void takeoff()

{

cout<<"\n flight will take few minutes for take off";

}

void fly()

{

cout<<"\n flight can fly over the river";

}

};

class bird:public features

{

public:

```

```

void takeoff()
{
    cout<<"\n birds will take few seconds for take off";
}

void fly()
{
    cout<<"\n birds can fly only their living area";
}
};

```

```

void main()
{
    clrscr();

    airoplane ob;

    ob.takeoff();

    ob.fly();

    bird b;

    b.takeoff();

    b.fly();

    getch();
}

```

- In above program class **Flight and bird** having two features like **fly and takeoff** .
- The above two functions are derived from the class **features** .
- class Flight is implementing the features like fly() and takeoff() function from **features** class is called **pure virtual function**.
- in above example **features** class is also having another method is called **display()** , it is not a pure

virtual function , it not necessary to implmenting in **derived class** because it is normal function.

- A class is abstract if it has at least one **pure virtual function**.
- if we do not override the pure virtual function in the derived class. then the derived class also become an abstract class.

POINTER

- A pointer is a memory variable that stores a memory address.
- Pointer can have any name that is legal for other variables, and it is declared in the same fashion as other variables, but it is always denoted by "*" operator.
- Pointers save memory space.
- Execution time with pointers is faster , because data are manipulated with the address , that is , direct access to memory location.
- Memory is accessed efficiently with the pointers. the pointer assigns as well as release the memory space. Memory is dynamically allocated.
- Pointers are used to with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.
- We can access the elements of any type of array , irrespective of its subscript range.
- Pointers are used for file handling.
- pointers are used to allocate memory in a dynamic manner.
- In C++. a pointer declared to a base class could access the object of a derived class , However a pointer to a derived class cannot access the object of a base class.
- The compiler will generate an error message "**cannot convert**" **A* to B*** , where A is the base class and B is the derived class.

Pointer Declaration

- Pointer variables can be declared

Example

```
int *x ;
```

```
float *f;
```

```
char *y ;
```

- In the first statement , "**X**" is an integer pointer, and it informs the compiler that it holds the address of any integer variable.
- In the same way "**F**" is a float pointer that stores the address of any float variable, and "**y**" is a character pointer which stores the address of any character variable.
- A pointer variable stores stores the memory address of any type of variable .

- The pointer variable and normal variable should be of the same type.
- The pointer is denoted by * asterisk symbol
- Address denoted by & (ambersand) symbol

Write a program to display the address of the variable

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

int n;

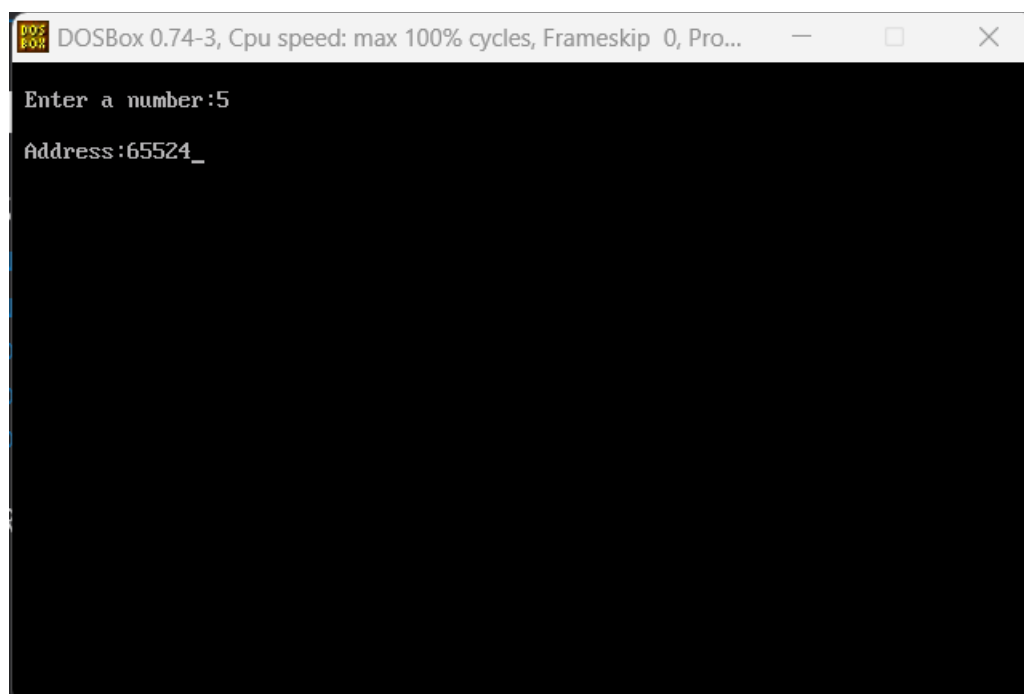
cout<<"\n Enter a number:";

cin>>n;

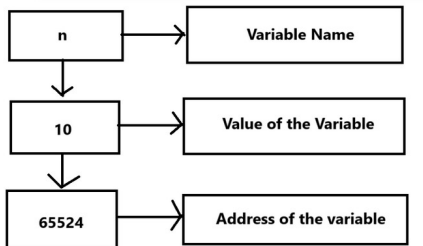
cout<<"\n Address:"<<(unsigned)&n;

getch();

}
```



Explanation



- The memory location of a variable is system dependent . Hence the address of a variable cannot be predicted immediately
- In the above example , the address of the variable 'n' that is observed is **65524** .
- The first block contains the variable name .
- The second block represents the value of the variable
- The third block is the address of the variable 'n' , where **10** is stored , Here 65524 is the memory address.
- The address of the variable depends on various things , for **instance** , **memory model** , **address scheme**, and **present system settings**.

Write a program to declare pointer variable and display the value and address of the variable using pointer

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

int n=10;

int *x;

x=&n;

cout<<"\n value of n is:"<<n;

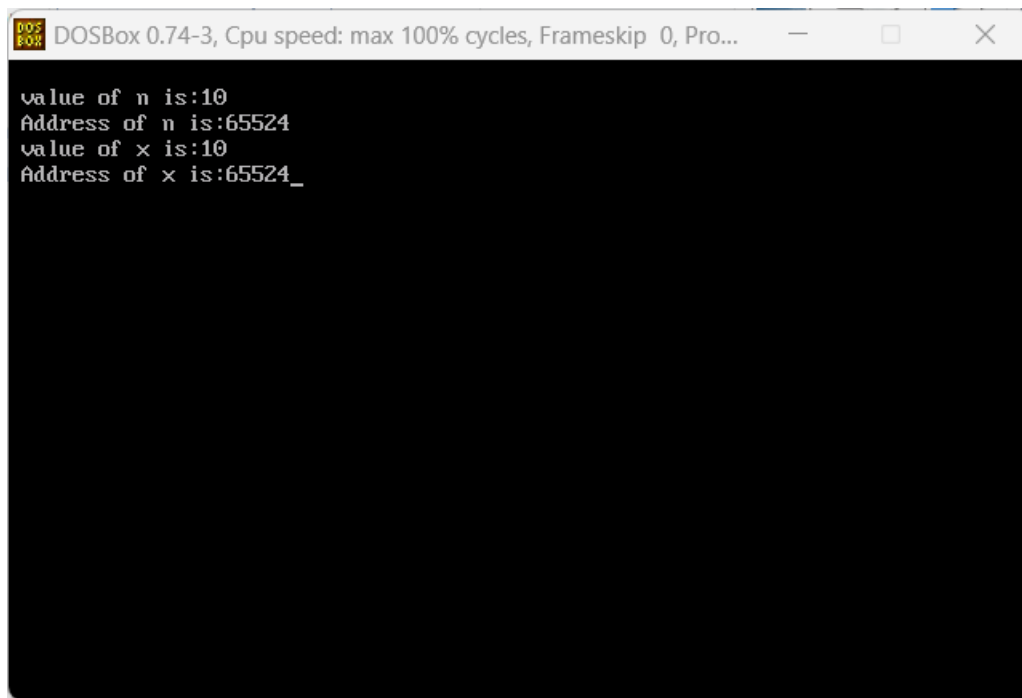
cout<<"\n Address of n is:"<<(unsigned)&n;

cout<<"\n value of x is:"<<*x;

cout<<"\n Address of x is:"<<(unsigned)x;
```

```
getch();
```

```
}
```



DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...

```
value of n is:10  
Address of n is:65524  
value of x is:10  
Address of x is:65524_
```

Pointer to Class, Object

- We know that a pointer is a variable that holds the address of another data variable.
- The variable may be of any data type , that is **int , float m or double .**
- In the same way , we can also define **pointer to class .** Here , the starting address of the member variable can be accessed , thus such pointer are called **class pointers**

Example

```
class book
{
    char name[30];
    int pages;
};

(a) class book *ptr;

(or)

struct book *ptr;
```

in the above example , ***ptr** is a pointer to a class book , both statements (a) and (b) are alid the syntax for using pointes with members is as given below

1) **ptr --> name** 2) **ptr --> author** 3) **ptr --> pages**

by executing thses three statements , the starting address of each member can be estimated.

Pointer to Object

Similar to variable , objects also have on address . A pointe can point to a specified object

Write a program to delcare an object and pointer and class

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class man
```

```
{
```

```
public:
```

```
char name[20];
```

```
int age;
```

```
};
```

```
void main()
```

```
{
```

```
clrscr();
```

```
man m={"krish",34};
```

```
man *ptr;
```

```
ptr=&m;
```

```
cout<<"\n name:"<<m.name;
```

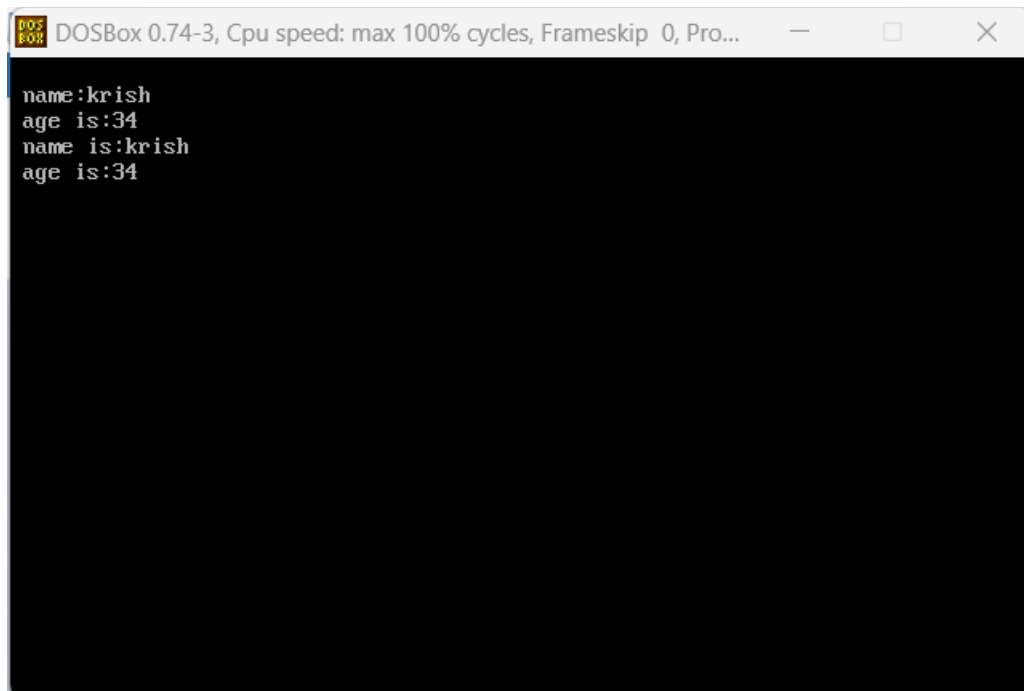
```
cout<<"\n age is:"<<m.age;
```

```
cout<<"\n name is:"<<ptr->name;
```

```
cout<<"\n age is:"<<ptr->age;
```

```
getch();
```

```
}
```



this pointer

- Objects are used to invoke the **non-static** member function of the class.
- For example: if p is an object of class P and get() is a member function of P , the statement p.get() is used to call the function
- The statement p.get() operates on p . In the same way , if **ptr** is a pointer to a P object , the function called **ptr ->get()** operates on *ptr.

Write a program to enter name and age of two persons

```
#include<iostream.h>

#include<conio.h>

class employee
{
public:
char *str;

int age;

void get(char *str,int age)
{
this->str=str;

this->age=age;

cout<<"\n name:"<<this->str;

cout<<"\n age:"<<this->age;

}

};

void main()
{

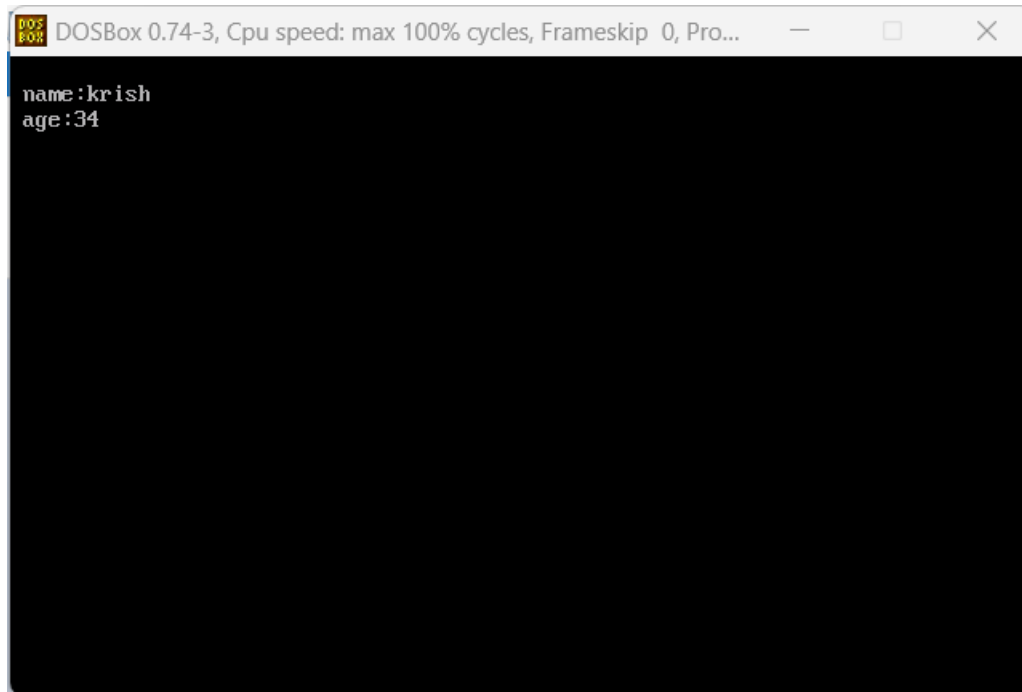
clrscr();

employee e1;
```

```
e1.get("krish",34);
```

```
getch();
```

```
}
```



Pointer to Derived class and Base class

- It is possible to declare a pointer that points to the base class as well as the derived class.
- One pointer can point to different class. **For Example:** X is a base class and Y is a derived class .
- The pointer pointing to **X** can also point to **Y**.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class A
```

```
{
```

```
public:
```

```
int b;
```

```
void display()
```

```
{
```

```
cout<<"\n B is "<<b;
```

```
}
```

```
};
```

```
class B:public A
```

```
{
```

```
public:
```

```
int d;
```

```
void display()
```

```
{
```

```
cout<<"B is "<<b<<"\n d is:"<<d;
```

```
}
```

```
};
```

```
void main()
```

```

{
clrscr();

A *cp;

A base;

cp=&base;

cp->b=100;

cp->display();

//cp->d=200; it is not accessible

```

```

B b;

cp=&b;

cp->b=150;

cp->display();

//cp->d=100; is not accessible

getch();

}

```

Explanation

- In the above program , A and B are two classes with a one-integer member variable and member function.
- The class **B** is derived from class A . The pointer **cp** points to the class A.
- The variable base is an object of the class A.
- The address of the object base is assigned to pointer **cp**.
- The pointer **cp** can access the member variable **b** , a member of the base class , but **cannot** access the variable **d** , a member of the derived class

A) cp->d=200; The variable **b** is an object of class **B**. the address of the object **b** is assigned to the pointer **CP** , However , **b** is an object of the derived class ; access to the member variable **d** is not possible

B) cp --> d=300 , here **cp** is a pointer to the base class , and it cannot access the member of the

derived class.

ARRAY and Characterstics Of Array

- An array is a very popular , linear , homogenous , and useful data structure that is used to store similar types of data elements in contiguous memory locations under one variable name.
- An array can be declared to be of any standard or custom data type.
- The array of character(strings) type works somewhat differently from the array of integer . flating numbers and so on.

One-Dimentional-Array

- The declaration of a one-dimentional array is as follows

Declaration

```
int a[5];
```

- It tells the compiler that 'a' is an integer type of array , and it should store five **integer**.
- In the Example : **5** is the subscript enclosed within square brackets.
- The compiler reserves two bytes of memory for each integer array element; that is , 10 bytes are reserved for storing five integers as the memory
- In the same way arrays of different data types are declared as follows

One-Dimensional Array declaration

```
char name[20];
```

```
float marks[10];
```

```
long num[10];
```

The array initialization is done as follows

Array initialization

```
int a[5]={1,2,3,4,5};
```

- Here , five elements are stored in an **array** 'a' , the array elements are stored sequentially in separate locations .then the question arises of how to call each element individually from this bunch of integer elements.
- The reading of array elements begin from **zero**.
- Array elements are accessed with the name of the array , and the number within the square brackets specifies the elements number .

- In other words , array elements are called with array names followed by element number.

Calling Array Elements

a[0] refers to 1st element i,e:1

a[1] refers to 1st element i,e:2

a[2] refers to 1st element i,e:3

a[3] refers to 1st element i,e:4

a[4] refers to 1st element i,e:5

Characterstice of Arrays

1. The Declaration **int a[5]** is nothing but a creation of five variables of integer types. instead of declaring five variables for five values.
2. All the elements of an array share the same name , and they are distinguished from one another with the help of an element number.
3. The element number in an array plays a major role for calling each element
4. Any particular element of an array can be modified separately without disturbing the other elements

for example : int a[5]={1,2,3,4,8};

if the programmer needs to replace **8** with **10** , he/she is not required to change all the other elements except **8** . To carry out this task , the statement **a[4]=10** can ne used .Here , the other four elements are left unchanged.

5. Any element of an array **a[]** can be assigned/equated to another ordinary variable or an array variable of its type.

For example : b=a[2];

a[2]=a[3] ;

(a) In the statement **b=a[2]** or vice versa , the value of **a[2]** is assigned to "**B**" where "**b**" is an integer.

(B) In the statement **a[2]=a[3]** or vice versa , the value of **a[3]** is assigned to **a[2]** , where both the elements are of the same data type

6. The array elements are stored in contiguous memory locations , The amount of storage required for holding the elements of the array depends on its type and size.The toal size in bytes for a single-dimensional array is a computed

Total_bytes = size of(data type) x size of array

Write a program to read 5-integer through keyboard and display them

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr() ;

int num[5],i;

for(i=0;i<5 ; i++)

{

cout<<"\n Enter any NO:";

cin>>num[i];

}

cout<<"\n Numbers are:";

for(i=0;i<5;i++)

{

cout<<num[i];

}

getch();

}
```

OUTPUT

Enter any no: 12

Enter any no: 34

Enter any no: 45

Enter any no: 67

Enter any no: 89

Enter any no: 12 34 45 67 89

Two - Dimentional Arrays

- Two dimentional arrays can be considered a rectangular display of elements with **row** and **columns**, and this is also known as a **matrix** .consider the following example `int x[3][3]` .
- The two-dimentional array can be delcared as below

Array of two-dimentional Array elements

	column 0	column-1	column2
row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>

Write a program for two dimentional array

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

int i,j;

int a[3][3]={1,2,3,4,5,6,7,8,9};

for(i=0;i<3;i++)

{

for(j=0;j<3 ;j++)

{

cout<<"\t  "<<a[i][j];

if(i==2)

break;

cout<<"\n Row:<<i+1;
```

```
}  
getch();  
}
```

OUTPUT

Row0	1	2	3
Row1	4	5	6
Row2	7	8	9

Array of Classes

- As we know , an array is a collection of similar data types .
- In the same way , we can also define an array of classes.
- In such a type of an array , every element is of class type.
- An array of class objects can be declared as follows

```
class stud
{
public:
char name[30];
int rollno;
char grade[2];
};

void main()
{
clrscr();

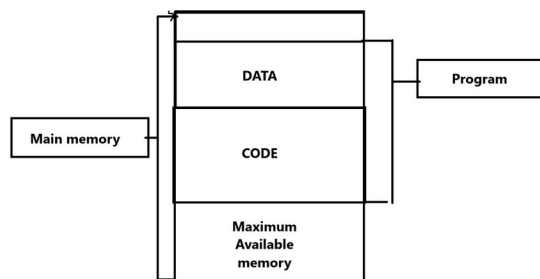
stud st[3] ; // declaration of array of class object
```

- in above example , st[3] is an array of three elements containing three objects of class stud.
- Each elements of **st[3]** has its own set class of member variables ; that is **char name[21]** int **rollno**, and **char grade[2]** .

Write program for Array of class (same as Array of object) we already discussed program for it in the unit-1

Memory Models

- The memory model sets the supportable **size of code** and **data area**.
- We need to specify an appropriate memory model before **compiling and linking** the **source code**.
- Using memory model we can set the **size** limits of the data and code.
- C++ program always use different segments for code and data.
- The memory model your option for decides the default method of memory addressing , and the default memory model is small.



Tiny : Use the tiny model when memory is at an absolute premium . All four segment registers (CS,DS , ES , and SS) are initialized with the same address, and all addressing is accomplished using 16 bits .

small: All codes should fit into a single 64-KB segment , and all data should fit into a second 64-KB segment . All pointers are 16 bits in length.Execution speed is similar to that in the **tiny** model.

Medium : All data should fit into single 64-KB segment . However the code is allowed to dresses. With this the access to data is fast.However , slower program execution is observed with this model.This model is suitable for big programs that do not store a large amount of data in memory

Compact : All codes should fit into a 64KB segment , but the data can be use multiple segments , However , no data item can surpass 64KB . All pointers to data are 32 bits, but jumps and calls can use 16-bit address.

Large : Both code and data are allowed to use multiple segments. All pointers are 32-BITs in length. However , no single data item can exceed 64-KB . Code execution is slower.

Huge: Both code and data are allowed to use multiple segments. Every pointer is 32 bits in length and it is different from large memory model only vy pointer math and not by segments.

Segments and offset Address

- Every address has two parts : segment and offset . we can separate these address parts using the following two macros defined in **dos.h** header file.
- **FP_SEG()** - This macros is used to obtain the segment address of the given pointer varaible.
- **FP_OFF()** - This macro is used to obtain the offset address of the given pointer variable.

Write a program to obtain segment and offset address

```
#include<iostream.h>

#include<conio.h>

void main()
{
clrscr();

int ch;

cout<<"\n Complete address:"<<&ch;

cout<<"\n Segment address:"<<FP_SEG(&ch);

cout<<"\n Offset address:"<<FP_OFF(&ch);

getch();
}
```

OUTPUT

Complete Address: 0x8f34ff2

Segment address: 8f34

Offset address: fff2

Dynamic memory Allocation (new , delete)

- The technique by which a program can obtain storage space in the main memory at **run time** is called **dynamic allocations**.
- In this method , the space for the program is allocated from the free space during the execution of the program.
- Operators **new** and **delete** that can be used for acheiving dynamic memory allocation and deallocation , respectively.

THE **new** AND **delete** OPERATORS

- The **new** operator not only creates an object but also allocated memory.
- The **new** operator allocates memory from the **heap** that is also called a free store.
- The object created and the memory allocated by using the **new** operator should be deleted and memory should be released by the delete operator , otherwise , such a mismatch operation may corrupt the heap or may crash the system, accroding to the ANSI standard.
- The **delete** operator not only destroy not only destroys objects but also releases allocated memory.
- The **new** operator creates an objects , and it remains in the memory until it is released using the **delete** operator. Sometimes, the object deleted using the **delete** operator remains in the memory.
- If we send a null pointer to the **delete** operator, it is secure. Using delete to zero has no result.
- The statement **delete X** does not destroy the pointer X, it destroys the objecct associated with it.
- Do not apply C function such as **malloc()** , **realloc()** , or **free()** with new and delete operators . These functions are until for object - oriented techniques
- Do not destroy the pointer repetitively or more than once. First time , the object is destroyed and memory is released . If second time the same object is deleted , the object is sent to the destroy the object and release the system resources.
- If the object created is not deleted. It occupies the memory unnecessarily . It is good habit to destroy the object and release the system resources.

new	malloc()
creates objects	Allocated memory
returns pointer or relevant type	Returns void pointer

It is possible to overload a new op malloc() cannot be overloaded

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

int i,*p;

p=&i;

p=new int[3];

*p=2;

*(p+1)=3;

*(p+2)=4;

cout<<"\n Value address:";

for(int x=0;x<3;x++)

cout<<"\n "<<*(p+x)<<"\t"<<(unsigned)(p+x);

delete []p;

getch();

}
```

output

value	addres
2	3350
3	3352
4	3354

Dynamic Object

- **C++** allocates memory and initializes the member variables . An object can be created at run time; such an object is called a **dynamic object**.
- The **constructor and destructor** of the dynamic object is explicitly done by the programmer.
- The **new and delete operators** are used to allocate and deallocate memory to such object .
- A dynamic object can be created using the **new** operator as follows:

ptr = new class_name ;

- The new operator returns the address of the object created, and it is stored in the pointer **ptr**.
- The variable **ptr** is a pointer object of the same class.
- The member variable of the object can be accessed using the pointer and -> (arrow) operator.
- A dynamic object can be destroyed using the **delete** operator as follows

delete ptr;

- The **delete** operator destroys the object pointed by the pointer **ptr**.
- It also invokes the destructor of a class.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class data
```

```
{
```

```
public :
```

```
int x,y;
```

```
data()
```

```
{
```

```
cout<<"\n Constructor";
```

```
x=10;
```

```
y=50;
```

```
}
```

```

~data()

{

cout<<"\n Destructor";

}

void display()

{

cout<<"\n X="<<x;

cout<<"\n Y="<<y;

}

};

void main()

{

clrscr();

data *d;

d=new data;

d->display();

delete d;

getch();

}

```

OUTPUT

Construtor

x=10

y=50

Destructor

Binding

- A program in C++ is executed sequentially line by line.
- Each line of the source program after translation is in the form of machine language .
- Each line in the machine language is assigned a unique address.
- Similarly , when a function call is encountered by the compiler during execution , the function call is translated into machine language, and a sequential address is provided.
- Thus **binding** refers to the process that is to be used for converting function and variable into machine language address.
- The C++ supports **two types of binding**

1 . static early binding

2. dynamic late binding

Static early binding

- In this section , we are going to focus on static or early binding.
- Even though similar function names are used at many places, their references and their positions are indicated explicitly by the compiler time.

```
class first
{
public:
int d;
void display()
{
-----// base class member function
}
};

class second : public first
{
public:
```

```
int k;

void display()

{

----- // member function of derived class

}

};
```

write a program for static binding

```
#include<iostream.h>

#include<conio.h>

class first

{

public:

int b;

first()

{

b=10;

}

void display()

{

cout<<"\n b "<<b;

}

};

class second : public first

{

public:
```

```

int d;

second()
{
d=20;
}

void display()
{
cout<<"\n d:"<<d;
}

};

void main()
{
clrscr();

second s;

s.first::display(); // invokes base class function

s.display(); // invokes derived class function

getch();
}

```

OUTPUT

b=10 ;

d=20

Dynamic (late) Binding

- In the case of a few programs , it is impossible to know which function is to be called until run time.
- This is called **dynamic binding** . Dynamic binding of member function in C++ can be done using

the **virtual** keyword.

- The member function followed by the **virtual keyword** is called a **virtual** function.

```
class first
{
public:
int d;
virtual void display()
{
-----// base class member function
}
};

class second : public first
{
public:
int k;
virtual void display()
{
----- // member function of derived class
}
};
```

Write a program for dynamic binding.

```
#include<iostream.h>

#include<conio.h>

float add(float m , float n)
{
```

```
return m+n;

}

float sub(float m,float n)

{

return m-n;

}

float mul(float m,float n)

{

return m*n;

}

void main()

{

clrscr();

float x,y;

cout<<"\n Enter 2 nos:";

cin>>x>>y;

int task;

do

{

cout<<"\n Enter task:";

cin>>task;

}

while(task<1 || task>3);

float (*pt)(float,float);

switch(task)
```

```
{  
case 1:  
pt=add;  
break;  
case 2:  
pt=sub;  
break;  
case 3:  
pt=mul;  
break;  
}  
cout<<"\n Result:"<<pt(x,y);  
getch();  
}
```

OUTPUT

Enter 2nos: 2.5 2.5

Enter task : 3

Result : 6.25

FILE HANDLING

- All programs we looked earlier
 1. Input data from the keyboard
 2. output data to the screen
- Output would be lost as soon as we exist from the program

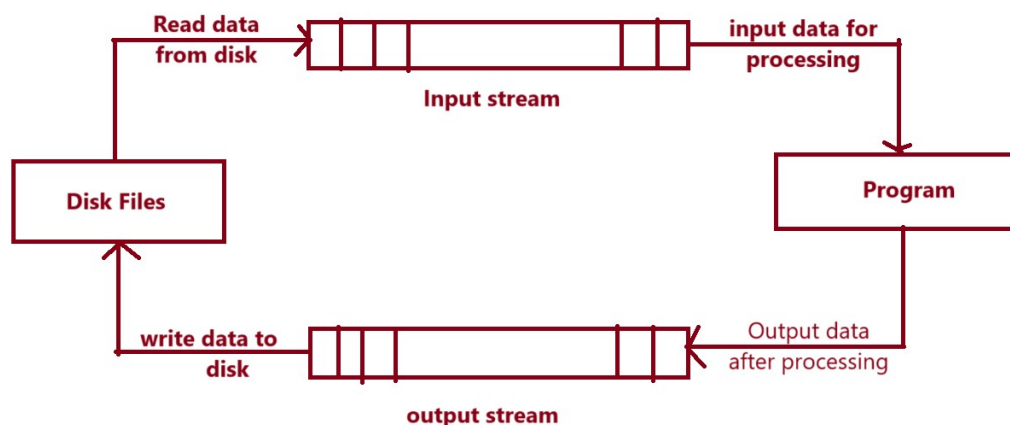
How do we store data permantly?

- We can use secondary storage devices.
- Data is packaged up on the storage device as data structures called files.

Stream Usage

- we have used streams already
 3. `cin>>` --> input from stream object connected to keyboard.
 4. `cout<<` --> Output to stream object conneted to screen.
 5. Can define other streams
- To or from files
- Used similarley as `cin` , `cout`

File Input and Output Stream



File Input Stream

reads data from disk file to the program.

File Output Stream

Writes data to the disk file from the program

I/O system of C++ contains

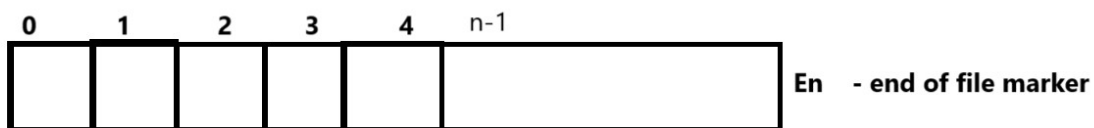
6. **ifstream** : provides input operations on files
7. **ofstream**: provides output operation on files
8. **fstream**: supports for simultaneous input and output operations files.

General File I/O steps

9. Declare a file name variables
10. associate the file name variable with the disk file name
11. open the file
12. use the file
13. close the files

File Stream will be in Memory

C++ Views file as sequence of bytes **Ends with** end of file marker



File Modes

1. **Write mode** (It only write the data into file , it override the previously stored data into as file)
2. **Append mode**(It read & Write data into a file)
3. **Read mode** (Read data from the file only)

Program-1 : Write the program for Write mode

```
#include<iostream.h>
```

```
#include<fstream.h>
```

```
#include<conio.h>
```

```
void main()

{

clrscr();

int pid;

char pname[20];

float rate;

cout<<"\n Enter PID and product name and Rate:";

cin>>pid>>pname>>rate;

ofstream ob;

ob.open("inventory.txt");

ob<<pid;

ob<<"\t";

ob<<pname;

ob<<"\t";

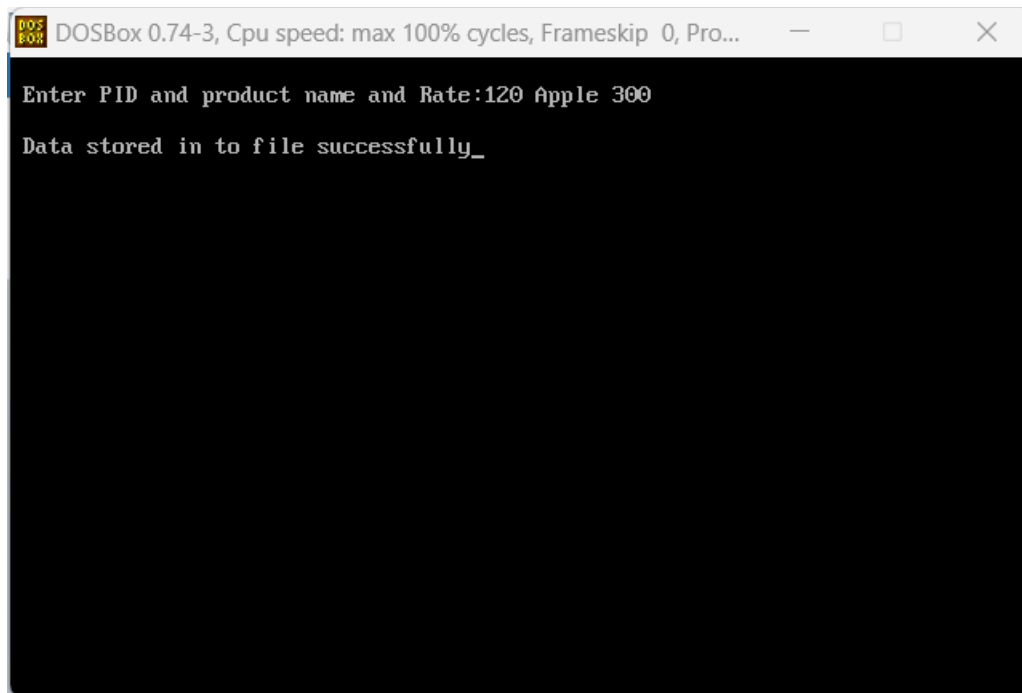
ob<<rate;

ob<<"\n";

cout<<"\n Data stored in to file successfully";

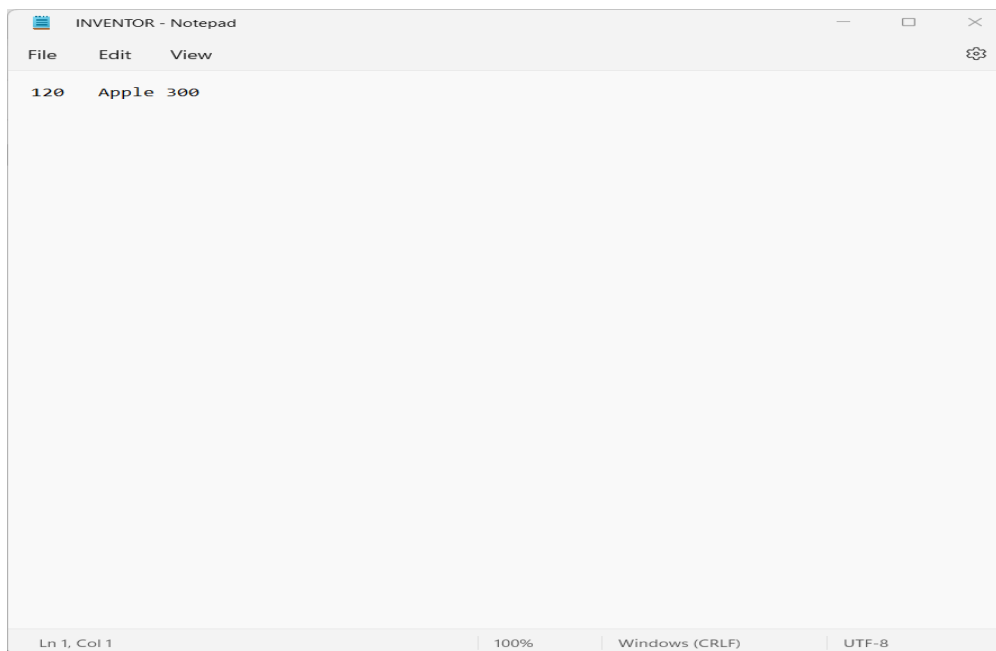
getch();

}
```



To Check the file location

press window+E (PC) -->local disk(C)--> TurboC3--> Bin--> inventory.txt



Program-2 : write a program for Append mode

```
#include<iostream.h>
```

```
#include<fstream.h>
```

```
#include<conio.h>

void main()

{

clrscr();

int pid;

char pname[20];

float rate;

cout<<"\n Enter PID and product name and Rate:";

cin>>pid>>pname>>rate;

ofstream ob;

ob.open("inventory.txt",ios::app);

ob<<pid;

ob<<"\t";

ob<<pname;

ob<<"\t";

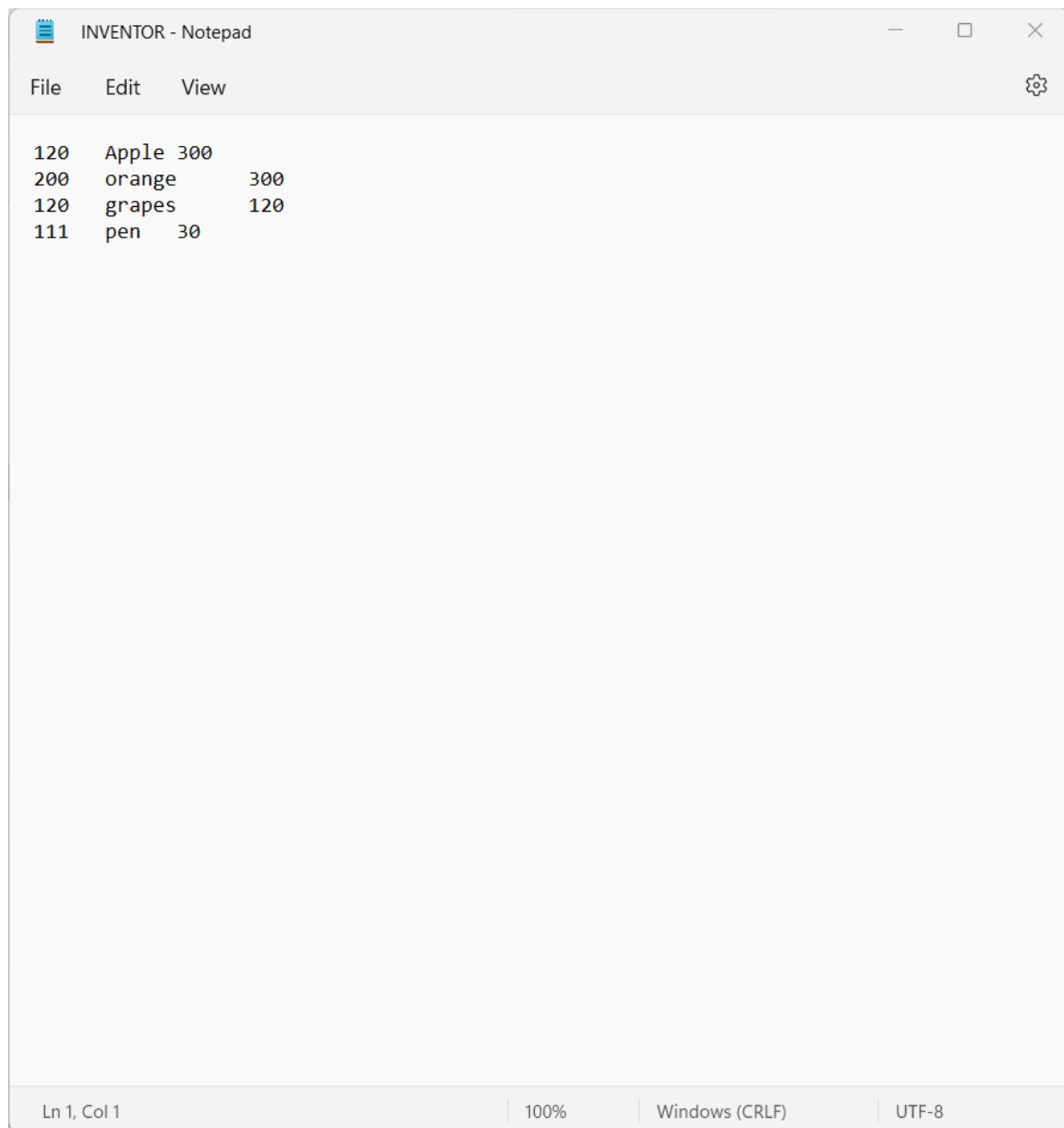
ob<<rate;

ob<<"\n";

cout<<"\n Data stored in to file successfully";

getch();

}
```



The screenshot shows a Notepad window with the title 'INVENTOR - Notepad'. The menu bar includes 'File', 'Edit', and 'View'. The text content is as follows:

120	Apple	300
200	orange	300
120	grapes	120
111	pen	30

The status bar at the bottom indicates 'Ln 1, Col 1', '100%', 'Windows (CRLF)', and 'UTF-8'.

Program-3 : write a program for read mode

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<fstream.h>
```

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
clrscr();

fstream fp;

char fname[30],str[100];

int i=0;

cout<<"\n Enter file name to open";

gets(fname);

fp.open(fname,fstream::in);

fp.getline(str,1000);

while(strlen(str)>0)

{

cout<<str<<"\n";

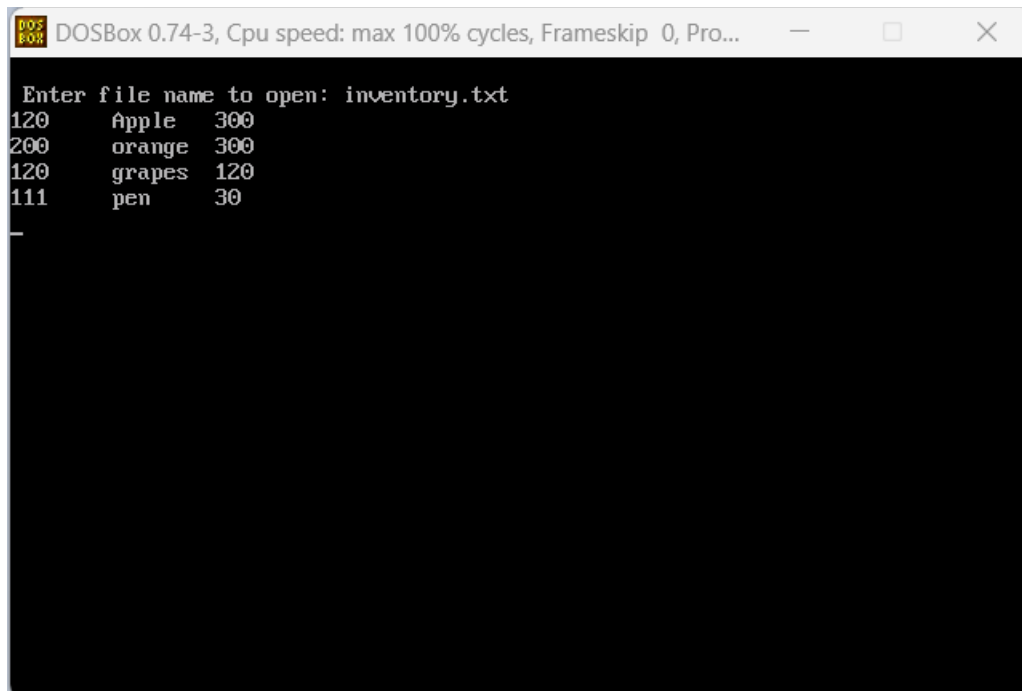
fp.getline(str,1000);

}

fp.close();

getch();

}
```



DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...

```
Enter file name to open: inventory.txt
120  Apple  300
200  orange 300
120  grapes 120
111  pen    30
-
```

Sequential Read/Write operations

- C++ allows the **file** manipulation command to access the file sequentially or randomly.
- The data of the sequential file should be accessed sequentially , that is , one character at a time.
- In order to access the **nth** number of **bytes**, all previous characters are **read and ignored**.
- There are number of functions to perform **read and write** operations with the files.
- Some functions **read/write** single characters and some functions **read/write** a single characters, and some functions **read/write** blocks of binary data.
- The **put()** and **get()** functions are used to **read or write** a **single** character , whereas **write()** and **read()** are used to read or write blocks of binary data.

put() - function

- The function **put()** to writes a character to the specified file by the stream object.
- It is also member of the **fstream** class.
- The **put()** function places a character in the file indicated by the put pointer.

get() function

- The function **get()** is a member function of the class **fstream**.
- This function reads a single character from the file pointed by the get pointer, that is , the character at the current get pointer position is caught by the **get()** function

Write a program to write and read string to the file using put() and get() functions

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<fstream.h>
```

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
fstream fp;
```

```
char str[100];

int i=0;

char ch;

cout<<"\n Enter A Text:";

cin.getline(str,50);

fp.open("krish.txt",ios::in | ios::out);

while(i[str]!='\0')

{

fp.put(str[i++]);

}

fp.seekg(0);


cout<<"\n Entered Text :";

while(fp)

{

fp.get(ch);

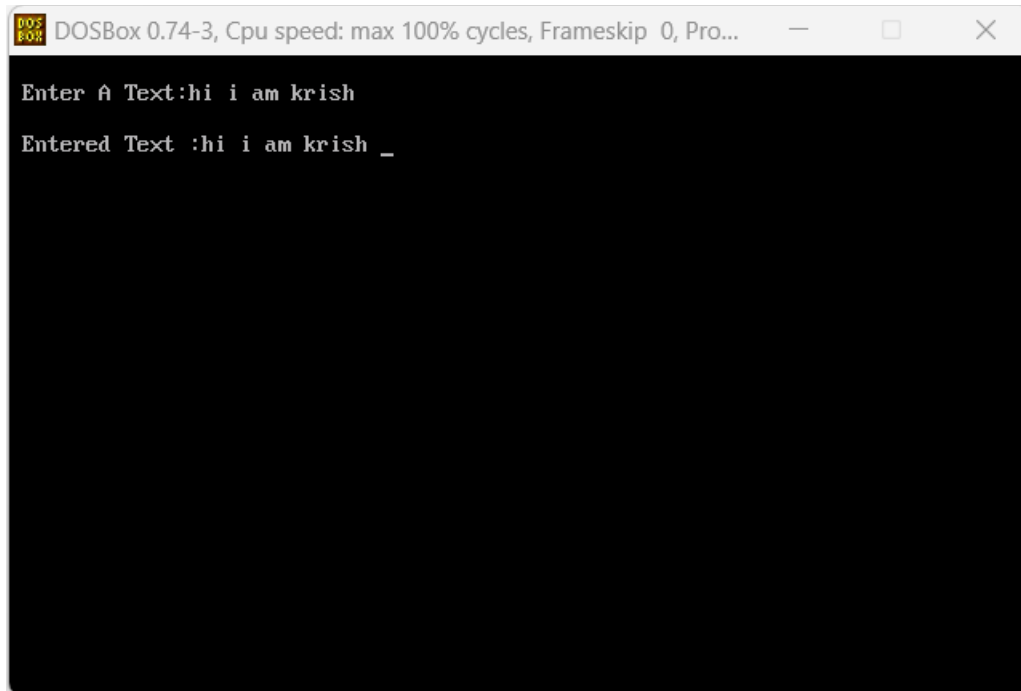
cout<<ch;

}

fp.close();

getch();

}
```



read() and write() operators

- The **read()** and **write()** functions perform read and write operations in a binary format that is exactly the same as an internal representation of data in the computer.
- Due to the capabilities of these functions , large data can be stored in a small amount of memory.
- Both these functions are also used to write and read class object to and from files.

Write a program for binary file using write()

```
#include<iostream.h>

#include<conio.h>

#include<fstream.h>

void main()

{

clrscr();

int num[]={100,120,130,140,150,160};

ofstream out;

out.open("one.bin");
```

```
out.write((char *) &num,sizeof(num));

out.close();

cout<<"\n file writed as binary using write()";

getch();

}
```

Program to read a character from binary file using read()

```
#include<iostream.h>

#include<conio.h>

#include<fstream.h>

void main()

{

clrscr();

int num[90];

ifstream in;

in.open("one.bin");

in.read((char *) &num,sizeof(num));

for(int i=0;i<6;i++)

{

cout<<"\n"<<num[i];

}

getch();

}
```


Binary And ASCII files

- Binary file cannot easily be transferred from one computer system to another computer system due to variations in the internal variants in the internal representation which varies from computer to computer.
- It store data in binary format i.e: with the help of 0 and 1
- These files are not easily readable and modifiable because the content written in binary files is not human-readable and it is encrypted content.
- ios::binary has to be used with binary files while opening them.
- Error in a binary file corrupts the file and is not easily detected.

ASCII file

- The text file can easily to be transferred from one computer to another.
- It stores data using ASCII(format, i.e: human readable , graphics characters.
- These files are easily readable and modifiable because the content written in textfiles in human readable.
-

Templates

- A template is a technique that allows a **single function (or) class** to work with different data types.
- Using a template , we can create a single **function** that can process any type of data; that is , the **formal arguments** of a template function are of template(generic) Type.
- They can accept data of any Data type , such as **int, double , long , char**.
- Thus a single function can be used to accept values of a different data type.
- Between the **non-template function and template function** , usually we overload a function when we need to handle different data types.
- This approach increases the program size.
- The disadvantage are that not only the program length is increased but also more local variables are created in the memory.
- A template safely overcomes all the limitations occurring during the overloading function and allows better flexibility to the program.

Definition of class templates

In order to declare a class of templates type

Template Declaration

```
template < class T>
```

```
class name_of_class
```

```
{
```

```
// class member data & member function
```

```
};
```

- The first statement `template <class T>` tell the compiler that the following class declaration can use the template data type.
- The **"T"** is a variable of template type that can be used in the class to define a variable of template type.
- Both **template** and **class** are keyword
- The **<>** (angle bracket) is used to declare the variable of template type that can be used inside the class to define the variables of template types.

- **templates cannot be declared inside classes or functions . They should be global and should not be local.**

Write a program for function overloading without template

```
#include<iostream.h>

#include<conio.h>

void sum(int a,int b)

{

cout<<"\n Addition of two nos:"<<(a+b);

}

void sum(double x,double y)

{

cout<<"\n Addition of two nos : "<<(x+y);

}

void main()

{

clrscr();

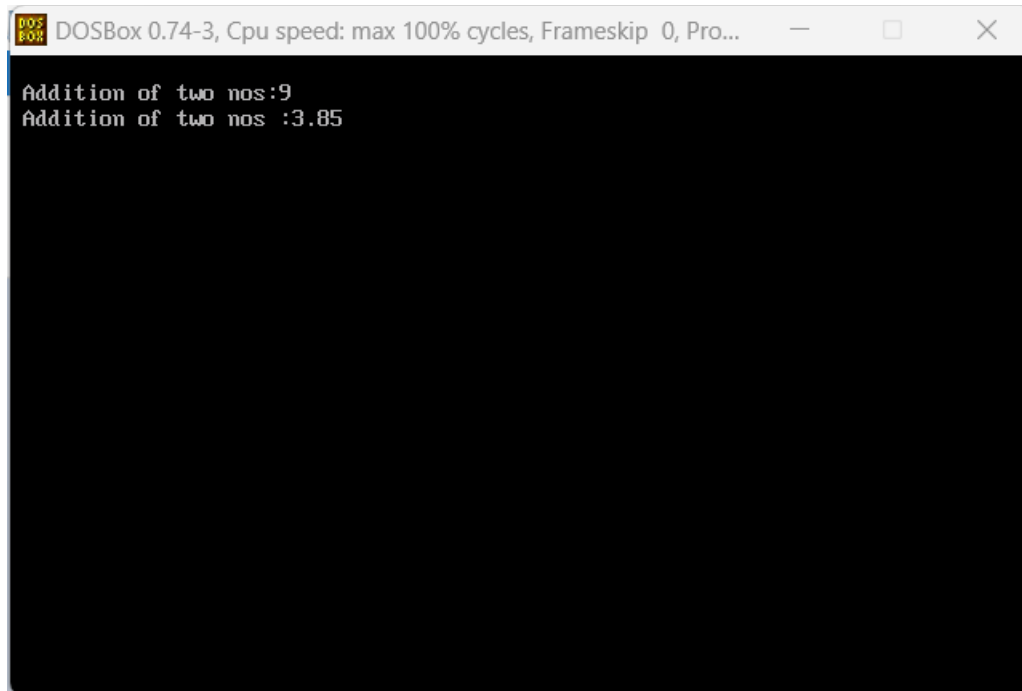
sum(5,4);

sum(1.35,2.50);

getch();

}
```

OUTPUT

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...". The window contains a black terminal area with white text. The text shows the output of a C++ program: "Addition of two nos:9" on the first line and "Addition of two nos :3.85" on the second line.

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Addition of two nos:9
Addition of two nos :3.85
```

In above program we have created two functions for adding different data types (or) different values , we can decrease that function overloading with the help of template class like as following

Write a program for function overloading using template class

```
#include<iostream.h>

#include<conio.h>

template <class T>

void sum(T a,T b)

{

cout<<"\n Addition of two nos:"<<(a+b);

}

void main()

{

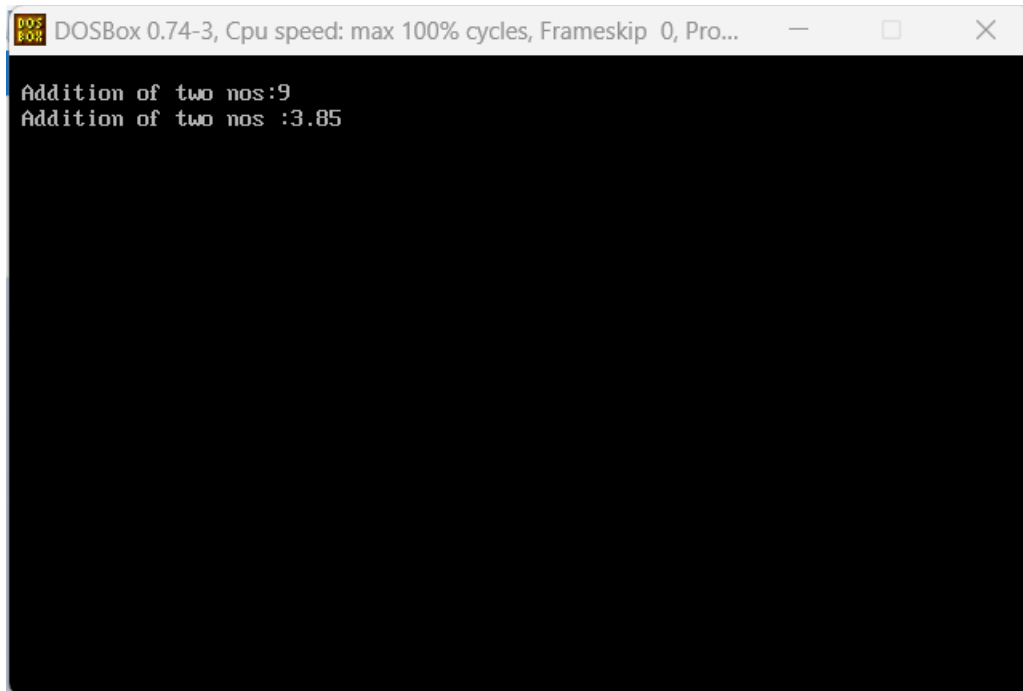
clrscr();

sum(5,4);

sum(1.35,2.50);

getch();

}
```



Exception handling

- While writing large programs , a programmer usually makes many mistakes.
- Due to this , bugs occur even in the released softwares.
- Developing an **error-free** program is the only objective and intention of the programmer.
- Programmer should take care to prevent errors.
- Error can be trapped using **exception-Handling** features.
- A few programming languages support **exception-handling** features . without this feature , the programmers should find out errors on their own.
- The error may be **logical errors (or) syntax error**.
- The **logical error remains** in the program due to a poor **understanding** of the program
- The **Syntax Error** made are due to a lack of understanding of the programming language
- C++ provides an **Exception-Handling** procedure to **reduce the errors** that the programmer makes.
- The programmer always faces unusual errors while writing programs.
- An exception is an **abnormal** termination of the program, which is executed in a program at **run time or it may be called at run time when the error occurs**.
- The **exception contains warning messages** such as
- **invalid argument , insufficient memory , and division by zero**

THE KEYWORDS try , throw & catch

- The exception - handling technique passes the control of a program from a location of exception in a program to an exception-handler routine linked with the **try** block .
- An exception - handler routine can only be called by the **throw** statement.

try

- The **try** keyword is followed by a series of statements enclosed in curly braces .

Syntax :

```
try
{
```

```
statement-1;  
statement-2;  
}
```

throw

- The function of the **throw** statement is to send the exception found .
- The declaration of the **throw statement** is as follow:

Syntax:

```
throw (except);  
throw excep;  
throw // re-throwing of an exception
```

catch

- Similar to the **try** block , the **catch** block also contains a series of statements enclosed in curly braced.
- It also contains an argument of an exception type in parentheses.

syntax

```
try  
{  
statement-1  
statement-2  
}  
catch(argument)  
{  
statement-3; // Action to be taken  
}
```

- when an exception found , the **catch** block is executed.
- The catch statement contains an argument of exception type, and it is optional.

- When an argument is declared , the argument can be used in the **catch** block.
- After execution of the **catch** block the statements inside the blocks are executed.
- In case **no exception** is **caught** , the **catch** block is **ignored**, and if a mismatch is found , the program is terminated

Write a program without exception

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();

int x=5,y=0,c;

c=x/y;

cout<<"\n Division is:"<<c;

getch();

}
```

OUTOUT

division by zero

- In the above program when user divided any number by **zero** means the output will throw the error like as above , but as a user it will not understandable for the user
- A programmer can solve the error with the help of **exception handling**

Write the above program using Exception handling

```
#include<iostream.h>

#include<conio.h>

void main()

{

clrscr();
```

```

int x=5,y=0,c=0;

try
{
    if(y==0)
    {
        throw " Don't user Zero as a divisor";
    }

    else
    {
        c=x/y;

        cout<<"\n Division is:"<<c;
    }
}

catch(char* msg)
{
    cout<<msg;
}

getch();
}

```

- In above program if x=5 and y=0 means the **if statement become true**
- so **try** will identify the error and throw the message "**Dont use zero as a divisor**" with the help of **throw** keyword
- **catch- block** will receive that upcomming error from **try block** with the help of **string** argument variable(**msg**) and display the erro to the user.

String

- A string is nothing but a sequence of characters.
- String can contain small and capital letters, numbers and special characters
- A string is to be treated as an array of character with terminating `\0`
- Each element of a string occupies a byte in the memory.
- Every string is terminated by a **null** character.
- The last character of such a string is a null `\0` character, and the compiler identifies the null character during the execution of the program.

Example

`char country[6]="INDIA";` // declaring and initialization of an array country with 6 characters

Here, text "INDIA" is assigned to an array `country[6]`. The text is enclosed within **double quotation marks**.

I	N	D	I	A	'\0'
73	78	68	73	65	00

- In above each character occupies a single byte in the memory. At the end of the string, a null character is inserted by the compiler.
- The first row shows the elements of the string, and the second row shows their corresponding ASCII values.

String Library functions

1. `strlen()` --> determines the **length** of a string
2. `strcpy()` ---> copies a string from the source to the destination
3. `strcmp()` ---> compares the characters of two strings (Function discriminates between small, caps)
4. `strlwr()` --> converts uppercase of a string into lowercase
5. `strupr()` ---> converts lowercase of a string into uppercase

6. strcat() ----> Appends the source string to the destination string

7. strrev() ---> reverse all the characters of a string

Write a program for string functions

```
#include<iostream.h>

#include<conio.h>

#include<string.h>

void main()

{

clrscr();

char fn[30],ln[30],pwd[30],cpwd[30];

int len;

cout<<"\n Enter first name:";

cin>>fn;

cout<<"\n Enter last name:";

cin>>ln;

cout<<"\n Enter password:";

cin>>pwd;

strcpy(cpwd,pwd); // here password copied from pwd to cpwd

len=strlen(pwd);

cout<<"\n password length is:"<<len;

if(len>=8)

{

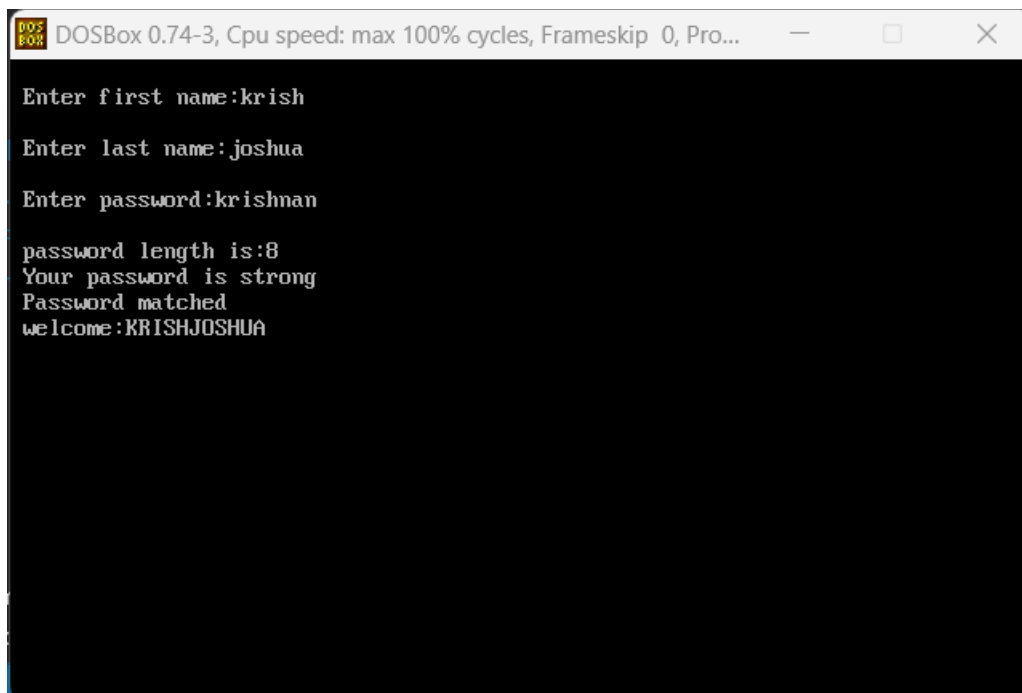
cout<<"\n Your password is strong";

// nested if else

if(strcmp(pwd,cpwd)==0)
```

```
{  
    cout<<"\n Password matched";  
    cout<<"\n welcome:"<<strupr(strcat(fn,ln));  
}  
  
else  
  
{  
    cout<<"\n Password not matched";  
}  
}  
  
else  
  
{  
    cout<<"\n Your password is weak";  
}  
  
getch();  
}
```

OUTPUT



DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...

```
Enter first name:krish
Enter last name:joshua
Enter password:krishnan
password length is:8
Your password is strong
Password matched
welcome:KRISHJOSHUA
```

String Declaring and initializing string objects

- The string object declaration and initialization can be done once by using a **constructor** of the string class.

Constructor	Meaning
<code>string();</code>	produces an empty string
<code>string(const char *text)</code>	produces a string object from a null-ended string
<code>string(const string & text)</code>	produces a string object from another string obj

we can declare and initialize string object as follows :

```
string text; // using constructors without arguments
```

```
string text("C++"); // using constructors with one argument
```

```
text1=text2; // Assignment of two string objects
```

```
text="C++" + text1 // concatenation of string objects
```

```
cin>>text          // reading string without spaces through the keyboard
```

```
getline(cin,text)  // reading string with blank page
```

Write a program for string objects

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
string text;
```

```
string text("C++");
```

```
string text2("OOP");
```

```
cout<<"\n text1:"<<text1;
```

```
cout<<"\n text2:"<<text2;

text=text1;

cout<<"\n text:"<<text;

text=text1+" " +text2;

cout<<"\n now text:"<<text;

getch();

}
```

OUTPUT

text1: C++

text2: OOP

text : C++

now text : C++ OOP

String Attributes

- The various attributes of a string such as
- **SIZE , length , and capacity, empty , find,substring,compare** can be obtained using member functions.

SIZE

- The size of the string indicates the total number of elements currently stored in the string.

capacity

- The capacity means the total numbers of elements that can be stored in a string.
- the maximums size means the largest valid size of the string supported by the system.

length

- The member function **length()** determines the length of the string object , that is , the number of characters present in the string.

empty

- The **empty** function determines whether the strin is empty or filled. If the string is empty , it returns 1 , otherwise it returns 0 .

find

- The **find** member funtion finds the given sub-string in the main string.

compare

The member function **compare** is used to compare two strings or sub-string. It returns 0 when the two strings are similar , otherwise , it returns a non-zero value

Write a program for string attributes

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
string s1;

s1="Hello";

s2="hello";

cout<<"\n Size of s1 is:"<<s1.size();

cout<<"\n capacity s1 is:"<<s1.capacity();

cout<<"\n S1 is empty:"<<s1.empty();

cout<<"\n find llo in hello:"<<s1.find("llo");

cout<<"\n compare string is:"<<s1.compare(s2);

getch();

}
```

OUTPUT

Size of s1 is: 6

capacity s1 is: 31

S1 is empty : 0

find llo in hello : 1

compare string is: false

Miscellaneous Functions

- **assign()** -- This function is used to assign a string wholly / partly to another string object .
- **beign()** -- This member function returns the references of the first character of the string

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
clrscr();
```

```
string s1(" C plus plus ");
```

```
string s2;
```

```
char *x;
```

```
x=s1.beign();
```

```
int x=0;
```

```
s2.assign(s1,0,6);
```

```
cout<<s2;
```

```
cout<<*x;
```

```
getch();
```

```
}
```

OUTPUT

C plus

C